WCH-GUI-Live 库参考手册

版本：1.0

https://wch.cn

1 概述

wvui 是面向 WCH 芯片的 GUI 组件库，采用“控件对象树 + 脏矩形 + 分块刷屏（PFB）”渲染模型。该库由核心模块与内置控件两部分组成：核心模块负责驱动、内存、渲染、页面、输入、动画与命令通道；内置控件以类回调的形式挂载到对象树上，并各自提供属性读写接口。

层次	模块	职责
应用层	wvui_app、页面描述数组	组织页面、创建控件、响应回调
控件层	widgets/*	绘制自身、处理事件、暴露属性接口
框架层	widget 、 layer 、 page 、 input、anim、cmd、dirty	对象树、层级与模态、页面栈、输入状态机、动画、命令通道、脏矩形
绘制层	render 、 draw 、 font 、 asset	切片上下文、抗锯齿图元、字体、资源解码
移植层	driver、port_template	屏幕分块推送、触摸采样

2 API 使用说明

2.1 命名与返回值约定

约定	说明
控件接口首参	控件 API 的第一个参数 w 为控件指针；当 w 为 NULL 或类型不匹配时，该调用为空操作，读取类接口返回 0。
布尔返回值	返回 uint8_t 的接口统一约定为：1 表示成立或成功，0 表示不成立、失败或类型不匹配。
无返回值接口	属性设置类接口返回 void，由内部完成参数裁剪与标脏，不返回错误码。
句柄返回值	返回 wvui_widget_t * 的接口在失败时（类型被裁剪、内存不足、未初始化）返回 NULL。
文本指针	const char *text 不做拷贝，仅保存指针，因此所指内容必须常驻；若其生命周期短于控件，将导致显示异常。

2.2 类型与单位

类型/单位	说明
wvui_rect_t	闭区间矩形 {x0, y0, x1, y1}，采用 int16 屏幕绝对坐标，端点包含在内。
wvui_color_t	像素颜色，其位宽随 WVUI_CFG_COLOR_DEPTH 为 16 位（RGB565）或 32 位。
opa	不透明度，取值 0..255；0 表示全透明（不绘制），255 表示不透明。
角度	512 步圆周，按顺时针方向递增，0 指向 3 点钟方向。
frames	帧数，帧率由应用主循环决定。

2.3 初始化流程

ARENA 模式下，先调用 wvui_widget_bind_arena() 绑定常驻区与临时区；
调用 wvui_init() 校验端口能力，并绑定屏幕参数；
可选调用 wvui_driver_bind_input() 绑定输入端口；
调用 wvui_page_reset() 清空页面栈，并构建首页；
在主循环中每帧调用 wvui_frame_poll()。

3 API 参考手册

3.1 wvui 系统初始化 API

3.1.1 wvui_init(const wvui_port_t *port, const wvui_cfg_t *cfg)

```
int wvui_init(const wvui_port_t *port, const wvui_cfg_t *cfg);
```

功能描述	初始化并校验端口能力，绑定屏幕尺寸、对齐粒度与帧缓冲配置，由内部完成脏矩形与渲染上下文的初始化。
输入参数	port：屏幕端口结构体，其五个回调均须全部实现；cfg：运行配置，传入 NULL 时使用默认配置。
输出参数	无
返回值	0 表示初始化成功，负值表示端口能力校验失败。

使用示例：

```
// 初始化 wvui，cfg 中包含屏幕尺寸与缓冲行数
wvui_init(&port, &cfg);
```

注意事项：

必须在调用其他 wvui 函数之前调用本函数；

port 中的回调不得为 NULL，否则返回失败；

ARENA 模式应在创建控件之前绑定内存区。

3.1.2 wvui_deinit(void)

```
void wvui_deinit(void);
```

功能描述	解绑显示端口、输入端口与全部内部状态。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_deinit();
```

注意事项：

调用后所有控件句柄均失效，如需再次使用须重新初始化。

3.1.3 wvui_ready(void)

```
uint8_t wvui_ready(void);
```

功能描述	查询驱动是否已完成初始化。
输入参数	无
输出参数	无

返回值	1 表示已就绪，0 表示未初始化。

使用示例：

```
if (wvui_ready()) { wvui_frame_poll(); }
```

3.1.4 wvui_push_rect(wvui_rect_t r, wvui_color_t c)

```
void wvui_push_rect(wvui_rect_t r, wvui_color_t c);
```

功能描述	立即向屏幕推送一个实心矩形，其流程依次为裁剪、按端口对齐扩展、分块与端口事务。
输入参数	r：目标矩形（屏幕坐标，闭区间）；c：填充颜色。
输出参数	无
返回值	无

使用示例：

```
wvui_push_rect((wvui_rect_t){0, 0, 239, 319}, WVUI_COLOR_WHITE);
```

注意事项：

该接口绕过脏矩形与控件树直接刷屏，仅建议用于上电清屏。

3.1.5 wvui_push_screen(wvui_color_t c)

```
void wvui_push_screen(wvui_color_t c);
```

功能描述	用单一颜色推送整屏。
输入参数	c：填充颜色。
输出参数	无
返回值	无

使用示例：

```
wvui_push_screen(WVUI_COLOR_BLACK);
```

3.1.6 wvui_screen_size(int16_t *w, int16_t *h)

```
void wvui_screen_size(int16_t *w, int16_t *h);
```

功能描述	读取初始化时确定的屏幕宽高。
输入参数	w：宽度输出指针，可为 NULL；h：高度输出指针，可为 NULL。
输出参数	w：屏幕宽度像素；h：屏幕高度像素。
返回值	无

使用示例：

```
int16_t sw, sh;
wvui_screen_size(&sw, &sh);
```

3.1.7 wvui_stats(void)

```
const wvui_stats_t *wvui_stats(void);
```

功能描述	获取只读的渲染统计信息，包括事务数、像素数与切片数等。
输入参数	无
输出参数	无
返回值	指向统计结构体的常量指针，未初始化时返回 NULL。

使用示例：

```
const wvui_stats_t *st = wvui_stats();
```

3.1.8 wvui_stats_reset(void)

```
void wvui_stats_reset(void);
```

功能描述	清零全部渲染统计计数，并复位虚拟时钟。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_stats_reset();
```

3.1.9 wvui_info(void)

```
const wvui_info_t *wvui_info(void);
```

功能描述	获取只读的有效配置，包括屏幕尺寸、缓冲行高与颜色深度。
输入参数	无
输出参数	无
返回值	指向配置结构体的常量指针。

使用示例：

```
const wvui_info_t *info = wvui_info();
```

3.1.10 wvui_driver_bind_arena(wvui_arena_t *persistent, wvui_arena_t *transient)

```
void wvui_driver_bind_arena(wvui_arena_t *persistent, wvui_arena_t *transient);
```

功能描述	绑定对象内存区，透传给内存后端。
输入参数	persistent：常驻区，可为 NULL；transient：临时区，可为 NULL。
输出参数	无
返回值	无

使用示例：

```
wvui_driver_bind_arena(&perm, &trans);
```

注意事项：

应在 wvui_init() 之后、创建控件之前调用。

3.1.11 wvui_driver_bind_input(const wvui_input_port_t *port)

```
void wvui_driver_bind_input(const wvui_input_port_t *port);
```

功能描述	绑定输入端口，帧循环将自动从该端口采样。
输入参数	port：输入端口；传入 NULL 表示由应用每帧显式传入采样。
输出参数	无
返回值	无

使用示例：

```
wvui_driver_bind_input(&in_port);
```

3.1.12 wvui_invalidate(wvui_rect_t r)

```
void wvui_invalidate(wvui_rect_t r);
```

功能描述	将矩形标记为脏，依次执行裁剪、对齐扩展与代价合并，随后加入脏矩形表。
输入参数	r：需要重绘的矩形。
输出参数	无
返回值	无

使用示例：

```
wvui_invalidate((wvui_rect_t){0, 0, 120, 60});
```

3.1.13 wvui_invalidate_clear(void)

```
void wvui_invalidate_clear(void);
```

功能描述	丢弃全部待处理的脏区。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_invalidate_clear();
```

3.1.14 wvui_build(const wvui_widget_desc_t *descs, uint16_t count, uint8_t persistent)

```
uint16_t wvui_build(const wvui_widget_desc_t *descs, uint16_t count, uint8_t persistent);
```

功能描述	按描述数组批量创建控件，数组顺序即为 z 序。
输入参数	descs：控件描述数组，驻留于 Flash；count：描述数量；persistent：非 0 时使用常驻内存区。
输出参数	无
返回值	成功创建的控件数量，可能小于 count。

使用示例：

```
uint16_t n = wvui_build(page_desc, DESC_COUNT, 0);
```

注意事项：

类型被裁剪或内存不足时，对应控件将被跳过且不报错；

返回数量可用于判断页面是否完整构建。

3.1.15 wvui_frame(const wvui_indev_t *in)

```
void wvui_frame(const wvui_indev_t *in);
```

功能描述	运行一帧，依次执行页面切换、输入处理、命令解析、页面 tick、动画推进与脏矩形刷屏。
输入参数	in：本帧输入采样，可为 NULL，表示本帧无输入。
输出参数	无
返回值	无

使用示例：

```
wvui_indev_t in = {0};
wvui_frame(&in);
```

3.1.16 wvui_frame_poll(void)

```
void wvui_frame_poll(void);
```

功能描述	运行一帧，输入从已绑定的输入端口采样。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
while (1) { wvui_frame_poll(); }
```

注意事项：

未绑定输入端口时，等价于以 NULL 输入运行一帧。

3.1.17 wvui_render_hold(void)

```
void wvui_render_hold(void);
```

功能描述	挂起刷屏，嵌套计数加一，用于批量更新时避免中间状态刷屏。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_render_hold();
wvui_btn_set_text(btn, "OK");
wvui_render_release();
```

注意事项：

hold 与 release 必须成对调用。

3.1.18 wvui_render_release(void)

```
void wvui_render_release(void);
```

功能描述	挂起计数减一，计数归零时恢复刷屏。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_render_release();
```

3.1.19 wvui_render_commit(void)

```
void wvui_render_commit(void);
```

功能描述	清空全部挂起层级，并立即消费待处理的脏区。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_render_hold();
/* 批量修改多个控件 */
wvui_render_commit();
```

3.1.20 wvui_render_held(void)

```
uint8_t wvui_render_held(void);
```

功能描述	查询当前是否处于刷屏挂起状态。
输入参数	无
输出参数	无
返回值	1 表示挂起中，0 表示正常。

使用示例：

```
if (!wvui_render_held()) { wvui_invalidate(rect); }
```

3.1.21 wvui_render_batch(wvui_update_fn fn, void *user)

```
void wvui_render_batch(wvui_update_fn fn, void *user);
```

功能描述	批量更新的封装接口：依次执行 hold、调用 fn(user) 与 commit。
输入参数	fn：更新回调；user：透传给该回调的参数。
输出参数	无
返回值	无

使用示例：

```
static void update(void *u) { wvui_label_set_text(u, "Done"); }
wvui_render_batch(update, label);
```

3.1.22 wvui_transient_reset(void)

```
void wvui_transient_reset(void);
```

功能描述	复位临时内存区，并按固定顺序重建无状态场景。
输入参数	无

输出参数	无
返回值	无

使用示例：

```
wvui_transient_reset();
```

3.2 wvui 内存管理 API

3.2.1 wvui_mem_bind(wvui_arena_t *persistent, wvui_arena_t *transient)

```
void wvui_mem_bind(wvui_arena_t *persistent, wvui_arena_t *transient);
```

功能描述	绑定常驻区与临时区；在 STATIC/HEAP 模式下为空操作。
输入参数	persistent：常驻内存区，可为 NULL；transient：临时内存区，可为 NULL。
输出参数	无
返回值	无

使用示例：

```
static wvui_arena_t pers, trans;
wvui_mem_bind(&pers, &trans);
```

注意事项：

两块 arena 的内存均由调用方提供，通常使用静态数组，绑定后所有分配均从中获取。

3.2.2 wvui_mem_alloc(uint32_t nbytes, uint8_t persistent)

```
void *wvui_mem_alloc(uint32_t nbytes, uint8_t persistent);
```

功能描述	按当前内存模式分配一个控件块。
输入参数	nbytes：申请字节数；persistent：非 0 时取自常驻区，为 0 时取自临时区。
输出参数	无
返回值	成功返回内存指针，失败返回 NULL。

使用示例：

```
void *buf = wvui_mem_alloc(256, 1);
```

注意事项：

第二参数非 0 时从持久区分配，适合控件等长期对象；传 0 时从临时区分配，可在帧末整体回收。

空间不足时返回 NULL，且失败次数会被统计，可用于运行期监控。

3.2.3 wvui_mem_free(void *p, uint32_t nbytes)

```
void wvui_mem_free(void *p, uint32_t nbytes);
```

功能描述	归还控件块；ARENA 模式下不回收，参数被忽略。

输入参数	p：内存指针；nbytes：块大小。
输出参数	无
返回值	无

使用示例：

```
wvui_mem_free(buf, 256);
```

注意事项：

arena 为线性分配器，归还时须传入与分配时相同的字节数。

释放顺序应与分配顺序相反，否则将造成碎片或断言失败。

3.2.4 wvui_mem_transient_release(void)

```
void wvui_mem_transient_release(void);
```

功能描述	将临时区游标归零，整体回收。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_mem_transient_release();
```

注意事项：

一次性回收临时区的全部内存，通常在每帧结束或界面重建后调用。

回收后仍持有临时区指针的对象将失效，须一并作废。

3.2.5 wvui_mem_transient_arena(void)

```
wvui_arena_t *wvui_mem_transient_arena(void);
```

功能描述	获取临时区结构指针。
输入参数	无
输出参数	无
返回值	区指针；在 STATIC/HEAP 模式下恒为 NULL。

使用示例：

```
wvui_arena_t *ta = wvui_mem_transient_arena();
```

注意事项：

返回临时区句柄，可直接配合 arena 系列接口进行容量统计或整体复位。

未绑定时返回 NULL。

3.2.6 wvui_mem_can_alloc(void)

```
uint8_t wvui_mem_can_alloc(void);
```

功能描述	查询当前模式是否还能分配新对象。
输入参数	无
输出参数	无
返回值	1 表示可分配，0 表示不可分配。

使用示例：

```
if (wvui_mem_can_alloc()) {
    void *p = wvui_mem_alloc(64, 1);
}
```

注意事项：

用于分配前预判，避免污染分配失败计数。

该结果仅为调用时刻的快照，在多任务环境下不能作为最终保证。

3.2.7 wvui_mem_used(void)

```
uint32_t wvui_mem_used(void);
```

功能描述	获取控件当前占用字节数。
输入参数	无
输出参数	无
返回值	当前占用字节数。

使用示例：

```
uint32_t u = wvui_mem_used();
```

注意事项：

为持久区与临时区的合计已用字节数，可配合 peak 评估内存余量。

该值随分配与释放实时变化；只读访问不影响统计。

3.2.8 wvui_mem_peak(void)

```
uint32_t wvui_mem_peak(void);
```

功能描述	获取历史峰值占用字节数。
输入参数	无
输出参数	无
返回值	历史峰值字节数。

使用示例：

```
uint32_t pk = wvui_mem_peak();
```

注意事项：

记录运行以来的最大占用，可用于反推静态内存池应申请的上限。

峰值只增不减，释放内存不会使该值回落。

3.2.9 wvui_mem_fails(void)

```
uint32_t wvui_mem_fails(void);
```

功能描述	获取累计分配失败次数。
输入参数	无
输出参数	无
返回值	失败次数。

使用示例：

```
uint32_t f = wvui_mem_fails();
```

注意事项：

累计分配失败次数；界面出现空白或控件缺失时，应优先查看该值。

仅在空间不足时累加，参数非法不计入。

3.2.10 wvui_arena_init(wvui_arena_t *a, void *mem, uint32_t size_bytes)

```
void wvui_arena_init(wvui_arena_t *a, void *mem, uint32_t size_bytes);
```

功能描述	将一块静态内存绑定到区，并清零游标、峰值与失败计数。
输入参数	a：区对象；mem：内存首地址；size_bytes：内存容量。
输出参数	无
返回值	无

使用示例：

```
static uint8_t pool[4096];
wvui_arena_init(&pers, pool, sizeof(pool));
```

注意事项：

内存块由调用方提供，其生命周期必须覆盖 arena 的使用期，一般使用静态数组。

起始地址建议按 4 字节对齐，以避免访问未对齐内存。

3.2.11 wvui_arena_alloc(wvui_arena_t *a, uint32_t nbytes)

```
void *wvui_arena_alloc(wvui_arena_t *a, uint32_t nbytes);
```

功能描述	从游标处按对齐规则切出一块内存。
输入参数	a：区对象；nbytes：申请字节数。

输出参数	无
返回值	成功返回指针，空间不足返回 NULL。

使用示例：

```
void *p = wvui_arena_alloc(&pers, 128);
```

注意事项：

线性分配，仅支持整体复位回收，不产生中间空洞。

空间不足时返回 NULL，并累加该 arena 的失败计数。

3.2.12 wvui_arena_reset(wvui_arena_t *a)

```
void wvui_arena_reset(wvui_arena_t *a);
```

功能描述	游标归零，整体回收该区内存。
输入参数	a：区对象。
输出参数	无
返回值	无

使用示例：

```
wvui_arena_reset(&pers);
```

注意事项：

一次性回收全部内存，此前返回的指针全部失效，须同步作废外部引用。

用于界面重建或单元测试复位，不适合单个对象的释放。

3.2.13 wvui_arena_used(const wvui_arena_t *a)

```
uint32_t wvui_arena_used(const wvui_arena_t *a);
```

功能描述	获取该区当前已用字节数。
输入参数	a：区对象。
输出参数	无
返回值	已用字节数。

使用示例：

```
uint32_t u = wvui_arena_used(&pers);
```

注意事项：

仅统计该 arena 的已用字节数，与 wvui_mem_used 的合计值相区分。

只读接口，不改变任何统计状态。

3.2.14 wvui_arena_peak(const wvui_arena_t *a)

```
uint32_t wvui_arena_peak(const wvui_arena_t *a);
```

功能描述	获取该区历史峰值字节数。
输入参数	a：区对象。
输出参数	无
返回值	峰值字节数。

使用示例：

```
uint32_t pk = wvui_arena_peak(&pers);
```

注意事项：

该池的历史最大占用，用于判断单池尺寸是否足够。

峰值只增不减，复位不会使其回落。

3.2.15 wvui_arena_fails(const wvui_arena_t *a)

```
uint32_t wvui_arena_fails(const wvui_arena_t *a);
```

功能描述	获取该区累计分配失败次数。
输入参数	a：区对象。
输出参数	无
返回值	失败次数。

使用示例：

```
uint32_t f = wvui_arena_fails(&pers);
```

注意事项：

该池的失败累计次数，便于定位是哪一块内存池不足。

只读接口，如需清零须重新初始化该 arena。

3.2.16 wvui_arena_assert_peak(const wvui_arena_t *a, uint32_t budget_bytes)

```
uint8_t wvui_arena_assert_peak(const wvui_arena_t *a, uint32_t budget_bytes);
```

功能描述	检查历史峰值是否仍在预算内，可用于启动自检。
输入参数	a：区对象；budget_bytes：预算上限。
输出参数	无
返回值	1 表示未超预算，0 表示超出。

使用示例：

```
uint8_t ok = wvui_arena_assert_peak(&pers, 3072);
```

注意事项：

峰值未超预算时返回 1，超出时返回 0，适合用于自检与回归测试。

仅比较预算而不执行任何回收动作，可在启动流程末尾调用一次。

3.3 wvui 脏矩形 API

3.3.1 wvui_dirty_bind(int16_t screen_w, int16_t screen_h, const wvui_port_t *port)

```
void wvui_dirty_bind(int16_t screen_w, int16_t screen_h, const wvui_port_t *port);
```

功能描述	绑定屏幕尺寸与端口对齐粒度，由 wvui_init() 调用一次。
输入参数	screen_w：屏幕宽度；screen_h：屏幕高度；port：用于获取对齐粒度的端口。
输出参数	无
返回值	无

使用示例：

```
wvui_dirty_bind(240, 240, &port);
```

注意事项：

屏幕尺寸用于将脏区裁剪到可视范围，端口则提供最终刷新回调。

应在渲染初始化与建界面之前完成绑定。

3.3.2 wvui_dirty_reset(void)

```
void wvui_dirty_reset(void);
```

功能描述	清空脏区列表与本帧统计，应在帧内消费完成后调用。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_dirty_reset();
```

注意事项：

清空全部脏区，通常在每帧刷新提交完成后调用。

只清列表不重绘，需要在下一帧重新标脏。

3.3.3 wvui_dirty_add(wvui_rect_t r)

```
void wvui_dirty_add(wvui_rect_t r);
```

功能描述	加入一个脏矩形，槽位超限时按代价模型进行合并。
输入参数	r：脏矩形。
输出参数	无
返回值	无

使用示例：

```
wvui_dirty_add(rect);
```

注意事项：

新区域将与已有区域自动合并，以减少刷新次数。

脏区表满时将强制合并整块，合并次数可在 merges 统计中查看。

3.3.4 wvui_dirty_set_fixed_cost_pixels(uint32_t px)

```
void wvui_dirty_set_fixed_cost_pixels(uint32_t px);
```

功能描述	设置代价模型参数，即省下一次事务等效的像素数；该数值越大，越倾向于合并。
输入参数	px：等效像素数。
输出参数	无
返回值	无

使用示例：

```
wvui_dirty_set_fixed_cost_pixels(400);
```

注意事项：

表示一次刷新操作的固定开销，单位为等价像素数，用于在分块刷新与整屏刷新之间取舍。

取值应结合屏接口的命令开销与总线速度估算；设得过大容易退化为整屏刷新。

3.3.5 wvui_dirty_fixed_cost_pixels(void)

```
uint32_t wvui_dirty_fixed_cost_pixels(void);
```

功能描述	读取当前代价模型参数。
输入参数	无
输出参数	无
返回值	等效像素数。

使用示例：

```
uint32_t px = wvui_dirty_fixed_cost_pixels();
```

注意事项：

只读回读当前阈值，不改变刷新策略。

未设置时返回内部默认值。

3.3.6 wvui_dirty_count(void)

```
uint8_t wvui_dirty_count(void);
```

功能描述	获取本帧脏区数量。
输入参数	无

输出参数	无
返回值	脏区数量。

使用示例：

```
uint8_t n = wvui_dirty_count();
```

注意事项：

当前脏区条目数，最大条数由内部表容量决定。

返回 0 表示无需刷新，可跳过本帧的提交流程。

3.3.7 wvui_dirty_get(uint8_t index)

```
wvui_rect_t wvui_dirty_get(uint8_t index);
```

功能描述	获取第 index 个脏区。
输入参数	index：脏区索引，从 0 开始。
输出参数	无
返回值	对应矩形；越界返回空矩形。

使用示例：

```
wvui_rect_t r = wvui_dirty_get(0);
```

注意事项：

索引越界时返回空矩形，调用前应先用 count 判断范围。

条目顺序会因合并而变化，不应假定其与标脏顺序一致。

3.3.8 wvui_dirty_total_area(void)

```
uint32_t wvui_dirty_total_area(void);
```

功能描述	获取本帧脏区总面积（未去重）。
输入参数	无
输出参数	无
返回值	像素总数。

使用示例：

```
uint32_t area = wvui_dirty_total_area();
```

注意事项：

所有脏区面积之和（单位为像素），用于估算本帧的刷新耗时。

各区域可能互相重叠，该值为各项面积累加值，而非并集面积。

3.3.9 wvui_dirty_merges(void)

```
uint32_t wvui_dirty_merges(void);
```

功能描述	获取本帧按代价模型合并的次数。
输入参数	无
输出参数	无
返回值	合并次数。

使用示例：

```
uint32_t m = wvui_dirty_merges();
```

注意事项：

合并发生次数；该值偏高说明标脏过于零碎，可考虑合并控件重绘。

累计值，只增不减。

3.3.10 wvui_dirty_forced_merges(void)

```
uint32_t wvui_dirty_forced_merges(void);
```

功能描述	获取本帧因槽位占满而强制合并的次数。
输入参数	无
输出参数	无
返回值	强制合并次数。

使用示例：

```
uint32_t fm = wvui_dirty_forced_merges();
```

注意事项：

因脏区表满而被强制合并的次数；该值持续增长说明单帧标脏块过多。

可据此调小合并阈值或减少零碎重绘，以避免退化为整屏刷新。

3.4 wvui 渲染 API

3.4.1 wvui_render_init(int16_t screen_w, int16_t screen_h)

```
void wvui_render_init(int16_t screen_w, int16_t screen_h);
```

功能描述	初始化切片渲染上下文。
输入参数	screen_w：屏幕宽度；screen_h：屏幕高度。
输出参数	无
返回值	无

使用示例：

```
wvui_render_init(240, 240);
```

注意事项：

尺寸必须与屏幕实际分辨率一致，否则裁剪与块划分均会出错。

应在 dirty_bind 之后、创建界面之前调用。

3.4.2 wvui_render_deinit(void)

```
void wvui_render_deinit(void);
```

功能描述	释放渲染上下文。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_render_deinit();
```

注意事项：

释放渲染内部资源；此后如需绘制，须重新调用 init。

调用前应确保没有进行中的块缓冲区。

3.4.3 wvui_render_begin_chunk(wvui_color_t *plane, uint32_t plane_pixels, wvui_rect_t chunk)

```
void wvui_render_begin_chunk(wvui_color_t *plane, uint32_t plane_pixels, wvui_rect_t chunk);
```

功能描述	开始一个切片：指定像素缓冲、可容纳像素数与切片矩形。
输入参数	plane：像素缓冲；plane_pixels：缓冲可容纳的像素数；chunk：本切片矩形。
输出参数	无
返回值	无

使用示例：

```
wvui_render_begin_chunk(plane, PLANE_PIXELS, chunk);
```

注意事项：

plane 为块缓冲区，其容量不得小于 chunk 的像素数。

块内绘制仍使用屏幕绝对坐标，超出块范围的像素由裁剪丢弃。

必须与 end_chunk 成对调用，其间不得嵌套调用另一个 begin。

3.4.4 wvui_render_end_chunk(void)

```
void wvui_render_end_chunk(void);
```

功能描述	结束当前切片并提交到端口事务。
输入参数	无
输出参数	无

返回值	无

使用示例：

```
wvui_render_end_chunk();
```

注意事项：

结束当前块，并将块内容交回端口刷新；必须与 begin_chunk 成对调用。

未开始块时调用为空操作。

3.4.5 wvui_render_in_chunk(void)

```
uint8_t wvui_render_in_chunk(void);
```

功能描述	查询是否处于切片绘制中。
输入参数	无
输出参数	无
返回值	1 表示在绘制中。

使用示例：

```
if (wvui_render_in_chunk()) { /* 处于块渲染上下文 */ }
```

注意事项：

未开始块时返回 0，可用于在绘制前判断上下文状态。

只查询状态，不改变块上下文。

3.4.6 wvui_render_set_clip(wvui_rect_t r)

```
void wvui_render_set_clip(wvui_rect_t r);
```

功能描述	设置裁剪矩形，与现有裁剪取交集。
输入参数	r：裁剪矩形。
输出参数	无
返回值	无

使用示例：

```
wvui_render_set_clip(r);
```

注意事项：

影响后续所有绘制调用，直至 clear_clip 或再次设置为止。

裁剪区不跨块边界，最终仍与当前块区域求交。

3.4.7 wvui_render_clear_clip(void)

```
void wvui_render_clear_clip(void);
```

功能描述	清除裁剪，恢复为切片范围。

输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_render_clear_clip();
```

注意事项：

取消裁剪，恢复为当前块的全部范围。

不支持嵌套保存；如需多层裁剪，应由调用方自行记录并重设。

3.4.8 wvui_render_clip(void)

```
wvui_rect_t wvui_render_clip(void);
```

功能描述	读取当前有效裁剪矩形。
输入参数	无
输出参数	无
返回值	当前裁剪矩形。

使用示例：

```
wvui_rect_t c = wvui_render_clip();
```

注意事项：

返回当前有效裁剪矩形，已与块区域求交。

只读接口，不改变裁剪状态。

3.4.9 wvui_render_fill_slice(wvui_color_t c)

```
void wvui_render_fill_slice(wvui_color_t c);
```

功能描述	用颜色填满当前切片。
输入参数	c：填充颜色。
输出参数	无
返回值	无

使用示例：

```
wvui_render_fill_slice(C_WHITE);
```

注意事项：

填充整个当前块，并受裁剪矩形约束，常用于块首清底。

必须在 begin_chunk 与 end_chunk 之间调用。

3.4.10 wvui_render_fill_rect(wvui_rect_t r, wvui_color_t c)

```
void wvui_render_fill_rect(wvui_rect_t r, wvui_color_t c);
```

功能描述	填充矩形，已按裁剪处理，不透明快路径。
输入参数	r：矩形；c：颜色。
输出参数	无
返回值	无

使用示例：

```
wvui_render_fill_rect(r, C_WHITE);
```

注意事项：

矩形完全落在裁剪区外时直接返回，不产生像素写入。

本接口不处理圆角；如需圆角，请改用 wvui_draw_round_rect。

3.4.11 wvui_render_rect_outline(wvui_rect_t r, wvui_color_t c)

```
void wvui_render_rect_outline(wvui_rect_t r, wvui_color_t c);
```

功能描述	绘制 1 像素矩形边框。
输入参数	r：矩形；c：颜色。
输出参数	无
返回值	无

使用示例：

```
wvui_render_rect_outline(r, C_GRAY);
```

注意事项：

只描边不填充，线宽固定为 1 像素。

需要多像素或圆角边框时，请改用 wvui_draw_round_rect 并指定 bw。

3.4.12 wvui_render_set_opa_scale(uint8_t scale)

```
void wvui_render_set_opa_scale(uint8_t scale);
```

功能描述	设置容器透明度乘子，影响随后绘制。
输入参数	scale：乘子 0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_render_set_opa_scale(128);
```

注意事项：

全局缩放会与控件自身不透明度相乘，用于实现整组淡入淡出或蒙层效果。

设为 0 时内容不可见，但仍参与绘制与布局，不会跳过绘制开销。

3.4.13 wvui_render_opa_scale(void)

```
uint8_t wvui_render_opa_scale(void);
```

功能描述	读取当前容器透明度乘子。
输入参数	无
输出参数	无
返回值	乘子 0..255。

使用示例：

```
uint8_t s = wvui_render_opa_scale();
```

注意事项：

回读当前全局不透明度缩放系数，默认值为 255。

只读接口，不改变渲染状态。

3.4.14 wvui_render_opa_apply(uint8_t opa)

```
uint8_t wvui_render_opa_apply(uint8_t opa);
```

功能描述	将控件透明度与容器乘子相乘，得到实际透明度。
输入参数	opa：控件透明度 0..255。
输出参数	无
返回值	实际透明度。

使用示例：

```
uint8_t a = wvui_render_opa_apply(128);
```

注意事项：

将控件自身不透明度与全局缩放相乘，得到实际值，用于统一计算。

纯计算接口，不改变任何渲染状态。

3.4.15 wvui_render_blend(wvui_color_t fg, wvui_color_t bg, uint8_t opa)

```
wvui_color_t wvui_render_blend(wvui_color_t fg, wvui_color_t bg, uint8_t opa);
```

功能描述	按透明度混合前景色与背景色。
输入参数	fg：前景色；bg：背景色；opa：透明度。
输出参数	无
返回值	混合后的颜色。

使用示例：

```
wvui_color_t c = wvui_render_blend(fg, bg, 128);
```

注意事项：

纯函数混色：opa 为 0 时返回背景色，为 255 时返回前景色。

不写入像素，仅返回计算结果，可用于预计算遮罩色。

3.4.16 wvui_render_fill_rect_blend(wvui_rect_t r, wvui_color_t c, uint8_t opa)

```
void wvui_render_fill_rect_blend(wvui_rect_t r, wvui_color_t c, uint8_t opa);
```

功能描述	半透明填充矩形。
输入参数	r：矩形；c：颜色；opa：透明度。
输出参数	无
返回值	无

使用示例：

```
wvui_render_fill_rect_blend(r, C_BLACK, 96);
```

注意事项：

半透明填充，常用作弹窗或对话框背后的蒙层。

逐像素混色，性能低于纯色填充；整屏蒙层可考虑用 opa_scale 一次性设置。

3.4.17 wvui_render_pixel(int16_t x, int16_t y, wvui_color_t c, uint8_t opa)

```
void wvui_render_pixel(int16_t x, int16_t y, wvui_color_t c, uint8_t opa);
```

功能描述	绘制单个像素；opa 为 255 时直接写入，否则与底色混合。
输入参数	x、y：屏幕坐标；c：颜色；opa：透明度。
输出参数	无
返回值	无

使用示例：

```
wvui_render_pixel(x, y, c, 255);
```

注意事项：

单像素写入，受当前裁剪矩形约束。

大量逐点绘制性能很差，应优先改用矩形或描边接口。

3.4.18 wvui_img_stats(void)

```
const wvui_img_stats_t *wvui_img_stats(void);
```

功能描述	获取图像解码统计，包括解码次数、跳过的资源数与错误数。
输入参数	无
输出参数	无

返回值	统计结构常量指针。

使用示例：

```
const wvui_img_stats_t *st = wvui_img_stats();
```

注意事项：

返回内部统计结构的地址，仅供只读访问，不得修改或释放。

内容随解码过程实时更新，跨帧长期保存意义不大。

3.4.19 wvui_img_stats_reset(void)

```
void wvui_img_stats_reset(void);
```

功能描述	清零图像解码统计。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_img_stats_reset();
```

注意事项：

清零解码统计，便于分段测量或对比优化前后的差异。

只复位计数器，不影响已解码的图片数据。

3.4.20 wvui_img_err_name(uint8_t err)

```
const char *wvui_img_err_name(uint8_t err);
```

功能描述	把图像错误码转换为可读字符串。
输入参数	err：错误码。
输出参数	无
返回值	字符串常量指针。

使用示例：

```
const char *n = wvui_img_err_name(err);
```

注意事项：

返回常量字符串，可直接用于日志输出，无需释放。

未知错误码返回通用名称，不能据此反推具体原因。

3.4.21 wvui_img_format_supported(uint8_t format)

```
uint8_t wvui_img_format_supported(uint8_t format);
```

功能描述	查询像素格式是否被当前构建支持。

输入参数	format：像素格式编号。
输出参数	无
返回值	1 表示支持。

使用示例：

```
uint8_t ok = wvui_img_format_supported(fmt);
```

注意事项：

用于加载前预判；不受支持的格式解码必然失败。

仅判断格式号，不校验数据长度或头部是否合法。

3.4.22 wvui_img_format_is_alpha(uint8_t format)

```
uint8_t wvui_img_format_is_alpha(uint8_t format);
```

功能描述	判断该格式是否为纯 alpha 掩膜，可用前景色重着色。
输入参数	format：像素格式编号。
输出参数	无
返回值	1 表示为 alpha 掩膜。

使用示例：

```
uint8_t is_alpha = wvui_img_format_is_alpha(fmt);
```

注意事项：

仅纯 alpha（遮罩）格式可被 wvui_img_set_color 重着色。

彩色格式返回 0，此时设置着色无效。

3.4.23 wvui_img_format_pixel_bytes(uint8_t format)

```
uint8_t wvui_img_format_pixel_bytes(uint8_t format);
```

功能描述	获取该格式每像素字节数。
输入参数	format：像素格式编号。
输出参数	无
返回值	字节数；不支持的格式返回 0。

使用示例：

```
uint8_t bpp = wvui_img_format_pixel_bytes(fmt);
```

注意事项：

用于估算解码后所需缓冲区：宽 × 高 × 每像素字节数。

未知格式返回 0，调用方需自行判断其合法性。

3.4.24 wvui_img_size(const wvui_blob_t *blob, int16_t *w, int16_t *h)

```
void wvui_img_size(const wvui_blob_t *blob, int16_t *w, int16_t *h);
```

功能描述	读取图像宽高。
输入参数	blob：已打开的资源；w：宽度输出，可为 NULL；h：高度输出，可为 NULL。
输出参数	w：图像宽度；h：图像高度。
返回值	无

使用示例：

```
int16_t w, h;
wvui_img_size(&blob, &w, &h);
```

注意事项：

仅解析文件头部以读取尺寸，不解码像素，开销很小。

blob 指向的数据在调用期间必须保持有效。

3.5 wvui 绘图 API

3.5.1 wvui_isqrt32(uint32_t v)

```
uint32_t wvui_isqrt32(uint32_t v);
```

功能描述	整数平方根运算，结果向下取整，用于距离与半径判定。
输入参数	v：被开方数。
输出参数	无
返回值	平方根整数部分。

使用示例：

```
uint32_t r = wvui_isqrt32(14400);
```

注意事项：

整数平方根，常用于半径与长度换算，结果为向下取整。

输入为 0 时返回 0，不会产生除零问题。

3.5.2 wvui_sin512(uint16_t a)

```
int16_t wvui_sin512(uint16_t a);
```

功能描述	正弦查表，512 步圆周，Q15 定点。
输入参数	a：角度 0..511。
输出参数	无
返回值	Q15 定点正弦值。

使用示例：

```
int16_t s = wvui_sin512(128);
```

注意事项：

角度以 512 步表示一整周，128 约相当于 90 度。

返回定点值而非浮点值，与半径相乘后需右移以还原量纲。

3.5.3 wvui_cos512(uint16_t a)

```
int16_t wvui_cos512(uint16_t a);
```

功能描述	余弦查表，512 步圆周，Q15 定点。
输入参数	a：角度 0..511。
输出参数	无
返回值	Q15 定点余弦值。

使用示例：

```
int16_t c = wvui_cos512(0);
```

注意事项：

与 sin512 同为 512 步圆周，二者相差 128 步（90 度）。

常用于按角度计算圆周坐标：x 用余弦、y 用正弦。

3.5.4 wvui_draw_line(int16_t x0, int16_t y0, int16_t x1, int16_t y1, uint8_t width, wvui_line_cap_t cap, wvui_color_t color, uint8_t opa)

```
void wvui_draw_line(int16_t x0, int16_t y0, int16_t x1, int16_t y1, uint8_t width, wvui_line_cap_t cap, wvui_color_t color, uint8_t opa);
```

功能描述	绘制抗锯齿线段，采用单遍胶囊覆盖计算，端点样式由 cap 决定。
输入参数	x0、y0：起点；x1、y1：终点；width：线宽，为 0 时不绘制；cap：端点样式；color：颜色；opa：透明度。
输出参数	无
返回值	无

使用示例：

```
wvui_draw_line(10, 10, 100, 10, 2, WVUI_LINE_CAP_ROUND, C_BLACK, 255);
```

注意事项：

端点为屏幕绝对坐标，绘制结果受当前裁剪矩形约束。

width 为 0 时不绘制；端点样式可选平头或圆头。

3.5.5 wvui_draw_round_rect(wvui_rect_t rect, uint8_t r, uint8_t bw, wvui_color_t fill, uint8_t fill_opa, wvui_color_t border, uint8_t border_opa)

```
void wvui_draw_round_rect(wvui_rect_t rect, uint8_t r, uint8_t bw, wvui_color_t fill, uint8_t fill_opa, wvui_color_t border, uint8_t border_opa);
```

功能描述	绘制统一圆角矩形，包含填充与可选边框，半径自动夹紧至 min(w,h)/2。
输入参数	rect：外接矩形；r：圆角半径；bw：边框宽度，0 表示无边框；fill：填充色；fill_opa：填充透明度；border：边框色；border_opa：边框透明度。
输出参数	无
返回值	无

使用示例：

```
wvui_draw_round_rect(r, 8, 1, C_WHITE, 255, C_GRAY, 255);
```

注意事项：

填充与边框各自带有独立的不透明度，可只填充（bw=0）或只描边（fill_opa=0）。

半径应小于矩形短边的一半，否则将被夹紧。

3.5.6 wvui_draw_box(wvui_rect_t rect, const uint8_t corners[4], uint8_t r, uint8_t bw, wvui_color_t fill, uint8_t fill_opa, wvui_color_t border, uint8_t border_opa)

```
void wvui_draw_box(wvui_rect_t rect, const uint8_t corners[4], uint8_t r, uint8_t bw, wvui_color_t fill, uint8_t fill_opa, wvui_color_t border, uint8_t border_opa);
```

功能描述	绘制四角样式可独立配置的盒子，四角可分别采用圆角、切角与直角。
输入参数	rect：外接矩形；corners：四角样式，按 WVUI_BOX_LT/RT/LB/RB 索引；r：圆角半径或切角长度；bw：边框宽度；fill、fill_opa：填充色与透明度；border、border_opa：边框色与透明度。
输出参数	无
返回值	无

使用示例：

```
static const uint8_t cs[4] = {1, 1, 1, 1};
wvui_draw_box(r, cs, 8, 1, C_WHITE, 255, C_GRAY, 255);
```

注意事项：

corners 为四角是否圆角的开关，可用于只圆上面两个角等非对称造型。

半径需小于矩形短边的一半，否则实际圆角将被夹紧变小。

3.5.7 wvui_draw_arc(int16_t cx, int16_t cy, uint16_t r_out, uint16_t r_in, uint16_t a0, uint16_t a1, wvui_color_t color, uint8_t opa)

```
void wvui_draw_arc(int16_t cx, int16_t cy, uint16_t r_out, uint16_t r_in, uint16_t a0, uint16_t a1, wvui_color_t color, uint8_t opa);
```

功能描述	绘制圆弧或环扇形，径向与角度判定均使用整数运算，无需逐点计算三角函数。
输入参数	cx、cy：圆心；r_out：外半径；r_in：内半径，0 表示实心饼图；a0：起始角；a1：结束角，可小于 a0；color：颜色；opa：透明度。

输出参数	无
返回值	无

注意事项：

角度判定使用叉积，其可靠性限于半圆范围；超过 180 度时应绘制补弧并取反；a1 等于 a0 表示整圆。

使用示例：

```
wvui_draw_arc(120, 120, 40, 32, 128, 384, C_BLUE, 255);
```

注意事项：

外半径与内半径之差即为环宽，r_in 应小于 r_out。

圆心与角度分别采用屏幕坐标与 512 步圆周；a0 等于 a1 时不绘制。

3.6 wvui 字体 API

3.6.1 wvui_font_glyph(const wvui_font_t *f, uint32_t cp)

```
const wvui_font_glyph_t *wvui_font_glyph(const wvui_font_t *f, uint32_t cp);
```

功能描述	按 Unicode 码点在字体表中查找字形描述。
输入参数	f：字体描述；cp：Unicode 码点。
输出参数	无
返回值	字形指针；字体为空或未收录该码点时返回 NULL。

使用示例：

```
const wvui_font_glyph_t *g = wvui_font_glyph(font, 0x5B57);
```

注意事项：

返回指针指向字体表内部，不得修改，其生命周期与字体描述一致。

3.6.2 wvui_utf8_next(const char **p)

```
uint32_t wvui_utf8_next(const char **p);
```

功能描述	解码一个 UTF-8 码点，并将游标推进到下一个字符。
输入参数	p：字符串游标指针的地址。
输出参数	*p：推进后的游标
返回值	解码得到的码点。

使用示例：

```
const char *s = "温度";
while (*s) { uint32_t cp = wvui_utf8_next(&s); }
```

注意事项：

遇到非法序列时按单字节处理，游标前进 1 字节，不会陷入死循环。

3.6.3 wvui_font_line_height(const wvui_font_t *f)

```
int16_t wvui_font_line_height(const wvui_font_t *f);
```

功能描述	获取相邻两行行顶之间的距离，即行高与行距之和。
输入参数	f：字体描述。
输出参数	无
返回值	行间距，单位像素。

使用示例：

```
int16_t pitch = wvui_font_line_height(font);
```

注意事项：

多行文本按行号乘以该值计算行顶，避免逐行累加误差。

3.6.4 wvui_font_measure(const wvui_font_t *f, const char *utf8, int16_t *w, int16_t *h)

```
void wvui_font_measure(const wvui_font_t *f, const char *utf8, int16_t *w, int16_t *h);
```

功能描述	测量文本的布局宽度与多行高度，支持按换行符分段。
输入参数	f：字体描述；utf8：UTF-8 文本；w、h：输出指针，可为 NULL。
输出参数	w：整体宽度；h：整体高度
返回值	无

使用示例：

```
int16_t tw, th;
wvui_font_measure(font, "TEMP", &tw, &th);
```

注意事项：

宽度取最长行，高度按行数乘行间距计算。

3.6.5 wvui_font_draw_text(const wvui_font_t *f, int16_t x, int16_t y, const char *utf8, wvui_color_t fg, uint8_t opa)

```
void wvui_font_draw_text(const wvui_font_t *f, int16_t x, int16_t y, const char *utf8, wvui_color_t fg, uint8_t opa);
```

功能描述	以左上角为基准绘制文本，仅写入当前切片范围内的像素。
输入参数	f：字体描述；x、y：首行左上角坐标；utf8：UTF-8 文本；fg：前景色；opa：透明度，取值 0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_font_draw_text(font, 10, 20, "Temp: 25C", WVUI_COLOR_WHITE, 255);
```

注意事项：

f 为 NULL、utf8 为 NULL 或 opa 为 0 时不绘制；

只能在切片绘制期间调用，否则可能越界写入。

3.6.6 wvui_font_line_width(const wvui_font_t *f, const char *utf8)

```
int16_t wvui_font_line_width(const wvui_font_t *f, const char *utf8);
```

功能描述	获取首行（换行符之前）的像素宽度。
输入参数	f：字体描述；utf8：UTF-8 文本。
输出参数	无
返回值	宽度像素值；参数为空返回 0。

使用示例：

```
int16_t w = wvui_font_line_width(font, "OK");
```

注意事项：

仅统计首个换行符之前的内容；多行宽度请使用 wvui_font_measure()。

3.6.7 wvui_font_prefix_width(const wvui_font_t *f, const char *utf8, uint16_t nbytes)

```
int16_t wvui_font_prefix_width(const wvui_font_t *f, const char *utf8, uint16_t nbytes);
```

功能描述	测量文本前 nbytes 个字节的布局宽度，用于跑马灯与逐字节排布。
输入参数	f：字体描述；utf8：UTF-8 文本；nbytes：字节数。
输出参数	无
返回值	宽度像素值。

使用示例：

```
int16_t w = wvui_font_prefix_width(font, "Temperature", 5);
```

注意事项：

参数为字节数而非字符数，中文等多字节字符需按整字对齐。

3.6.8 wvui_font_draw_prefix(const wvui_font_t *f, int16_t x, int16_t y, const char *utf8, uint16_t nbytes, wvui_color_t fg, uint8_t opa)

```
void wvui_font_draw_prefix(const wvui_font_t *f, int16_t x, int16_t y, const char *utf8, uint16_t nbytes, wvui_color_t fg, uint8_t opa);
```

功能描述	仅绘制文本的前 nbytes 个字节，换行语义与整串绘制一致。

输入参数	f：字体描述；x、y：首行左上角坐标；utf8：UTF-8 文本；nbytes：字节数；fg：前景色；opa：透明度。
输出参数	无
返回值	无

使用示例：

```
wvui_font_draw_prefix(font, 0, 0, text, 6, WVUI_COLOR_WHITE, 255);
```

注意事项：

nbytes 超过字符串长度时，按至 NUL 为止处理。

3.6.9 wvui_font_line_count(const char *utf8)

```
uint16_t wvui_font_line_count(const char *utf8);
```

功能描述	统计文本的行数。
输入参数	utf8：UTF-8 文本，可为 NULL。
输出参数	无
返回值	行数；NULL 与空串返回 0。

使用示例：

```
uint16_t n = wvui_font_line_count("A\nB");   /* 返回 2 */
```

注意事项：

末尾带换行符的字符串按多一行计算，如 "a\n" 返回 2。

3.6.10 wvui_font_line_equal(const char *a, const char *b, uint16_t index)

```
uint8_t wvui_font_line_equal(const char *a, const char *b, uint16_t index);
```

功能描述	逐字节比较两段文本的第 index 行是否相同。
输入参数	a、b：待比较的两段文本；index：行号，从 0 开始。
输出参数	无
返回值	1 表示相同，0 表示不同。

使用示例：

```
if (!wvui_font_line_equal(old, now, 1)) { /* 第 2 行变化，标脏该行 */ }
```

注意事项：

双方均缺少该行时视为相同，便于按行增量刷新。

3.6.11 wvui_font_line_rect(const wvui_font_t *f, const char *utf8, uint16_t index, int16_t x, int16_t width, int16_t line_top, wvui_rect_t *out)

```
uint8_t wvui_font_line_rect(const wvui_font_t *f, const char *utf8, uint16_t index, int16_t x, int16_t
width, int16_t line_top, wvui_rect_t *out);
```

功能描述	计算第 index 行的布局矩形，用于按行标记脏区。
输入参数	f：字体描述；utf8：文本；index：行号；x：文本块左边缘；width：文本块宽度；line_top：首行行顶坐标。
输出参数	out：该行的屏幕矩形
返回值	1 表示该行存在，0 表示该行不存在或超出坐标范围。

使用示例：

```
wvui_rect_t r;
if (wvui_font_line_rect(font, text, 2, x, w, y, &r)) { wvui_invalidate(r); }
```

注意事项：

行顶坐标按 line_top 加 index 与行间距之积计算，矩形高度固定为行高。

3.6.12 wvui_font_strip_line_height(const wvui_font_strip_t *s)

```
uint16_t wvui_font_strip_line_height(const wvui_font_strip_t *s);
```

功能描述	获取等宽位图字体的行高。
输入参数	s：位图字体描述。
输出参数	无
返回值	行高像素值；s 为 NULL 时返回 0。

使用示例：

```
uint16_t lh = wvui_font_strip_line_height(strip);
```

注意事项：

位图字体为等宽字格，行高与字高一致，不含额外行距。

3.6.13 wvui_font_strip_measure(const wvui_font_strip_t *s, const char *utf8, int16_t *w, int16_t *h)

```
void wvui_font_strip_measure(const wvui_font_strip_t *s, const char *utf8, int16_t *w, int16_t *h);
```

功能描述	测量等宽位图字体的文本宽高。
输入参数	s：位图字体描述；utf8：文本；w、h：输出指针，可为 NULL。
输出参数	w：宽度；h：高度
返回值	无

使用示例：

```
int16_t w, h;
wvui_font_strip_measure(strip, "12:00", &w, &h);
```

注意事项：

宽度按行内字形数与字符步进之积计算，高度按行数与行高之积计算。

3.6.14 wvui_font_strip_draw(const wvui_font_strip_t *s, int16_t x, int16_t y, const char *utf8, wvui_color_t fg, uint8_t opa)

```
void wvui_font_strip_draw(const wvui_font_strip_t *s, int16_t x, int16_t y, const char *utf8, wvui_color_t fg, uint8_t opa);
```

功能描述	以固定字格绘制文本，每个字形按资源 ID 逐个取图。
输入参数	s：位图字体描述；x、y：首行左上角坐标；utf8：文本；fg：前景色；opa：透明度。
输出参数	无
返回值	无

使用示例：

```
wvui_font_strip_draw(strip, 0, 0, "12:00", WVUI_COLOR_WHITE, 255);
```

注意事项：

字形资源缺失时跳过该字符且不推进游标；

仅 [first_char, last_char] 范围内的字符具有字形。

3.7 wvui 资源 API

3.7.1 wvui_asset_hdr_decode(const uint8_t *hdr, wvui_asset_hdr_t *out)

```
void wvui_asset_hdr_decode(const uint8_t *hdr, wvui_asset_hdr_t *out);
```

功能描述	解析资源自描述头，将宽、高、格式等字段按大端序还原。
输入参数	hdr：头部缓冲首地址；out：解析结果。
输出参数	out：填充后的头结构
返回值	无

使用示例：

```
wvui_asset_hdr_t hdr;
wvui_asset_hdr_decode(blob.hdr, &hdr);
```

注意事项：

调用前需保证头部已完整读入缓冲，本函数不做长度校验。

3.7.2 wvui_asset_hdr_check(const uint8_t *hdr, wvui_asset_id_t id, uint8_t *err)

```
uint8_t wvui_asset_hdr_check(const uint8_t *hdr, wvui_asset_id_t id, uint8_t *err);
```

功能描述	校验头部与资源 ID 是否自洽，包括魔数、版本与长度字段。
输入参数	hdr：头部缓冲；id：待校验的资源 ID；err：失败原因输出指针，可为 NULL。
输出参数	err：错误码

返回值	1 表示通过，0 表示校验失败。

使用示例：

```
uint8_t err = 0;
if (!wvui_asset_hdr_check(h, id, &err)) { /* wvui_asset_err_name(err) */ }
```

注意事项：

err 仅在失败时写入，可用于统计与日志。

3.7.3 wvui_asset_bind(const wvui_asset_backend_t *be)

```
uint8_t wvui_asset_bind(const wvui_asset_backend_t *be);
```

功能描述	绑定资源存储后端，可在运行期切换 Flash 与外部存储。
输入参数	be：后端描述，包含资源表与读回调。
输出参数	无
返回值	1 表示绑定成功，0 表示参数非法。

使用示例：

```
wvui_asset_bind(&flash_backend);
```

注意事项：

切换后端前应先关闭已打开的资源块。

3.7.4 wvui_asset_unbind(void)

```
void wvui_asset_unbind(void);
```

功能描述	解绑当前存储后端。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_asset_unbind();
```

注意事项：

解绑后所有资源读取接口返回失败。

3.7.5 wvui_asset_backend(void)

```
const wvui_asset_backend_t *wvui_asset_backend(void);
```

功能描述	获取当前绑定的后端描述。
输入参数	无

输出参数	无
返回值	后端指针；未绑定时返回 NULL。

使用示例：

```
const wvui_asset_backend_t *be = wvui_asset_backend();
```

3.7.6 wvui_asset_err_name(uint8_t err)

```
const char *wvui_asset_err_name(uint8_t err);
```

功能描述	将资源错误码转换为可读字符串，便于日志输出。
输入参数	err：错误码。
输出参数	无
返回值	字符串常量指针。

使用示例：

```
printf("asset error: %s\n", wvui_asset_err_name(err));
```

注意事项：

返回静态字符串，不可释放或修改。

3.7.7 wvui_asset_stats(void)

```
const wvui_asset_stats_t *wvui_asset_stats(void);
```

功能描述	获取资源解析统计，包括打开次数、失败次数与字节数。
输入参数	无
输出参数	无
返回值	统计结构常量指针。

使用示例：

```
const wvui_asset_stats_t *st = wvui_asset_stats();
```

3.7.8 wvui_asset_stats_reset(void)

```
void wvui_asset_stats_reset(void);
```

功能描述	清零资源统计计数。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_asset_stats_reset();
```

3.7.9 wvui_asset_open(wvui_asset_id_t id, wvui_blob_t *out)

```
uint8_t wvui_asset_open(wvui_asset_id_t id, wvui_blob_t *out);
```

功能描述	按 ID 打开资源，读取并校验头部后填充资源块。
输入参数	id：资源 ID；out：资源块输出。
输出参数	out：填充后的资源块
返回值	1 表示成功，0 表示后端未绑定或头部校验失败。

使用示例：

```
wvui_blob_t b;
if (wvui_asset_open(ASSET_LOGO, &b)) { /* 使用 b */ }
```

注意事项：

资源块仅描述位置与格式，不持有解码缓存。

3.7.10 wvui_asset_read(const wvui_blob_t *b, uint32_t off, uint8_t *dst, uint32_t len)

```
wvui_asset_read_t wvui_asset_read(const wvui_blob_t *b, uint32_t off, uint8_t *dst, uint32_t len);
```

功能描述	读取资源体的一段数据，越界部分按擦除态填充，以保证缓冲被写满。
输入参数	b：资源块；off：资源体内偏移；dst：目标缓冲；len：读取长度。
输出参数	dst：读出的数据
返回值	读取结果码：完整、部分、越界或失败。

使用示例：

```
uint8_t buf[64];
wvui_asset_read(&b, 0, buf, sizeof(buf));
```

注意事项：

部分读取时，超出资源末尾的字节被写为 0，调用方需按返回码判断。

3.7.11 wvui_asset_read_u8(const wvui_blob_t *b, uint32_t off)

```
uint8_t wvui_asset_read_u8(const wvui_blob_t *b, uint32_t off);
```

功能描述	读取资源体中的 1 个字节。
输入参数	b：资源块；off：资源体内偏移。
输出参数	无
返回值	字节值；越界返回 0。

使用示例：

```
uint8_t fmt = wvui_asset_read_u8(&b, 4);
```

注意事项：

该接口不返回错误码；如需区分“越界”与“值为 0”，请改用 wvui_asset_read()。

3.8 wvui 控件基类 API

3.8.1 wvui_class_bind(uint16_t type, const wvui_class_t *cls)

```
uint8_t wvui_class_bind(uint16_t type, const wvui_class_t *cls);
```

功能描述	在运行期把自定义控件类注册到类表。
输入参数	type：类型号，0 与越界值会被拒绝；cls：类描述。
输出参数	无
返回值	1 表示注册成功，0 表示参数非法或槽位不足。

使用示例：

```
wvui_class_bind(MY_TYPE, &my_class);
```

注意事项：

同一类型号重复注册会覆盖原描述；

已发布的内置类型号不应复用。

3.8.2 wvui_class_get(uint16_t type)

```
const wvui_class_t *wvui_class_get(uint16_t type);
```

功能描述	按类型号获取类描述。
输入参数	type：类型号。
输出参数	无
返回值	类描述指针；未注册返回 NULL。

使用示例：

```
const wvui_class_t *c = wvui_class_get(WVUI_TYPE_BTN);
```

3.8.3 wvui_class_bound_count(void)

```
uint16_t wvui_class_bound_count(void);
```

功能描述	获取已注册类的槽位数，即最高非空索引加一。
输入参数	无
输出参数	无

返回值	槽位数。

使用示例：

```
uint16_t n = wvui_class_bound_count();
```

3.8.4 wvui_widget_bind_arena(wvui_arena_t *persistent, wvui_arena_t *transient)

```
void wvui_widget_bind_arena(wvui_arena_t *persistent, wvui_arena_t *transient);
```

功能描述	绑定控件对象所使用的常驻内存区与临时内存区。
输入参数	persistent：常驻区；transient：临时区。
输出参数	无
返回值	无

使用示例：

```
wvui_widget_bind_arena(&perm, &trans);
```

注意事项：

应在 wvui_init() 之后、创建任何控件之前调用。

3.8.5 wvui_widget_create(const wvui_widget_desc_t *desc, uint8_t persistent)

```
wvui_widget_t *wvui_widget_create(const wvui_widget_desc_t *desc, uint8_t persistent);
```

功能描述	按描述创建控件，并追加到顶层链尾。
输入参数	desc：控件描述；persistent：非 0 时使用常驻区。
输出参数	无
返回值	控件指针；类型被裁剪或内存不足时返回 NULL。

使用示例：

```
wvui_widget_t *w = wvui_widget_create(&desc, 1);
```

注意事项：

返回 NULL 时不会破坏已有控件树。

3.8.6 wvui_widget_create_raw(uint16_t type, uint16_t id, int16_t x, int16_t y, int16_t w, int16_t h, uint8_t persistent)

```
wvui_widget_t *wvui_widget_create_raw(uint16_t type, uint16_t id, int16_t x, int16_t y, int16_t w,
int16_t h, uint8_t persistent);
```

功能描述	以展开参数创建控件，无需构造描述结构体。
输入参数	type：类型号；id：逻辑 ID；x、y：屏幕坐标；w、h：尺寸；persistent：内存区选择。
输出参数	无

返回值	控件指针；失败返回 NULL。

使用示例：

```
wvui_widget_t *l = wvui_widget_create_raw(WVUI_TYPE_LABEL, 1, 0, 0, 80, 20, 1);
```

3.8.7 wvui_widget_init_static(wvui_widget_t *storage, uint32_t bytes, const wvui_widget_desc_t *desc)

```
wvui_widget_t *wvui_widget_init_static(wvui_widget_t *storage, uint32_t bytes, const wvui_widget_desc_t *desc);
```

功能描述	将描述绑定到调用方提供的静态存储，实现零分配创建。
输入参数	storage：静态存储首地址；bytes：存储容量；desc：控件描述。
输出参数	无
返回值	控件指针；容量不足返回 NULL。

使用示例：

```
static uint8_t mem[128];
wvui_widget_t *w = wvui_widget_init_static((wvui_widget_t *)mem, sizeof(mem), &desc);
```

注意事项：

bytes 不得小于“控制头 + 类运行时块”，否则断言失败；

存储需按指针宽度对齐。

3.8.8 wvui_widget_delete(wvui_widget_t *w)

```
void wvui_widget_delete(wvui_widget_t *w);
```

功能描述	删除控件及其全部后代，依次执行回调、标脏、摘链与解除引用。
输入参数	w：待删除控件。
输出参数	无
返回值	无

使用示例：

```
wvui_widget_delete(box);   /* box 及其子控件全部释放 */
```

注意事项：

返回后所有后代句柄立即失效；

禁止在事件回调中直接删除自身或祖先，应置标志后于下一帧删除。

3.8.9 wvui_kernel_forget_widget(wvui_widget_t *w)

```
void wvui_kernel_forget_widget(wvui_widget_t *w);
```

功能描述	从图层、页面、输入与动画各子系统解除对该控件的引用。

输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_kernel_forget_widget(w);
```

注意事项：

由删除流程内部调用；外部单独调用将使控件处于“无引用但仍存在”的状态。

3.8.10 wvui_widget_first(void)

```
wvui_widget_t *wvui_widget_first(void);
```

功能描述	获取顶层控件链首。
输入参数	无
输出参数	无
返回值	控件指针；链为空返回 NULL。

使用示例：

```
for (wvui_widget_t *p = wvui_widget_first(); p; p = p->next) { }
```

3.8.11 wvui_widget_count(void)

```
uint32_t wvui_widget_count(void);
```

功能描述	统计顶层控件数量。
输入参数	无
输出参数	无
返回值	控件数量。

使用示例：

```
uint32_t n = wvui_widget_count();
```

注意事项：

仅统计顶层链，不含子控件。

3.8.12 wvui_widget_find_by_id(uint16_t id)

```
wvui_widget_t *wvui_widget_find_by_id(uint16_t id);
```

功能描述	按逻辑 ID 在控件树中查找控件。
输入参数	id：逻辑 ID，0 不参与查找。

输出参数	无
返回值	控件指针；未找到返回 NULL。

使用示例：

```
wvui_widget_t *t = wvui_widget_find_by_id(ID_TITLE);
```

注意事项：

遍历深度优先，耗时与控件数量线性相关，不宜在每帧中高频调用。

3.8.13 wvui_widget_reset_top(void)

```
void wvui_widget_reset_top(void);
```

功能描述	清空顶层控件链引用，但不释放对象内存。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_widget_reset_top();   /* 配合 arena 复位整体重建 */
```

注意事项：

仅用于整体重建场景；普通页面切换请使用页面栈接口。

3.8.14 wvui_widget_detach_top(void)

```
wvui_widget_t *wvui_widget_detach_top(void);
```

功能描述	把顶层链首摘下并返回，其余链保持不变。
输入参数	无
输出参数	无
返回值	被摘下的控件指针；链为空时返回 NULL。

使用示例：

```
wvui_widget_t *head = wvui_widget_detach_top();
```

注意事项：

摘下后需自行保管，或调用 wvui_widget_attach_top() 还原。

3.8.15 wvui_widget_attach_top(wvui_widget_t *head)

```
void wvui_widget_attach_top(wvui_widget_t *head);
```

功能描述	把整条控件链挂回顶层。
输入参数	head：链首指针，可为 NULL。

输出参数	无
返回值	无

使用示例：

```
wvui_widget_attach_top(head);
```

注意事项：

传入 NULL 为空操作，可用于恢复“空场景”。

3.8.16 wvui_widget_transient_arena(void)

```
wvui_arena_t *wvui_widget_transient_arena(void);
```

功能描述	获取控件子系统使用的临时内存区。
输入参数	无
输出参数	无
返回值	区指针；STATIC/HEAP 模式返回 NULL。

使用示例：

```
wvui_arena_t *t = wvui_widget_transient_arena();
```

注意事项：

用于统计与调试，不建议外部直接从中分配。

3.8.17 wvui_widget_set_pos(wvui_widget_t *w, int16_t x, int16_t y)

```
void wvui_widget_set_pos(wvui_widget_t *w, int16_t x, int16_t y);
```

功能描述	设置控件绝对坐标，自动标脏并触发 set_pos 钩子。
输入参数	w：控件；x、y：新的左上角坐标。
输出参数	无
返回值	无

使用示例：

```
wvui_widget_set_pos(btn, 20, 40);
```

注意事项：

子控件坐标随之平移，无需逐个修改。

3.8.18 wvui_widget_set_hit_pad(wvui_widget_t *w, int8_t l, int8_t t, int8_t r, int8_t b)

```
void wvui_widget_set_hit_pad(wvui_widget_t *w, int8_t l, int8_t t, int8_t r, int8_t b);
```

功能描述	设置触摸命中区相对几何矩形的四边扩展量。

输入参数	w：控件；l、t、r、b：左、上、右、下扩展量，负值表示收缩。
输出参数	无
返回值	无

使用示例：

```
wvui_widget_set_hit_pad(btn, -2, -2, -2, -2);
```

注意事项：

只影响命中测试，不影响绘制范围。

3.8.19 wvui_widget_set_hidden(wvui_widget_t *w, uint8_t hidden)

```
void wvui_widget_set_hidden(wvui_widget_t *w, uint8_t hidden);
```

功能描述	隐藏或显示整棵子树。
输入参数	w：控件；hidden：非 0 隐藏。
输出参数	无
返回值	无

使用示例：

```
wvui_widget_set_hidden(panel, 1);
```

注意事项：

隐藏后不参与绘制与命中测试，但对象仍然存在。

3.8.20 wvui_widget_is_hidden(const wvui_widget_t *w)

```
uint8_t wvui_widget_is_hidden(const wvui_widget_t *w);
```

功能描述	查询控件自身的隐藏标志。
输入参数	w：控件。
输出参数	无
返回值	1 表示隐藏。

使用示例：

```
if (wvui_widget_is_hidden(panel)) { /* ... */ }
```

注意事项：

父控件隐藏时本函数仍返回 0，需结合父链判断实际可见性。

3.8.21 wvui_widget_bring_to_front(wvui_widget_t *w)

```
void wvui_widget_bring_to_front(wvui_widget_t *w);
```

功能描述	把控件移动到同级的链尾，即最上层。

输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_widget_bring_to_front(menu);
```

注意事项：

仅调整同级顺序，不改变父子关系。

3.8.22 wvui_widget_detach(wvui_widget_t *w)

```
uint8_t wvui_widget_detach(wvui_widget_t *w);
```

功能描述	把控件从父链摘除，但保留对象本身。
输入参数	w：控件。
输出参数	无
返回值	1 表示摘除成功，0 表示该控件本就不在链上。

使用示例：

```
wvui_widget_detach(item);   /* 稍后可重新 append 到别处 */
```

注意事项：

摘除后对象不再绘制，需重新挂入才能显示。

3.8.23 wvui_widget_attach_root(wvui_widget_t *w)

```
void wvui_widget_attach_root(wvui_widget_t *w);
```

功能描述	把控件挂到顶层链尾。
输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_widget_attach_root(panel);
```

注意事项：

若控件已在其他链上，应先摘除以避免重复挂载。

3.8.24 wvui_widget_append_child(wvui_widget_t *parent, wvui_widget_t *child)

```
void wvui_widget_append_child(wvui_widget_t *parent, wvui_widget_t *child);
```

功能描述	把子控件追加到父控件的子链尾。

输入参数	parent：父控件；child：子控件。
输出参数	无
返回值	无

使用示例：

```
wvui_widget_append_child(panel, label);
```

注意事项：

父控件移动、隐藏或删除时，子控件随之联动；

child 必须尚未挂入其他链表。

3.8.25 wvui_widget_rect(const wvui_widget_t *w)

```
wvui_rect_t wvui_widget_rect(const wvui_widget_t *w);
```

功能描述	获取控件的几何矩形（屏幕绝对坐标）。
输入参数	w：控件。
输出参数	无
返回值	矩形；w 为 NULL 返回空矩形。

使用示例：

```
wvui_rect_t r = wvui_widget_rect(btn);
```

3.8.26 wvui_widget_hit_rect(const wvui_widget_t *w)

```
wvui_rect_t wvui_widget_hit_rect(const wvui_widget_t *w);
```

功能描述	获取包含命中扩展量的矩形。
输入参数	w：控件。
输出参数	无
返回值	命中矩形。

使用示例：

```
wvui_rect_t hr = wvui_widget_hit_rect(btn);
```

注意事项：

标脏应使用几何矩形，命中矩形仅用于触摸判定。

3.8.27 wvui_widget_hit_test_point(const wvui_widget_t *w, int16_t x, int16_t y)

```
uint8_t wvui_widget_hit_test_point(const wvui_widget_t *w, int16_t x, int16_t y);
```

功能描述	判断给定屏幕坐标是否落在控件命中区内。

输入参数	w：控件；x、y：屏幕坐标。
输出参数	无
返回值	1 表示命中。

使用示例：

```
if (wvui_widget_hit_test_point(btn, px, py)) { /* 命中 */ }
```

注意事项：

本函数不考虑父子遮挡与隐藏状态；层级判定请使用 wvui_input_hit_test()。

3.8.28 wvui_widget_invalidate(wvui_widget_t *w)

```
void wvui_widget_invalidate(wvui_widget_t *w);
```

功能描述	**把控件自身的几何矩形标记为脏。**
输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_widget_invalidate(label);
```

注意事项：

子控件区域不在标脏范围内，需分别标记。

3.8.29 wvui_widget_invalidate_rect(wvui_widget_t *w, wvui_rect_t r)

```
void wvui_widget_invalidate_rect(wvui_widget_t *w, wvui_rect_t r);
```

功能描述	**将控件内的局部矩形标记为脏，用于局部内容变化。**
输入参数	w：控件；r：控件局部坐标矩形。
输出参数	无
返回值	无

使用示例：

```
wvui_widget_invalidate_rect(bar, (wvui_rect_t){0, 0, 40, 10});
```

注意事项：

局部矩形会按控件位置换算为屏幕坐标后再行裁剪。

3.8.30 wvui_widgets_register_builtin(void)

```
uint16_t wvui_widgets_register_builtin(void);
```

功能描述	**返回内置类表当前可用的槽位数，类表在编译期已填充。**

输入参数	无
输出参数	无
返回值	槽位数。

使用示例：

```
uint16_t n = wvui_widgets_register_builtin();
```

注意事项：

本函数不做运行期绑定；自定义控件请使用 wvui_class_bind()。

3.8.31 wvui_prop_u16(const uint8_t *p)

```
uint16_t wvui_prop_u16(const uint8_t *p);
```

功能描述	从属性负载中读取一个小端 16 位无符号数。
输入参数	p：负载指针，至少 2 字节。
输出参数	无
返回值	16 位值。

使用示例：

```
uint16_t id = wvui_prop_u16(payload);
```

注意事项：

不做长度与空指针检查，调用方需先校验负载长度。

3.8.32 wvui_prop_i16(const uint8_t *p)

```
int16_t wvui_prop_i16(const uint8_t *p);
```

功能描述	从属性负载中读取一个小端 16 位有符号数。
输入参数	p：负载指针，至少 2 字节。
输出参数	无
返回值	有符号值。

使用示例：

```
int16_t v = wvui_prop_i16(payload);
```

注意事项：

按补码解释，负数原样返回。

3.8.33 wvui_prop_write_u16(uint8_t *p, uint16_t v)

```
void wvui_prop_write_u16(uint8_t *p, uint16_t v);
```

功能描述	把 16 位值按小端写入目标缓冲。

输入参数	p：目标缓冲，至少包含 2 个可写字节；v：待写入值。
输出参数	p：写入结果
返回值	无

使用示例：

```
uint8_t buf[8];
wvui_prop_write_u16(buf, 320);
```

3.8.34 wvui_prop_write_text(uint8_t *out, uint16_t cap, const char *s)

```
uint16_t wvui_prop_write_text(uint8_t *out, uint16_t cap, const char *s);
```

功能描述	将字符串按原始字节写入缓冲，超长时截断且不补 NUL。
输入参数	out：目标缓冲；cap：容量；s：源字符串。
输出参数	out：写入结果
返回值	实际写入字节数；参数非法返回 0。

使用示例：

```
uint8_t buf[16];
uint16_t n = wvui_prop_write_text(buf, sizeof(buf), "OK");
```

注意事项：

输出不保证以 NUL 结尾，接收方须按返回长度处理。

3.8.35 wvui_slice_w(const wvui_slice_t *s)

```
int16_t wvui_slice_w(const wvui_slice_t *s);
```

功能描述	获取切片宽度。
输入参数	s：切片对象。
输出参数	无
返回值	像素宽度。

使用示例：

```
int16_t w = wvui_slice_w(sl);
```

3.8.36 wvui_slice_h(const wvui_slice_t *s)

```
int16_t wvui_slice_h(const wvui_slice_t *s);
```

功能描述	获取切片高度。
输入参数	s：切片对象。

输出参数	无
返回值	像素高度。

使用示例：

```
int16_t h = wvui_slice_h(sl);
```

3.8.37 wvui_draw_slice(void)

```
const wvui_slice_t *wvui_draw_slice(void);
```

功能描述	获取当前正在绘制的切片对象。
输入参数	无
输出参数	无
返回值	切片指针；不在绘制中返回 NULL。

使用示例：

```
const wvui_slice_t *sl = wvui_draw_slice();
```

注意事项：

只能在 draw 回调内使用，于帧循环之外调用将返回 NULL。

3.9 wvui 图层与页面 API

3.9.1 wvui_layer_attach(wvui_widget_t *w)

```
uint8_t wvui_layer_attach(wvui_widget_t *w);
```

功能描述	将控件追加到顶层链尾（即最上层），挂入前先从原链摘除。
输入参数	w：控件。
输出参数	无
返回值	1 表示成功，0 表示参数为空。

使用示例：

```
wvui_layer_attach(menu);   /* 置顶显示 */
```

注意事项：

控件已挂在某页时将被摘下，页面卸载逻辑将不再管理该控件。

3.9.2 wvui_layer_detach(wvui_widget_t *w)

```
void wvui_layer_detach(wvui_widget_t *w);
```

功能描述	把控件从顶层链移除，对象本身仍然有效。
输入参数	w：控件。

输出参数	无
返回值	无

使用示例：

```
wvui_layer_detach(menu);
```

注意事项：

移除后不再绘制与响应，可再次 attach 恢复。

3.9.3 wvui_layer_first(void)

```
wvui_widget_t *wvui_layer_first(void);
```

功能描述	获取顶层链首控件。
输入参数	无
输出参数	无
返回值	控件指针；链为空返回 NULL。

使用示例：

```
wvui_widget_t *top = wvui_layer_first();
```

3.9.4 wvui_layer_unlink_widget(wvui_widget_t *w)

```
void wvui_layer_unlink_widget(wvui_widget_t *w);
```

功能描述	控件被删除时，清除顶层链与模态槽中的引用。
输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_layer_unlink_widget(w);
```

注意事项：

由删除流程自动调用，外部无需手动处理。

3.9.5 wvui_layer_reset(void)

```
void wvui_layer_reset(void);
```

功能描述	清空顶层链与模态槽，但不删除任何对象。
输入参数	无
输出参数	无

返回值	无

使用示例：

```
wvui_layer_reset();
```

注意事项：

仅用于整体重建前的清理；普通场景请使用页面接口。

3.9.6 wvui_modal_open(wvui_widget_t *w)

```
uint8_t wvui_modal_open(wvui_widget_t *w);
```

功能描述	将控件设为模态，此后命中测试仅作用于顶层链。
输入参数	w：控件。
输出参数	无
返回值	1 表示成功，0 表示已有模态或参数为空。

使用示例：

```
wvui_modal_open(dialog);
```

注意事项：

同一时刻只允许一个模态，需先关闭已有模态；

模态控件应位于顶层链，否则仍可能被遮挡。

3.9.7 wvui_modal_close(wvui_widget_t *w)

```
void wvui_modal_close(wvui_widget_t *w);
```

功能描述	关闭模态，恢复普通命中测试。
输入参数	w：控件，仅当其为当前模态时生效。
输出参数	无
返回值	无

使用示例：

```
wvui_modal_close(dialog);
```

注意事项：

关闭后控件仍在链上，如有需要可再调用 detach 或删除。

3.9.8 wvui_modal_get(void)

```
wvui_widget_t *wvui_modal_get(void);
```

功能描述	获取当前模态控件。
输入参数	无

输出参数	无
返回值	控件指针；无模态返回 NULL。

使用示例：

```
if (wvui_modal_get() == NULL) { /* 无模态 */ }
```

3.9.9 wvui_page_go(wvui_page_t *p)

```
void wvui_page_go(wvui_page_t *p);
```

功能描述	切换到目标页面，不压栈，当前页被替换。
输入参数	p：页面描述。
输出参数	无
返回值	无

使用示例：

```
wvui_page_go(&page_main);
```

注意事项：

切换在帧边界生效，本帧内仍为旧页。

3.9.10 wvui_page_push(wvui_page_t *p)

```
uint8_t wvui_page_push(wvui_page_t *p);
```

功能描述	把目标页压入页面栈并进入该页。
输入参数	p：页面描述。
输出参数	无
返回值	1 表示入栈成功，0 表示栈已满或参数非法。

使用示例：

```
wvui_page_push(&page_setting);
```

注意事项：

入栈后原页面对象链被停放，返回时按页面策略恢复或重建。

3.9.11 wvui_page_back(void)

```
uint8_t wvui_page_back(void);
```

功能描述	返回上一页；无状态页将执行重建而非恢复。
输入参数	无
输出参数	无

返回值	1 表示成功，0 表示已在栈底。

使用示例：

```
wvui_page_back();
```

注意事项：

返回过程会停止被卸载页链上的动画。

3.9.12 wvui_page_reset(wvui_page_t *p)

```
uint8_t wvui_page_reset(wvui_page_t *p);
```

功能描述	清空页面栈并立即构建首页。
输入参数	p：首页描述。
输出参数	无
返回值	1 表示成功。

使用示例：

```
wvui_page_reset(&page_home);
```

注意事项：

常用于上电初始化，将丢弃此前所有页面状态。

3.9.13 wvui_page_apply(void)

```
uint8_t wvui_page_apply(void);
```

功能描述	在帧边界应用挂起的页面切换请求。
输入参数	无
输出参数	无
返回值	1 表示本帧发生了切换。

使用示例：

```
if (wvui_page_apply()) { wvui_stats_reset(); }
```

注意事项：

由 wvui_frame() 内部调用，常规用法无需手动调用。

3.9.14 wvui_page_leave(void)

```
void wvui_page_leave(void);
```

功能描述	离开当前页并清空页面栈，不整屏标脏。
输入参数	无
输出参数	无

返回值	无

使用示例：

```
wvui_page_leave();
```

注意事项：

离开后顶层为空，需自行重新构建内容。

3.9.15 wvui_page_tick(void)

```
void wvui_page_tick(void);
```

功能描述	调用当前页注册的 on_tick 回调。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_page_tick();
```

注意事项：

由帧循环统一调用，回调内应避免长时间阻塞。

3.9.16 wvui_page_current(void)

```
const wvui_page_t *wvui_page_current(void);
```

功能描述	获取当前页描述。
输入参数	无
输出参数	无
返回值	页指针；无当前页返回 NULL。

使用示例：

```
const wvui_page_t *p = wvui_page_current();
```

3.9.17 wvui_page_pending(void)

```
const wvui_page_t *wvui_page_pending(void);
```

功能描述	获取待切换的目标页。
输入参数	无
输出参数	无
返回值	页指针；无挂起切换返回 NULL。

使用示例：

```
const wvui_page_t *next = wvui_page_pending();
```

3.9.18 wvui_page_depth(void)

```
uint16_t wvui_page_depth(void);
```

功能描述	获取页面栈深度。
输入参数	无
输出参数	无
返回值	深度值。

使用示例：

```
uint16_t d = wvui_page_depth();
```

3.9.19 wvui_page_opa(void)

```
uint8_t wvui_page_opa(void);
```

功能描述	获取页面层透明度乘子，用于淡入过渡。
输入参数	无
输出参数	无
返回值	0..255。

使用示例：

```
uint8_t opa = wvui_page_opa();
```

3.9.20 wvui_page_set_fade(uint8_t on)

```
void wvui_page_set_fade(uint8_t on);
```

功能描述	开启或关闭页面切换淡入效果。
输入参数	on：非 0 开启。
输出参数	无
返回值	无

使用示例：

```
wvui_page_set_fade(1);
```

注意事项：

淡入需要多帧合成，低性能屏可将其关闭以避免闪烁。

3.9.21 wvui_page_fade_enabled(void)

```
uint8_t wvui_page_fade_enabled(void);
```

功能描述	查询淡入开关。
输入参数	无
输出参数	无
返回值	1 表示已开启。

使用示例：

```
if (wvui_page_fade_enabled()) { /* ... */ }
```

3.9.22 wvui_page_unlink_widget(wvui_widget_t *w)

```
uint8_t wvui_page_unlink_widget(wvui_widget_t *w);
```

功能描述	从停放页的对象链中解除控件引用，包含链首情形。
输入参数	w：控件。
输出参数	无
返回值	1 表示找到并解除。

使用示例：

```
wvui_page_unlink_widget(w);
```

注意事项：

由删除流程内部调用，以避免停放页持有悬空指针。

3.9.23 wvui_page_stop_anims(wvui_widget_t *head)

```
void wvui_page_stop_anims(wvui_widget_t *head);
```

功能描述	递归停止指定控件链（含子控件）上的全部动画节点。
输入参数	head：链首。
输出参数	无
返回值	无

使用示例：

```
wvui_page_stop_anims(old_head);
```

注意事项：

页面卸载时调用，防止已停放页面的动画继续消耗 CPU。

3.10 wvui 输入、动画与滚动 API

3.10.1 wvui_input_stats(void)

```
const wvui_input_stats_t *wvui_input_stats(void);
```

功能描述	获取输入侧统计，包括命中次数、事件数与滑动次数。
输入参数	无
输出参数	无
返回值	统计结构常量指针。

使用示例：

```
const wvui_input_stats_t *st = wvui_input_stats();
```

3.10.2 wvui_input_stats_reset(void)

```
void wvui_input_stats_reset(void);
```

功能描述	清零输入统计计数，并保留按下与手势接管状态。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_input_stats_reset();
```

3.10.3 wvui_input_reset(void)

```
void wvui_input_reset(void);
```

功能描述	清除按下捕获与手势接管状态。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_input_reset();   /* 页面切换后清理残留按压 */
```

注意事项：

不会重置统计计数，两者可分别调用。

3.10.4 wvui_input_feed(const wvui_indev_t *in)

```
void wvui_input_feed(const wvui_indev_t *in);
```

功能描述	送入一帧采样，由内部合成按下、保持、释放、拖动与滑动事件。
输入参数	in：输入采样，包含坐标与按下标志。
输出参数	无
返回值	无

使用示例：

```
wvui_indev_t in = { .x = 30, .y = 60, .pressed = 1 };
wvui_input_feed(&in);
```

注意事项：

每帧最多调用一次；重复调用会将同一次采样当作多次状态变化。

3.10.5 wvui_input_pressed(void)

```
wvui_widget_t *wvui_input_pressed(void);
```

功能描述	获取当前被按下捕获的控件。
输入参数	无
输出参数	无
返回值	控件指针；无捕获返回 NULL。

使用示例：

```
wvui_widget_t *p = wvui_input_pressed();
```

3.10.6 wvui_input_grab(void)

```
wvui_widget_t *wvui_input_grab(void);
```

功能描述	获取当前手势接管者。
输入参数	无
输出参数	无
返回值	控件指针；无接管返回 NULL。

使用示例：

```
wvui_widget_t *g = wvui_input_grab();
```

3.10.7 wvui_indev_grab(wvui_widget_t *w)

```
uint8_t wvui_indev_grab(wvui_widget_t *w);
```

功能描述	由控件显式接管后续手势事件，直至释放。
输入参数	w：控件。

输出参数	无
返回值	1 表示接管成功，0 表示参数为空。

使用示例：

```
case WVUI_EV_PRESS: wvui_indev_grab(w); break;
```

注意事项：

接管期间其他控件收不到拖动与滑动事件；

释放后自动解除接管。

3.10.8 wvui_input_forget(wvui_widget_t *w)

```
void wvui_input_forget(wvui_widget_t *w);
```

功能描述	控件被删除时清除输入层对它的引用。
输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_input_forget(w);
```

注意事项：

由删除流程调用，避免残留悬空指针。

3.10.9 wvui_input_hit_test(int16_t x, int16_t y)

```
wvui_widget_t *wvui_input_hit_test(int16_t x, int16_t y);
```

功能描述	按层级与裁剪进行命中测试，并向下钻取至最深层命中控件。
输入参数	x、y：屏幕坐标。
输出参数	无
返回值	命中的控件指针；无命中返回 NULL。

使用示例：

```
wvui_widget_t *hit = wvui_input_hit_test(px, py);
```

注意事项：

综合考虑隐藏状态、裁剪与模态，是推荐的外部命中查询接口。

3.10.10 wvui_anim_init(wvui_anim_t *a)

```
void wvui_anim_init(wvui_anim_t *a);
```

功能描述	初始化动画节点，使其处于未挂链状态。

输入参数	a：动画节点。
输出参数	无
返回值	无

使用示例：

```
wvui_anim_init(&w->anim);
```

注意事项：

必须在控件 init 回调中调用一次，之后才能调用 start。

3.10.11 wvui_anim_start(wvui_anim_t *a, wvui_widget_t *owner, wvui_anim_step_fn step)

```
void wvui_anim_start(wvui_anim_t *a, wvui_widget_t *owner, wvui_anim_step_fn step);
```

功能描述	启动或刷新动画节点，操作幂等，不会重复挂链。
输入参数	a：节点；owner：宿主控件；step：步进回调，返回非 0 表示继续。
输出参数	无
返回值	无

使用示例：

```
static uint8_t step(wvui_anim_t *a) { return 1; }
wvui_anim_start(&w->anim, w, step);
```

注意事项：

step 返回 0 时节点自动摘链，无需再次调用 stop。

3.10.12 wvui_anim_start_permanent(wvui_anim_t *a, wvui_widget_t *owner, wvui_anim_step_fn step)

```
void wvui_anim_start_permanent(wvui_anim_t *a, wvui_widget_t *owner, wvui_anim_step_fn step);
```

功能描述	启动常驻动画，用于闪烁、跑马灯、自动轮播、提示存活与淡出等场景。
输入参数	a：节点；owner：宿主控件；step：步进回调。
输出参数	无
返回值	无

使用示例：

```
wvui_anim_start_permanent(&w->anim, w, blink_step);
```

注意事项：

常驻节点不受普通 stop 以外的自动回收影响，控件销毁前须确保其已摘除。

3.10.13 wvui_anim_stop(wvui_anim_t *a)

```
void wvui_anim_stop(wvui_anim_t *a);
```

功能描述	把动画节点从活动链摘除，幂等。
输入参数	a：动画节点。
输出参数	无
返回值	无

使用示例：

```
wvui_anim_stop(&w->anim);
```

3.10.14 wvui_anim_is_active(const wvui_anim_t *a)

```
uint8_t wvui_anim_is_active(const wvui_anim_t *a);
```

功能描述	查询节点是否在活动链上。
输入参数	a：动画节点。
输出参数	无
返回值	1 表示处于活动状态，编译期关闭动画时恒为 0。

使用示例：

```
if (!wvui_anim_is_active(&w->anim)) { wvui_anim_start(...); }
```

3.10.15 wvui_anim_step_all(void)

```
void wvui_anim_step_all(void);
```

功能描述	推进所有活动节点一帧。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_anim_step_all();
```

注意事项：

由帧循环统一调用，在回调内标脏即可触发重绘。

3.10.16 wvui_ramp_start(wvui_ramp_t *r, uint16_t inc, uint16_t frames)

```
void wvui_ramp_start(wvui_ramp_t *r, uint16_t inc, uint16_t frames);
```

功能描述	配置定帧补间参数，每帧增量与总帧数。

输入参数	r：补间对象；inc：每帧增量；frames：总帧数。
输出参数	无
返回值	无

使用示例：

```
wvui_ramp_start(&w->ramp, 8, 32);
```

注意事项：

本函数仅配置参数，需再调用 wvui_anim_start() 方会运行。

3.10.17 wvui_ramp_step(wvui_ramp_t *r)

```
uint8_t wvui_ramp_step(wvui_ramp_t *r);
```

功能描述	推进补间一帧。
输入参数	r：补间对象。
输出参数	无
返回值	1 表示尚未结束，0 表示已到目标值。

使用示例：

```
static uint8_t step(wvui_anim_t *a) {
    return wvui_ramp_step(&((my_t *)a->owner)->ramp);
}
```

3.10.18 wvui_ramp_finish(wvui_ramp_t *r)

```
void wvui_ramp_finish(wvui_ramp_t *r);
```

功能描述	直接把补间跳到目标值，用于取消动画。
输入参数	r：补间对象。
输出参数	无
返回值	无

使用示例：

```
wvui_ramp_finish(&w->ramp);
wvui_anim_stop(&w->anim);
```

3.10.19 wvui_anim_forget_widget(wvui_widget_t *w)

```
void wvui_anim_forget_widget(wvui_widget_t *w);
```

功能描述	摘除宿主为该控件的全部动画节点。

输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_anim_forget_widget(w);
```

注意事项：

由 wvui_kernel_forget_widget() 调用，避免宿主销毁后仍有步进。

3.10.20 wvui_anim_drop_range(const void *lo, const void *hi)

```
void wvui_anim_drop_range(const void *lo, const void *hi);
```

功能描述	摘除地址落在 [lo, hi) 区间内的动画节点。
输入参数	lo：区间下界；hi：区间上界，不包含。
输出参数	无
返回值	无

使用示例：

```
wvui_anim_drop_range(arena_start, arena_end);
```

注意事项：

内存区整体回收后必须调用，否则活动链将指向已失效内存。

3.10.21 wvui_anim_active(void)

```
uint16_t wvui_anim_active(void);
```

功能描述	统计当前活动节点数量，开销与节点数成正比。
输入参数	无
输出参数	无
返回值	节点数。

使用示例：

```
uint16_t n = wvui_anim_active();
```

注意事项：

仅用于统计与诊断，不要放在每帧热路径。

3.10.22 wvui_anim_steps(void)

```
uint32_t wvui_anim_steps(void);
```

功能描述	获取自启动以来的累计步进帧数。

输入参数	无
输出参数	无
返回值	帧数。

使用示例：

```
uint32_t s = wvui_anim_steps();
```

3.10.23 wvui_anim_stats_reset(void)

```
void wvui_anim_stats_reset(void);
```

功能描述	只清动画计数，不影响活动链。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_anim_stats_reset();
```

3.10.24 wvui_scroll_init(wvui_scroll_t *s, int32_t max)

```
void wvui_scroll_init(wvui_scroll_t *s, int32_t max);
```

功能描述	初始化滚动状态并设置最大滚动距离。
输入参数	s：滚动状态；max：可滚动范围。
输出参数	无
返回值	无

使用示例：

```
wvui_scroll_init(&w->scroll, content_h - view_h);
```

注意事项：

内容高度不大于可视高度时应传 0，表示不可滚动。

3.10.25 wvui_scroll_set_range(wvui_scroll_t *s, int32_t max)

```
void wvui_scroll_set_range(wvui_scroll_t *s, int32_t max);
```

功能描述	更新可滚范围，并将当前位置夹紧至新范围内。
输入参数	s：滚动状态；max：新的范围。
输出参数	无

返回值	无

使用示例：

```
wvui_scroll_set_range(&w->scroll, new_max);
```

注意事项：

内容变短时位置会自动回夹，避免出现空白区。

3.10.26 wvui_scroll_press(wvui_scroll_t *s, int16_t c)

```
void wvui_scroll_press(wvui_scroll_t *s, int16_t c);
```

功能描述	记录按下起点并停止惯性滚动。
输入参数	s：滚动状态；c：按下坐标。
输出参数	无
返回值	无

使用示例：

```
case WVUI_EV_PRESS: wvui_scroll_press(&w->scroll, d->y); break;
```

3.10.27 wvui_scroll_stay(wvui_scroll_t *s, int16_t c)

```
uint8_t wvui_scroll_stay(wvui_scroll_t *s, int16_t c);
```

功能描述	保持按住并更新坐标，计算是否产生拖动。
输入参数	s：滚动状态；c：当前坐标。
输出参数	无
返回值	1 表示已进入拖动状态。

使用示例：

```
if (wvui_scroll_stay(&w->scroll, d->y)) { wvui_widget_invalidate(w); }
```

注意事项：

超过内部阈值后才返回 1，可用于与点击手势相区分。

3.10.28 wvui_scroll_take_delta(wvui_scroll_t *s)

```
int32_t wvui_scroll_take_delta(wvui_scroll_t *s);
```

功能描述	取出自上次调用以来的位移增量，并清零累加值。
输入参数	s：滚动状态。
输出参数	无
返回值	位移值，正负表示方向。

使用示例：

```
int32_t dy = wvui_scroll_take_delta(&w->scroll);
```

注意事项：

读取即清零，漏用一次会丢失该帧位移。

3.10.29 wvui_scroll_release(wvui_scroll_t *s)

```
uint8_t wvui_scroll_release(wvui_scroll_t *s);
```

功能描述	释放按压，并根据最后速度启动惯性滚动。
输入参数	s：滚动状态。
输出参数	无
返回值	1 表示仍在滚动，0 表示已停止。

使用示例：

```
wvui_scroll_release(&w->scroll);
```

3.10.30 wvui_scroll_swipe(wvui_scroll_t *s, int16_t c)

```
uint8_t wvui_scroll_swipe(wvui_scroll_t *s, int16_t c);
```

功能描述	按滑动方向注入一次快速滚动。
输入参数	s：滚动状态；c：方向量。
输出参数	无
返回值	1 表示仍在滚动。

使用示例：

```
wvui_scroll_swipe(&w->scroll, d->delta);
```

注意事项：

用于实现“快速滑动翻页”，需自行根据返回值决定是否继续步进。

3.10.31 wvui_scroll_step(wvui_scroll_t *s)

```
uint8_t wvui_scroll_step(wvui_scroll_t *s);
```

功能描述	推进惯性滚动一帧，包含边界回弹处理。
输入参数	s：滚动状态。
输出参数	无
返回值	1 表示仍在滚动。

使用示例：

```
if (wvui_scroll_step(&w->scroll)) { wvui_widget_invalidate(w); }
```

注意事项：

应在每帧 tick 中调用，返回 0 后可停止标脏。

3.10.32 wvui_scroll_set(wvui_scroll_t *s, int32_t pos)

```
uint8_t wvui_scroll_set(wvui_scroll_t *s, int32_t pos);
```

功能描述	直接设置滚动位置，并夹紧到有效范围。
输入参数	s：滚动状态；pos：目标位置。
输出参数	无
返回值	1 表示位置发生变化。

使用示例：

```
if (wvui_scroll_set(&w->scroll, 0)) { wvui_widget_invalidate(w); }
```

注意事项：

设置位置不会停止已有的惯性，必要时先调用 wvui_anim_stop()。

3.10.33 wvui_scroll_active(const wvui_scroll_t *s)

```
uint8_t wvui_scroll_active(const wvui_scroll_t *s);
```

功能描述	查询是否处于滚动或惯性状态。
输入参数	s：滚动状态。
输出参数	无
返回值	1 表示活动。

使用示例：

```
if (!wvui_scroll_active(&w->scroll)) { /* 已静止 */ }
```

3.10.34 wvui_scroll_progress(const wvui_scroll_t *s)

```
uint8_t wvui_scroll_progress(const wvui_scroll_t *s);
```

功能描述	获取滚动进度，可用于绘制滚动条。
输入参数	s：滚动状态。
输出参数	无
返回值	进度值 0..255。

使用示例：

```
uint8_t p = wvui_scroll_progress(&w->scroll);
```

注意事项：

范围为 0 时返回 0，需自行判断是否显示滚动条。

3.11 wvui 命令通道 API

3.11.1 wvui_cmd_dispatch(uint16_t id, uint8_t op, const uint8_t *payload, uint16_t len)

```
int wvui_cmd_dispatch(uint16_t id, uint8_t op, const uint8_t *payload, uint16_t len);
```

功能描述	分发一帧命令，按对象 ID 与操作码路由至属性写入或系统操作。
输入参数	id：目标对象 ID；op：操作码；payload：TLV 负载；len：负载长度。
输出参数	无
返回值	0 表示成功，负值表示对象不存在或参数非法。

使用示例：

```
wvui_cmd_dispatch(1, WVUI_CMD_SET_TEXT, buf, len);
```

注意事项：

负载按属性表解析；长度不匹配时直接返回错误，不写入任何属性。

3.11.2 wvui_cmd_text_store(const uint8_t *bytes, uint16_t len)

```
const char *wvui_cmd_text_store(const uint8_t *bytes, uint16_t len);
```

功能描述	将命令携带的文本拷入文本池并自动补 NUL。
输入参数	bytes：源字节；len：长度。
输出参数	无
返回值	池内字符串指针；空间不足时返回 NULL。

使用示例：

```
const char *t = wvui_cmd_text_store(payload, len);
if (t) { wvui_label_set_text(label, t); }
```

注意事项：

指针在下次 reset 或池满后失效，控件不得长期持有。

3.11.3 wvui_cmd_text_reset(void)

```
void wvui_cmd_text_reset(void);
```

功能描述	清空文本池，不影响环形缓冲、解析状态与计数。
输入参数	无
输出参数	无

返回值	无

使用示例：

```
wvui_cmd_text_reset();
```

注意事项：

复位后此前存入的文本指针全部失效。

3.11.4 wvui_cmd_push(const uint8_t *bytes, uint16_t len)

```
uint16_t wvui_cmd_push(const uint8_t *bytes, uint16_t len);
```

功能描述	将字节流写入环形缓冲，可由串口中断调用。
输入参数	bytes：数据；len：长度。
输出参数	无
返回值	实际写入字节数；小于 len 表示缓冲将满。

使用示例：

```
void uart_rx_isr(void) {
    uint8_t c = read_reg();
    wvui_cmd_push(&c, 1);
}
```

注意事项：

该接口为中断安全，但不得在中断中调用 wvui_cmd_poll()。

3.11.5 wvui_cmd_poll(void)

```
uint16_t wvui_cmd_poll(void);
```

功能描述	推进解析状态机，把完整帧取出并分发。
输入参数	无
输出参数	无
返回值	本次分发的帧数。

使用示例：

```
if (wvui_cmd_poll() > 0) { wvui_render_commit(); }
```

注意事项：

应每帧调用一次，并集中于主循环上下文执行。

3.11.6 wvui_cmd_bind_tx(wvui_cmd_tx_fn tx)

```
void wvui_cmd_bind_tx(wvui_cmd_tx_fn tx);
```

功能描述	绑定应答回传通道，用于返回命令执行结果。

输入参数	tx：发送回调，传 NULL 表示不应答。
输出参数	无
返回值	无

使用示例：

```
wvui_cmd_bind_tx(uart_send);
```

注意事项：

回调在 poll 上下文执行，内部不得长时间阻塞。

3.11.7 wvui_cmd_reset(void)

```
void wvui_cmd_reset(void);
```

功能描述	重置环形缓冲、解析状态、文本池与全部计数。
输入参数	无
输出参数	无
返回值	无

使用示例：

```
wvui_cmd_reset();   /* 通信异常后重新同步 */
```

3.11.8 wvui_cmd_stats(void)

```
const wvui_cmd_stats_t *wvui_cmd_stats(void);
```

功能描述	获取命令通道计数，包括收帧、丢帧、校验错误与溢出。
输入参数	无
输出参数	无
返回值	统计结构常量指针。

使用示例：

```
const wvui_cmd_stats_t *st = wvui_cmd_stats();
```

3.11.9 wvui_cmd_stats_reset(void)

```
void wvui_cmd_stats_reset(void);
```

功能描述	只清计数，保留环形缓冲与解析状态。
输入参数	无
输出参数	无

返回值	无

使用示例：

```
wvui_cmd_stats_reset();
```

3.11.10 wvui_cmd_pending(void)

```
uint16_t wvui_cmd_pending(void);
```

功能描述	获取环形缓冲中尚未取走的字节数。
输入参数	无
输出参数	无
返回值	字节数。

使用示例：

```
if (wvui_cmd_pending() > 64) { /* 处理能力不足 */ }
```

注意事项：

该值持续增长说明 poll 频率不足或单帧处理过长。

3.12 wvui 文本标签 API（WVUI_TYPE_LABEL）

3.12.1 wvui_label_set_text(wvui_widget_t *w, const char *text)

```
void wvui_label_set_text(wvui_widget_t *w, const char *text);
```

功能描述	设置标签文本并标记脏区。
输入参数	w：控件；text：UTF-8 字符串，不做拷贝。
输出参数	无
返回值	无

使用示例：

```
wvui_label_set_text(label, "25.4 C");
```

注意事项：

传入局部缓冲会导致悬空指针，必须使用常驻字符串。

3.12.2 wvui_label_set_font(wvui_widget_t *w, const wvui_font_t *font)

```
void wvui_label_set_font(wvui_widget_t *w, const wvui_font_t *font);
```

功能描述	设置文本字体，并重新计算布局。
输入参数	w：控件；font：字体描述，传 NULL 表示不绘制文字。
输出参数	无

返回值	无

使用示例：

```
wvui_label_set_font(label, &font_16);
```

3.12.3 wvui_label_set_color(wvui_widget_t *w, wvui_color_t color)

```
void wvui_label_set_color(wvui_widget_t *w, wvui_color_t color);
```

功能描述	设置文字颜色。
输入参数	w：控件；color：前景色。
输出参数	无
返回值	无

使用示例：

```
wvui_label_set_color(label, WVUI_COLOR_WHITE);
```

3.12.4 wvui_label_set_align(wvui_widget_t *w, uint8_t align)

```
void wvui_label_set_align(wvui_widget_t *w, uint8_t align);
```

功能描述	设置文本在控件宽度内的水平对齐方式。
输入参数	w：控件；align：左、中、右。
输出参数	无
返回值	无

使用示例：

```
wvui_label_set_align(label, WVUI_ALIGN_CENTER);
```

注意事项：

超出控件宽度时不会自动换行；如需换行，请使用 label_ex。

3.12.5 wvui_label_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_label_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_label_set_opa(label, 128);
```

3.12.6 wvui_label_text(const wvui_widget_t *w)

```
const char *wvui_label_text(const wvui_widget_t *w);
```

功能描述	读回当前文本指针。
输入参数	w：控件。
输出参数	无
返回值	文本指针；未设置返回 NULL。

使用示例：

```
const char *t = wvui_label_text(label);
```

3.13 wvui 自动换行标签 API（WVUI_TYPE_LABEL_EX）

3.13.1 wvui_label_ex_set_text(wvui_widget_t *w, const char *text)

```
void wvui_label_ex_set_text(wvui_widget_t *w, const char *text);
```

功能描述	设置文本，并按控件当前宽度重新自动换行。
输入参数	w：控件；text：UTF-8 字符串，不做拷贝。
输出参数	无
返回值	无

使用示例：

```
wvui_label_ex_set_text(desc, "当前温度高于设定上限，请检查风扇转速。");
```

注意事项：

换行按字体宽度逐字计算，多字节字符按整字断行；

文本必须常驻。

3.13.2 wvui_label_ex_set_font(wvui_widget_t *w, const wvui_font_t *font)

```
void wvui_label_ex_set_font(wvui_widget_t *w, const wvui_font_t *font);
```

功能描述	设置字体并重新换行。
输入参数	w：控件；font：字体描述。
输出参数	无
返回值	无

使用示例：

```
wvui_label_ex_set_font(desc, &font_12);
```

3.13.3 wvui_label_ex_set_color(wvui_widget_t *w, wvui_color_t color)

```
void wvui_label_ex_set_color(wvui_widget_t *w, wvui_color_t color);
```

功能描述	设置文字颜色。
输入参数	w：控件；color：前景色。
输出参数	无
返回值	无

使用示例：

```
wvui_label_ex_set_color(desc, WVUI_COLOR_GRAY);
```

3.13.4 wvui_label_ex_set_align(wvui_widget_t *w, uint8_t align)

```
void wvui_label_ex_set_align(wvui_widget_t *w, uint8_t align);
```

功能描述	设置每一行的水平对齐方式。
输入参数	w：控件；align：左、中、右。
输出参数	无
返回值	无

使用示例：

```
wvui_label_ex_set_align(desc, WVUI_ALIGN_LEFT);
```

3.13.5 wvui_label_ex_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_label_ex_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_label_ex_set_opa(desc, 200);
```

3.13.6 wvui_label_ex_refresh(wvui_widget_t *w)

```
void wvui_label_ex_refresh(wvui_widget_t *w);
```

功能描述	按当前宽度重新换行并重算行表。
输入参数	w：控件。

输出参数	无
返回值	无

使用示例：

```
wvui_widget_set_pos(desc, 0, 40);   /* 宽度变化后 */
wvui_label_ex_refresh(desc);
```

注意事项：

仅修改坐标不会触发重新换行，需显式调用本函数。

3.13.7 wvui_label_ex_text(const wvui_widget_t *w)

```
const char *wvui_label_ex_text(const wvui_widget_t *w);
```

功能描述	读回当前文本指针。
输入参数	w：控件。
输出参数	无
返回值	文本指针；未设置返回 NULL。

使用示例：

```
const char *t = wvui_label_ex_text(desc);
```

3.13.8 wvui_label_ex_line_count(const wvui_widget_t *w)

```
uint8_t wvui_label_ex_line_count(const wvui_widget_t *w);
```

功能描述	读回实际排版后的行数。
输入参数	w：控件。
输出参数	无
返回值	行数。

使用示例：

```
uint8_t n = wvui_label_ex_line_count(desc);
```

注意事项：

行数受控件宽度影响，宽度改变后需先 refresh 再读取。

3.13.9 wvui_label_ex_truncated(const wvui_widget_t *w)

```
uint8_t wvui_label_ex_truncated(const wvui_widget_t *w);
```

功能描述	读回是否因高度不足发生截断。
输入参数	w：控件。

输出参数	无
返回值	1 表示已截断。

使用示例：

```
if (wvui_label_ex_truncated(desc)) { /* 提示内容过长 */ }
```

3.13.10 wvui_label_ex_line_width(const wvui_widget_t *w, uint8_t index)

```
int16_t wvui_label_ex_line_width(const wvui_widget_t *w, uint8_t index);
```

功能描述	获取指定行的像素宽度。
输入参数	w：控件；index：行号，从 0 开始。
输出参数	无
返回值	像素宽度；越界返回 0。

使用示例：

```
int16_t w = wvui_label_ex_line_width(desc, 0);
```

3.14 wvui 按钮 API（WVUI_TYPE_BTN）

3.14.1 wvui_btn_set_text(wvui_widget_t *w, const char *text)

```
void wvui_btn_set_text(wvui_widget_t *w, const char *text);
```

功能描述	设置按钮文本并标脏。
输入参数	w：控件；text：文本，不拷贝。
输出参数	无
返回值	无

使用示例：

```
wvui_btn_set_text(btn, "确定");
```

注意事项：

文本必须常驻，不能用栈上缓冲。

3.14.2 wvui_btn_set_font(wvui_widget_t *w, const wvui_font_t *font)

```
void wvui_btn_set_font(wvui_widget_t *w, const wvui_font_t *font);
```

功能描述	设置按钮文本字体。
输入参数	w：控件；font：字体描述。
输出参数	无

返回值	无

使用示例：

```
wvui_btn_set_font(btn, &font_16);
```

3.14.3 wvui_btn_set_colors(wvui_widget_t *w, wvui_color_t bg, wvui_color_t bg_pressed, wvui_color_t fg)

```
void wvui_btn_set_colors(wvui_widget_t *w, wvui_color_t bg, wvui_color_t bg_pressed, wvui_color_t fg);
```

功能描述	设置常态底色、按下底色与文字颜色。
输入参数	w：控件；bg：常态底色；bg_pressed：按下底色；fg：文字色。
输出参数	无
返回值	无

使用示例：

```
wvui_btn_set_colors(btn, C_BLUE, C_DARKBLUE, WVUI_COLOR_WHITE);
```

3.14.4 wvui_btn_set_border(wvui_widget_t *w, wvui_color_t border)

```
void wvui_btn_set_border(wvui_widget_t *w, wvui_color_t border);
```

功能描述	设置按钮边框颜色。
输入参数	w：控件；border：边框色。
输出参数	无
返回值	无

使用示例：

```
wvui_btn_set_border(btn, C_GRAY);
```

3.14.5 wvui_btn_set_radius(wvui_widget_t *w, uint8_t radius)

```
void wvui_btn_set_radius(wvui_widget_t *w, uint8_t radius);
```

功能描述	设置圆角半径，绘制时自动夹紧至控件短边的一半。
输入参数	w：控件；radius：半径像素。
输出参数	无
返回值	无

使用示例：

```
wvui_btn_set_radius(btn, 6);
```

3.14.6 wvui_btn_radius(const wvui_widget_t *w)

```
uint8_t wvui_btn_radius(const wvui_widget_t *w);
```

功能描述	读回圆角半径。
输入参数	w：控件。
输出参数	无
返回值	半径像素。

使用示例：

```
uint8_t r = wvui_btn_radius(btn);
```

3.14.7 wvui_btn_set_enabled(wvui_widget_t *w, uint8_t enabled)

```
void wvui_btn_set_enabled(wvui_widget_t *w, uint8_t enabled);
```

功能描述	设置可用状态，禁用时变灰且不响应点击。
输入参数	w：控件；enabled：非 0 表示可用。
输出参数	无
返回值	无

使用示例：

```
wvui_btn_set_enabled(btn, 0);   /* 条件未满足时禁用 */
```

注意事项：

禁用状态下点击回调不会被触发。

3.14.8 wvui_btn_set_click_cb(wvui_widget_t *w, wvui_btn_click_cb_t cb, void *user)

```
void wvui_btn_set_click_cb(wvui_widget_t *w, wvui_btn_click_cb_t cb, void *user);
```

功能描述	设置点击回调及透传参数。
输入参数	w：控件；cb：回调，NULL 表示无；user：透传给回调的指针。
输出参数	无
返回值	无

使用示例：

```
static void on_ok(wvui_widget_t *w, void *u) { /* ... */ }
wvui_btn_set_click_cb(btn, on_ok, NULL);
```

注意事项：

回调在输入处理阶段执行，禁止在内部删除控件自身。

3.14.9 wvui_btn_is_pressed(const wvui_widget_t *w)

```
uint8_t wvui_btn_is_pressed(const wvui_widget_t *w);
```

功能描述	读回按钮当前是否处于按下状态。
输入参数	w：控件。
输出参数	无
返回值	1 表示按下。

使用示例：

```
if (wvui_btn_is_pressed(btn)) { /* 常按检测 */ }
```

3.15 wvui 盒子 API（WVUI_TYPE_BOX）

3.15.1 wvui_box_set_colors(wvui_widget_t *w, wvui_color_t fill, wvui_color_t border)

```
void wvui_box_set_colors(wvui_widget_t *w, wvui_color_t fill, wvui_color_t border);
```

功能描述	设置盒子的填充色与边框色。
输入参数	w：控件；fill：填充色；border：边框色。
输出参数	无
返回值	无

使用示例：

```
wvui_box_set_colors(card, C_WHITE, C_GRAY);
```

3.15.2 wvui_box_set_corners(wvui_widget_t *w, uint8_t lt, uint8_t rt, uint8_t lb, uint8_t rb)

```
void wvui_box_set_corners(wvui_widget_t *w, uint8_t lt, uint8_t rt, uint8_t lb, uint8_t rb);
```

功能描述	一次性设置四个角的样式。
输入参数	w：控件；lt、rt、lb、rb：左上、右上、左下、右下样式。
输出参数	无
返回值	无

使用示例：

```
wvui_box_set_corners(card, BOX_ROUND, BOX_ROUND, BOX_SQUARE, BOX_SQUARE);
```

注意事项：

四个角可混合采用圆角、切角与直角，半径由 set_radius 统一决定。

3.15.3 wvui_box_set_corner(wvui_widget_t *w, uint8_t corner, uint8_t style)

```
void wvui_box_set_corner(wvui_widget_t *w, uint8_t corner, uint8_t style);
```

功能描述	单独设置一个角的样式。
输入参数	w：控件；corner：WVUI_BOX_LT/RT/LB/RB；style：样式。
输出参数	无
返回值	无

使用示例：

```
wvui_box_set_corner(card, WVUI_BOX_LB, BOX_ROUND);
```

3.15.4 wvui_box_set_radius(wvui_widget_t *w, uint8_t radius)

```
void wvui_box_set_radius(wvui_widget_t *w, uint8_t radius);
```

功能描述	设置圆角半径或切角长度。
输入参数	w：控件；radius：像素值。
输出参数	无
返回值	无

使用示例：

```
wvui_box_set_radius(card, 8);
```

3.15.5 wvui_box_set_border_width(wvui_widget_t *w, uint8_t bw)

```
void wvui_box_set_border_width(wvui_widget_t *w, uint8_t bw);
```

功能描述	设置边框宽度。
输入参数	w：控件；bw：宽度像素，0 表示无边框。
输出参数	无
返回值	无

使用示例：

```
wvui_box_set_border_width(card, 1);
```

3.15.6 wvui_box_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_box_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。

输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_box_set_opa(card, 220);
```

3.15.7 wvui_box_pack_corners(uint8_t lt, uint8_t rt, uint8_t lb, uint8_t rb)

```
uint8_t wvui_box_pack_corners(uint8_t lt, uint8_t rt, uint8_t lb, uint8_t rb);
```

功能描述	将四个角的样式打包进一个字节，便于放入描述表。
输入参数	lt、rt、lb、rb：四个角样式，各占 2 位。
输出参数	无
返回值	打包后的字节。

使用示例：

```
uint8_t c = wvui_box_pack_corners(1, 1, 0, 0);
```

3.15.8 wvui_box_corner_of(uint8_t packed, uint8_t corner)

```
uint8_t wvui_box_corner_of(uint8_t packed, uint8_t corner);
```

功能描述	从打包字节中取出指定角的样式。
输入参数	packed：打包值；corner：角索引。
输出参数	无
返回值	该角样式。

使用示例：

```
uint8_t s = wvui_box_corner_of(c, WVUI_BOX_RB);
```

3.16 wvui 线段 API（WVUI_TYPE_LINE）

3.16.1 wvui_line_set_points(wvui_widget_t *w, int16_t x0, int16_t y0, int16_t x1, int16_t y1)

```
void wvui_line_set_points(wvui_widget_t *w, int16_t x0, int16_t y0, int16_t x1, int16_t y1);
```

功能描述	设置线段两端点，坐标为屏幕绝对坐标。
输入参数	w：控件；x0、y0：起点；x1、y1：终点。
输出参数	无

返回值	无

使用示例：

```
wvui_line_set_points(sep, 10, 40, 200, 40);
```

注意事项：

端点不受控件矩形限制，控件仅决定绘制顺序与可见性。

3.16.2 wvui_line_reset_points(wvui_widget_t *w)

```
void wvui_line_reset_points(wvui_widget_t *w);
```

功能描述	按控件矩形重新计算默认端点（水平分隔线）。
输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_line_reset_points(sep);
```

3.16.3 wvui_line_set_width(wvui_widget_t *w, uint8_t width)

```
void wvui_line_set_width(wvui_widget_t *w, uint8_t width);
```

功能描述	设置线宽。
输入参数	w：控件；width：像素，0 表示不绘制。
输出参数	无
返回值	无

使用示例：

```
wvui_line_set_width(sep, 2);
```

3.16.4 wvui_line_set_cap(wvui_widget_t *w, uint8_t cap)

```
void wvui_line_set_cap(wvui_widget_t *w, uint8_t cap);
```

功能描述	设置端点样式（平头或圆头）。
输入参数	w：控件；cap：端点样式。
输出参数	无
返回值	无

使用示例：

```
wvui_line_set_cap(sep, WVUI_LINE_CAP_ROUND);
```

3.16.5 wvui_line_set_color(wvui_widget_t *w, wvui_color_t color)

```
void wvui_line_set_color(wvui_widget_t *w, wvui_color_t color);
```

功能描述	设置线段颜色。
输入参数	w：控件；color：颜色。
输出参数	无
返回值	无

使用示例：

```
wvui_line_set_color(sep, C_GRAY);
```

3.16.6 wvui_line_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_line_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_line_set_opa(sep, 128);
```

3.17 wvui 进度条 API（WVUI_TYPE_PROGRESS）

3.17.1 wvui_progress_set_value(wvui_widget_t *w, uint8_t value)

```
void wvui_progress_set_value(wvui_widget_t *w, uint8_t value);
```

功能描述	设置目标值；与当前显示值不同时启动过渡动画。
输入参数	w：控件；value：目标值，取值 0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_progress_set_value(bar, 128);
```

注意事项：

过渡按帧推进；频繁改写目标值会不断重置动画，建议按固定周期更新。

3.17.2 wvui_progress_add(wvui_widget_t *w, uint8_t delta)

```
void wvui_progress_add(wvui_widget_t *w, uint8_t delta);
```

功能描述	目标值增加，超过 255 时饱和。
输入参数	w：控件；delta：增量。
输出参数	无
返回值	无

使用示例：

```
wvui_progress_add(bar, 10);
```

3.17.3 wvui_progress_sub(wvui_widget_t *w, uint8_t delta)

```
void wvui_progress_sub(wvui_widget_t *w, uint8_t delta);
```

功能描述	目标值减少，低于 0 时饱和。
输入参数	w：控件；delta：减量。
输出参数	无
返回值	无

使用示例：

```
wvui_progress_sub(bar, 5);
```

3.17.4 wvui_progress_set_colors(wvui_widget_t *w, wvui_color_t track, wvui_color_t fill)

```
void wvui_progress_set_colors(wvui_widget_t *w, wvui_color_t track, wvui_color_t fill);
```

功能描述	设置轨道色与填充色。
输入参数	w：控件；track：轨道色；fill：填充色。
输出参数	无
返回值	无

使用示例：

```
wvui_progress_set_colors(bar, C_LGRAY, C_GREEN);
```

3.17.5 wvui_progress_set_radius(wvui_widget_t *w, uint8_t radius)

```
void wvui_progress_set_radius(wvui_widget_t *w, uint8_t radius);
```

功能描述	设置圆角半径。
输入参数	w：控件；radius：像素。

输出参数	无
返回值	无

使用示例：

```
wvui_progress_set_radius(bar, 4);
```

3.17.6 wvui_progress_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_progress_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_progress_set_opa(bar, 255);
```

3.17.7 wvui_progress_value(const wvui_widget_t *w)

```
uint8_t wvui_progress_value(const wvui_widget_t *w);
```

功能描述	读回目标值。
输入参数	w：控件。
输出参数	无
返回值	目标值 0..255。

使用示例：

```
uint8_t v = wvui_progress_value(bar);
```

3.17.8 wvui_progress_display(const wvui_widget_t *w)

```
uint8_t wvui_progress_display(const wvui_widget_t *w);
```

功能描述	读回当前显示值，即动画中间态。
输入参数	w：控件。
输出参数	无
返回值	显示值 0..255。

使用示例：

```
uint8_t d = wvui_progress_display(bar);
```

注意事项：

动画未结束时该值与目标值不同，可用于判断过渡是否完成。

3.18 wvui 开关 API（WVUI_TYPE_TOGGLE）

3.18.1 wvui_toggle_set_on(wvui_widget_t *w, uint8_t on)

```
void wvui_toggle_set_on(wvui_widget_t *w, uint8_t on);
```

功能描述	设置目标状态，并启动滑块位移动画。
输入参数	w：控件；on：非 0 表示开。
输出参数	无
返回值	无

使用示例：

```
wvui_toggle_set_on(sw, 1);
```

注意事项：

状态未变化时为空操作，不会重复触发变化回调。

3.18.2 wvui_toggle_flip(wvui_widget_t *w)

```
void wvui_toggle_flip(wvui_widget_t *w);
```

功能描述	翻转目标状态。
输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_toggle_flip(sw);
```

注意事项：

翻转目标状态并重新启动滑块位移动画。

与 set_on 一致，状态未变化时不触发变化回调。

3.18.3 wvui_toggle_set_radius(wvui_widget_t *w, uint8_t radius)

```
void wvui_toggle_set_radius(wvui_widget_t *w, uint8_t radius);
```

功能描述	设置圆角半径。
输入参数	w：控件；radius：像素。
输出参数	无
返回值	无

使用示例：

```
wvui_toggle_set_radius(sw, 10);
```

注意事项：

半径通常取控件高度的一半，即可得到胶囊形轨道。

取值大于短边一半时按半圆处理。

3.18.4 wvui_toggle_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_toggle_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_toggle_set_opa(sw, 255);
```

注意事项：

轨道与滑块同时受该值影响。

禁用状态会在此基础上再叠加灰化，二者独立生效。

3.18.5 wvui_toggle_set_enabled(wvui_widget_t *w, uint8_t enabled)

```
void wvui_toggle_set_enabled(wvui_widget_t *w, uint8_t enabled);
```

功能描述	设置可用状态；禁用时不响应点击且显示为灰色。
输入参数	w：控件；enabled：非 0 表示可用。
输出参数	无
返回值	无

使用示例：

```
wvui_toggle_set_enabled(sw, 0);
```

注意事项：

禁用后不响应点按与拖动，当前状态保持不变。

灰化显示与 opa 设置相互独立。

3.18.6 wvui_toggle_set_change_cb(wvui_widget_t *w, wvui_toggle_change_cb_t cb, void *user)

```
void wvui_toggle_set_change_cb(wvui_widget_t *w, wvui_toggle_change_cb_t cb, void *user);
```

功能描述	设置状态变化回调及透传参数。
输入参数	w：控件；cb：回调；user：透传指针。

输出参数	无
返回值	无

使用示例：

```
static void on_sw(wvui_widget_t *w, void *u) { /* ... */ }
wvui_toggle_set_change_cb(sw, on_sw, NULL);
```

注意事项：

回调在输入处理阶段执行，禁止在内部删除控件自身。

3.18.7 wvui_toggle_is_on(const wvui_widget_t *w)

```
uint8_t wvui_toggle_is_on(const wvui_widget_t *w);
```

功能描述	读回目标状态。
输入参数	w：控件。
输出参数	无
返回值	1 表示开。

使用示例：

```
if (wvui_toggle_is_on(sw)) { /* 已开启 */ }
```

注意事项：

返回目标状态，滑块位移动画的中间态不影响该值。

只读接口，不触发变化回调。

3.18.8 wvui_toggle_is_pressed(const wvui_widget_t *w)

```
uint8_t wvui_toggle_is_pressed(const wvui_widget_t *w);
```

功能描述	读回按下状态。
输入参数	w：控件。
输出参数	无
返回值	1 表示按下。

使用示例：

```
if (wvui_toggle_is_pressed(sw)) { /* 常按 */ }
```

注意事项：

仅按下期间为 1，与开关当前状态无关。

松手后状态才切换，可用作按压态样式的切换依据。

3.18.9 wvui_toggle_knob_pos(const wvui_widget_t *w)

```
uint8_t wvui_toggle_knob_pos(const wvui_widget_t *w);
```

功能描述	读回滑块当前位置，即动画中间态。
输入参数	w：控件。
输出参数	无
返回值	位置值，0 为关端，255 为开端。

使用示例：

```
uint8_t k = wvui_toggle_knob_pos(sw);
```

注意事项：

0 表示完全关、255 表示完全开；动画过程中返回中间值。

可用于联动其他控件进行同步过渡，常规渲染无需读取。

3.19 wvui 复选框 API（WVUI_TYPE_CHECKBOX）

3.19.1 wvui_checkbox_set_on(wvui_widget_t *w, uint8_t on)

```
void wvui_checkbox_set_on(wvui_widget_t *w, uint8_t on);
```

功能描述	设置目标选中状态，并启动对勾描边动画。
输入参数	w：控件；on：非 0 选中。
输出参数	无
返回值	无

使用示例：

```
wvui_checkbox_set_on(ck, 1);
```

注意事项：

状态未变化时为空操作，不会重复触发变化回调。

3.19.2 wvui_checkbox_flip(wvui_widget_t *w)

```
void wvui_checkbox_flip(wvui_widget_t *w);
```

功能描述	翻转目标选中状态。
输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_checkbox_flip(ck);
```

注意事项：

翻转目标状态，与 set_on 一致，不触发变化回调。

会重新播放对勾描边动画，可配合 progress 观察过渡过程。

3.19.3 wvui_checkbox_set_colors(wvui_widget_t *w, wvui_color_t bg, wvui_color_t box, wvui_color_t mark, wvui_color_t border)

```
void wvui_checkbox_set_colors(wvui_widget_t *w, wvui_color_t bg, wvui_color_t box, wvui_color_t mark,
wvui_color_t border);
```

功能描述	一次性设置背景色、方框填充色、对勾色与边框色。
输入参数	w：控件；bg：背景色；box：方框色；mark：对勾色；border：边框色。
输出参数	无
返回值	无

使用示例：

```
wvui_checkbox_set_colors(ck, C_WHITE, C_BLUE, C_WHITE, C_GRAY);
```

注意事项：

四个参数依次为整体背景、方框填充、对勾与边框色，缺一不可。

背景色仅在控件矩形内可见，控件外仍由页面背景决定。

3.19.4 wvui_checkbox_set_radius(wvui_widget_t *w, uint8_t radius)

```
void wvui_checkbox_set_radius(wvui_widget_t *w, uint8_t radius);
```

功能描述	设置方框圆角半径。
输入参数	w：控件；radius：像素。
输出参数	无
返回值	无

使用示例：

```
wvui_checkbox_set_radius(ck, 3);
```

注意事项：

圆角只作用于方框，对勾与文字不受影响。

半径不应超过方框边长的一半。

3.19.5 wvui_checkbox_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_checkbox_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无

返回值	无

使用示例：

```
wvui_checkbox_set_opa(ck, 255);
```

注意事项：

作用于背景、方框与对勾整体。

禁用状态会在此基础上再叠加灰化效果，两者相互独立。

3.19.6 wvui_checkbox_set_enabled(wvui_widget_t *w, uint8_t enabled)

```
void wvui_checkbox_set_enabled(wvui_widget_t *w, uint8_t enabled);
```

功能描述	设置可用状态；禁用时不响应点击且显示为灰色。
输入参数	w：控件；enabled：非 0 表示可用。
输出参数	无
返回值	无

使用示例：

```
wvui_checkbox_set_enabled(ck, 0);
```

注意事项：

禁用后不响应点按，当前选中状态保持不变。

灰化显示与 opa 设置相互独立；恢复使能后 opacity 不变。

3.19.7 wvui_checkbox_set_change_cb(wvui_widget_t *w, wvui_checkbox_change_cb_t cb, void *user)

```
void wvui_checkbox_set_change_cb(wvui_widget_t *w, wvui_checkbox_change_cb_t cb, void *user);
```

功能描述	设置状态变化回调及透传参数。
输入参数	w：控件；cb：回调；user：透传指针。
输出参数	无
返回值	无

使用示例：

```
static void on_ck(wvui_widget_t *w, void *u) { /* ... */ }
wvui_checkbox_set_change_cb(ck, on_ck, NULL);
```

注意事项：

回调在输入处理阶段执行，禁止在内部删除控件自身。

3.19.8 wvui_checkbox_is_on(const wvui_widget_t *w)

```
uint8_t wvui_checkbox_is_on(const wvui_widget_t *w);
```

功能描述	读回目标选中状态。

输入参数	w：控件。
输出参数	无
返回值	1 表示选中。

使用示例：

```
if (wvui_checkbox_is_on(ck)) { /* 已选中 */ }
```

注意事项：

返回目标选中状态，描边动画的中间态不影响该值。

只读接口，不触发回调。

3.19.9 wvui_checkbox_is_pressed(const wvui_widget_t *w)

```
uint8_t wvui_checkbox_is_pressed(const wvui_widget_t *w);
```

功能描述	读回按下状态。
输入参数	w：控件。
输出参数	无
返回值	1 表示按下。

使用示例：

```
if (wvui_checkbox_is_pressed(ck)) { /* 常按 */ }
```

注意事项：

仅按下期间为 1，与是否已选中无关。

松开后才完成状态切换，可用其实现按压反馈。

3.19.10 wvui_checkbox_progress(const wvui_widget_t *w)

```
uint8_t wvui_checkbox_progress(const wvui_widget_t *w);
```

功能描述	读回对勾描边进度，即动画中间态。
输入参数	w：控件。
输出参数	无
返回值	取值 0..255，0 表示无对勾，255 表示完整。

使用示例：

```
uint8_t p = wvui_checkbox_progress(ck);
```

注意事项：

返回 0..255 的对勾描边进度，255 表示动画已完成。

供自定义绘制或外部同步使用，常规渲染无需读取。

3.20 wvui 指示灯 API（WVUI_TYPE_INDICATOR）

3.20.1 wvui_indicator_set_on(wvui_widget_t *w, uint8_t on)

```
void wvui_indicator_set_on(wvui_widget_t *w, uint8_t on);
```

功能描述	设置逻辑状态，决定背景使用点亮色还是熄灭色。
输入参数	w：控件；on：非 0 点亮。
输出参数	无
返回值	无

使用示例：

```
wvui_indicator_set_on(led, 1);
```

注意事项：

仅改变逻辑状态，不触发变化回调；回调仅在可点按翻转时产生。

开启闪烁时状态不变，仍按闪烁节奏显示。

3.20.2 wvui_indicator_flip(wvui_widget_t *w)

```
void wvui_indicator_flip(wvui_widget_t *w);
```

功能描述	翻转逻辑状态。
输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_indicator_flip(led);
```

注意事项：

翻转目标状态，与 set_on 一致，不触发变化回调。

可在定时任务中周期调用，实现与内置闪烁不同的慢节奏交替。

3.20.3 wvui_indicator_set_blink(wvui_widget_t *w, uint8_t blink, uint8_t period_frames)

```
void wvui_indicator_set_blink(wvui_widget_t *w, uint8_t blink, uint8_t period_frames);
```

功能描述	设置闪烁开关与闪烁周期。
输入参数	w：控件；blink：非 0 表示闪烁；period_frames：周期帧数。
输出参数	无
返回值	无

使用示例：

```
wvui_indicator_set_blink(led, 1, 30);
```

注意事项：

闪烁仅改变可见性，不改变逻辑状态；周期按帧计数，与刷新率相关。

3.20.4 wvui_indicator_set_colors(wvui_widget_t *w, wvui_color_t off, wvui_color_t on)

```
void wvui_indicator_set_colors(wvui_widget_t *w, wvui_color_t off, wvui_color_t on);
```

功能描述	设置熄灭色与点亮色。
输入参数	w：控件；off：熄灭色；on：点亮色。
输出参数	无
返回值	无

使用示例：

```
wvui_indicator_set_colors(led, C_GRAY, C_GREEN);
```

注意事项：

熄灭色建议取浅灰或与背景相近的颜色，点亮色宜用高饱和色以便区分。

两色均参与透明度叠加，不透明度设置会影响最终观感。

3.20.5 wvui_indicator_set_radius(wvui_widget_t *w, uint8_t radius)

```
void wvui_indicator_set_radius(wvui_widget_t *w, uint8_t radius);
```

功能描述	设置圆角半径。
输入参数	w：控件；radius：像素。
输出参数	无
返回值	无

使用示例：

```
wvui_indicator_set_radius(led, 6);
```

注意事项：

半径超过控件短边一半时按半圆处理，不会越界绘制。

指示灯一般为方形或圆形，取短边一半即可得到圆点效果。

3.20.6 wvui_indicator_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_indicator_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无

返回值	无

使用示例：

```
wvui_indicator_set_opa(led, 255);
```

注意事项：

与闪烁淡出效果叠加相乘，闪烁相位会在此基础上继续衰减。

设为 0 时不可见，但逻辑状态仍正常维护。

3.20.7 wvui_indicator_set_clickable(wvui_widget_t *w, uint8_t clickable)

```
void wvui_indicator_set_clickable(wvui_widget_t *w, uint8_t clickable);
```

功能描述	设置是否拦截手势；开启后点按可翻转状态。
输入参数	w：控件；clickable：非 0 表示可点按。
输出参数	无
返回值	无

使用示例：

```
wvui_indicator_set_clickable(led, 1);
```

注意事项：

默认不拦截手势，指示灯仅作为显示元素，不参与点击命中。

3.20.8 wvui_indicator_set_change_cb(wvui_widget_t *w, wvui_indicator_change_cb_t cb, void *user)

```
void wvui_indicator_set_change_cb(wvui_widget_t *w, wvui_indicator_change_cb_t cb, void *user);
```

功能描述	设置状态变化回调及透传参数。
输入参数	w：控件；cb：回调；user：透传指针。
输出参数	无
返回值	无

使用示例：

```
static void on_led(wvui_widget_t *w, void *u) { /* ... */ }
wvui_indicator_set_change_cb(led, on_led, NULL);
```

注意事项：

仅在可点按并发生翻转时触发；调用 set_on 设置同一状态不触发。

3.20.9 wvui_indicator_is_on(const wvui_widget_t *w)

```
uint8_t wvui_indicator_is_on(const wvui_widget_t *w);
```

功能描述	读回逻辑状态。

输入参数	w：控件。
输出参数	无
返回值	1 表示点亮。

使用示例：

```
if (wvui_indicator_is_on(led)) { /* 点亮 */ }
```

注意事项：

返回目标逻辑状态，闪烁相位不影响该值。

与 visible 的区别在于：后者反映当前相位是否需要绘制。

3.20.10 wvui_indicator_is_pressed(const wvui_widget_t *w)

```
uint8_t wvui_indicator_is_pressed(const wvui_widget_t *w);
```

功能描述	读回按下状态。
输入参数	w：控件。
输出参数	无
返回值	1 表示按下。

使用示例：

```
if (wvui_indicator_is_pressed(led)) { /* 常按 */ }
```

注意事项：

未开启 clickable 时该接口恒为 0。

按下不等于翻转，松手后才完成状态切换。

3.20.11 wvui_indicator_visible(const wvui_widget_t *w)

```
uint8_t wvui_indicator_visible(const wvui_widget_t *w);
```

功能描述	获取当前闪烁相位下是否应绘制。
输入参数	w：控件。
输出参数	无
返回值	1 表示可见。

使用示例：

```
uint8_t v = wvui_indicator_visible(led);
```

注意事项：

未开启闪烁时恒为 1；返回 0 仅表示当前相位应跳过绘制。

主要用于自定义绘制流程，常规渲染无需调用。

注意事项：

未开启闪烁时恒返回 1；该接口供自定义绘制使用，正常渲染无需调用。

3.21 wvui 圆弧 API（WVUI_TYPE_ARC）

3.21.1 wvui_arc_set_angles(wvui_widget_t *w, uint16_t a0, uint16_t a1)

```
void wvui_arc_set_angles(wvui_widget_t *w, uint16_t a0, uint16_t a1);
```

功能描述	设置圆弧起始角与结束角；角度以 512 步表示一整周，按顺时针递增。
输入参数	w：控件；a0：起始角；a1：结束角，可小于 a0 以表示跨 0 度。
输出参数	无
返回值	无

使用示例：

```
wvui_arc_set_angles(arc, 0, 128);
```

注意事项：

角度按 512 步一整周换算，128 约相当于 90 度；a0 与 a1 相等时不绘制任何内容。

3.21.2 wvui_arc_set_color(wvui_widget_t *w, wvui_color_t color)

```
void wvui_arc_set_color(wvui_widget_t *w, wvui_color_t color);
```

功能描述	设置弧线颜色。
输入参数	w：控件；color：颜色。
输出参数	无
返回值	无

使用示例：

```
wvui_arc_set_color(arc, C_BLUE);
```

注意事项：

颜色对整段弧线生效，不区分内外侧。

如需底槽效果，可再叠加一条灰色且更宽的圆弧作为背景。

3.21.3 wvui_arc_set_width(wvui_widget_t *w, uint8_t width)

```
void wvui_arc_set_width(wvui_widget_t *w, uint8_t width);
```

功能描述	设置环宽，即弧线的径向厚度。
输入参数	w：控件；width：像素。
输出参数	无
返回值	无

使用示例：

```
wvui_arc_set_width(arc, 8);
```

注意事项：

环宽应小于外半径，否则将退化为实心扇形。

3.21.4 wvui_arc_set_radius(wvui_widget_t *w, uint8_t radius)

```
void wvui_arc_set_radius(wvui_widget_t *w, uint8_t radius);
```

功能描述	设置外半径。
输入参数	w：控件；radius：像素。
输出参数	无
返回值	无

使用示例：

```
wvui_arc_set_radius(arc, 40);
```

注意事项：

半径以控件中心为圆心，超出控件矩形的部分将被裁剪。

3.21.5 wvui_arc_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_arc_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_arc_set_opa(arc, 255);
```

注意事项：

弧线整体受该值影响，最终效果还需与控件级及分组级不透明度相乘。

设为 0 时不可见但仍占布局位置。

3.21.6 wvui_arc_start(const wvui_widget_t *w)

```
uint16_t wvui_arc_start(const wvui_widget_t *w);
```

功能描述	读回起始角。
输入参数	w：控件。
输出参数	无
返回值	起始角，512 步圆周。

使用示例：

```
uint16_t a0 = wvui_arc_start(arc);
```

注意事项：

读回起始角，单位为 512 步圆周。

只读接口，不触发重绘。

3.21.7 wvui_arc_end(const wvui_widget_t *w)

```
uint16_t wvui_arc_end(const wvui_widget_t *w);
```

功能描述	读回结束角。
输入参数	w：控件。
输出参数	无
返回值	结束角，512 步圆周。

使用示例：

```
uint16_t a1 = wvui_arc_end(arc);
```

注意事项：

读回结束角；跨 0 度时返回设定值而非归一化结果。

只读接口，可与 start 一起用于计算弧长比例。

3.22 wvui 滑块 API（WVUI_TYPE_SLIDER）

3.22.1 wvui_slider_set_value(wvui_widget_t *w, uint8_t value)

```
void wvui_slider_set_value(wvui_widget_t *w, uint8_t value);
```

功能描述	设置滑块当前值。
输入参数	w：控件；value：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_slider_set_value(sd, 128);
```

注意事项：

该接口按程序设值处理，不触发值变化回调。

3.22.2 wvui_slider_set_radius(wvui_widget_t *w, uint8_t radius)

```
void wvui_slider_set_radius(wvui_widget_t *w, uint8_t radius);
```

功能描述	设置轨道圆角半径。
输入参数	w：控件；radius：像素。

输出参数	无
返回值	无

使用示例：

```
wvui_slider_set_radius(sd, 4);
```

注意事项：

圆角仅作用于轨道，滑块旋钮本身始终为圆形。

3.22.3 wvui_slider_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_slider_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_slider_set_opa(sd, 255);
```

注意事项：

轨道与旋钮同时受该值影响；禁用时会在此基础上再压暗。

3.22.4 wvui_slider_set_enabled(wvui_widget_t *w, uint8_t enabled)

```
void wvui_slider_set_enabled(wvui_widget_t *w, uint8_t enabled);
```

功能描述	设置可用状态；禁用时不响应拖动且显示为灰色。
输入参数	w：控件；enabled：非 0 表示可用。
输出参数	无
返回值	无

使用示例：

```
wvui_slider_set_enabled(sd, 0);
```

注意事项：

禁用后不再响应拖动，当前值保持，可用于条件未满足时锁定调节。

3.22.5 wvui_slider_set_change_cb(wvui_widget_t *w, wvui_slider_change_cb_t cb, void *user)

```
void wvui_slider_set_change_cb(wvui_widget_t *w, wvui_slider_change_cb_t cb, void *user);
```

功能描述	设置值变化回调，拖动过程中持续触发。

输入参数	w：控件；cb：回调；user：透传指针。
输出参数	无
返回值	无

使用示例：

```
static void on_sd(wvui_widget_t *w, void *u) { /* ... */ }
wvui_slider_set_change_cb(sd, on_sd, NULL);
```

注意事项：

拖动时每个变化的值都会回调一次，回调内不宜执行耗时操作。

3.22.6 wvui_slider_value(const wvui_widget_t *w)

```
uint8_t wvui_slider_value(const wvui_widget_t *w);
```

功能描述	读回当前值。
输入参数	w：控件。
输出参数	无
返回值	当前值 0..255。

使用示例：

```
uint8_t v = wvui_slider_value(sd);
```

注意事项：

拖动过程中实时更新；只读调用不会触发值变化回调。

3.22.7 wvui_slider_is_pressed(const wvui_widget_t *w)

```
uint8_t wvui_slider_is_pressed(const wvui_widget_t *w);
```

功能描述	读回按下状态。
输入参数	w：控件。
输出参数	无
返回值	1 表示按下。

使用示例：

```
if (wvui_slider_is_pressed(sd)) { /* 正在拖动 */ }
```

注意事项：

按住旋钮但未移动时也为 1，可用于区分"按下"与"值已改变"。

3.23 wvui 仪表 API（WVUI_TYPE_GAUGE）

3.23.1 wvui_gauge_set_value(wvui_widget_t *w, uint8_t value)

```
void wvui_gauge_set_value(wvui_widget_t *w, uint8_t value);
```

功能描述	设置目标值，指针按过渡动画转向新角度。
输入参数	w：控件；value：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_gauge_set_value(gg, 180);
```

注意事项：

数值先按表盘起止角映射为角度，再参与动画；频繁改写会重置过渡。

3.23.2 wvui_gauge_set_angles(wvui_widget_t *w, uint16_t a0, uint16_t a1)

```
void wvui_gauge_set_angles(wvui_widget_t *w, uint16_t a0, uint16_t a1);
```

功能描述	设置表盘起止角，决定数值映射的角度范围。
输入参数	w：控件；a0：起始角；a1：结束角，采用 512 步圆周。
输出参数	无
返回值	无

使用示例：

```
wvui_gauge_set_angles(gg, 128, 384);
```

注意事项：

常用 180 度表盘取 128 至 384；跨 0 度时按顺时针方向解释。

3.23.3 wvui_gauge_set_width(wvui_widget_t *w, uint8_t width)

```
void wvui_gauge_set_width(wvui_widget_t *w, uint8_t width);
```

功能描述	设置弧环宽度。
输入参数	w：控件；width：像素。
输出参数	无
返回值	无

使用示例：

```
wvui_gauge_set_width(gg, 6);
```

注意事项：

环宽自外半径向内占用，宽度过大时指针的可视长度会被压缩。

3.23.4 wvui_gauge_set_needle(wvui_widget_t *w, uint8_t width)

```
void wvui_gauge_set_needle(wvui_widget_t *w, uint8_t width);
```

功能描述	设置指针粗细。
输入参数	w：控件；width：像素，0 表示不绘制指针。
输出参数	无
返回值	无

使用示例：

```
wvui_gauge_set_needle(gg, 3);
```

注意事项：

取 0 可隐藏指针，此时表盘仅靠弧环填充示意数值。

3.23.5 wvui_gauge_set_radius(wvui_widget_t *w, uint8_t radius)

```
void wvui_gauge_set_radius(wvui_widget_t *w, uint8_t radius);
```

功能描述	设置外半径。
输入参数	w：控件；radius：像素。
输出参数	无
返回值	无

使用示例：

```
wvui_gauge_set_radius(gg, 45);
```

注意事项：

半径以控件中心为圆心，超出控件矩形的部分将被裁掉。

3.23.6 wvui_gauge_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_gauge_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_gauge_set_opa(gg, 255);
```

注意事项：

表盘弧环、指针与刻度同时受该值影响。

3.23.7 wvui_gauge_value(const wvui_widget_t *w)

```
uint8_t wvui_gauge_value(const wvui_widget_t *w);
```

功能描述	读回目标值。
输入参数	w：控件。
输出参数	无
返回值	目标值 0..255。

使用示例：

```
uint8_t v = wvui_gauge_value(gg);
```

注意事项：

返回目标值；指针位移动画未结束时，与 angle 换算出的显示值不一致。

3.23.8 wvui_gauge_angle(const wvui_widget_t *w)

```
uint16_t wvui_gauge_angle(const wvui_widget_t *w);
```

功能描述	读回当前指针角度，即动画中间态。
输入参数	w：控件。
输出参数	无
返回值	角度，512 步圆周。

使用示例：

```
uint16_t a = wvui_gauge_angle(gg);
```

注意事项：

返回当前指针的实际角度（512 步圆周），可据此驱动外部联动指示。

3.24 wvui 折线图 API（WVUI_TYPE_CHART）

3.24.1 wvui_chart_bind(wvui_widget_t *w, const uint8_t *samples, uint16_t count)

```
uint8_t wvui_chart_bind(wvui_widget_t *w, const uint8_t *samples, uint16_t count);
```

功能描述	绑定外部样本数组，不拷贝数据，直接引用该缓冲区。
输入参数	w：控件；samples：样本数组首地址；count：样本数量。
输出参数	无
返回值	1 表示绑定成功，0 表示参数无效。

使用示例：

```
static uint8_t buf[64];
wvui_chart_bind(ch, buf, 64);
```

注意事项：

缓冲区必须常驻，且生命周期长于控件；控件销毁前不可释放。

绑定后写入数据需调用 wvui_chart_set 或 wvui_chart_push 标记刷新。

3.24.2 wvui_chart_set(wvui_widget_t *w, uint16_t index, uint8_t value)

```
void wvui_chart_set(wvui_widget_t *w, uint16_t index, uint8_t value);
```

功能描述	写入指定索引的样本值并触发重绘。
输入参数	w：控件；index：样本索引；value：样本值，取值 0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_chart_set(ch, 10, 200);
```

注意事项：

索引越界时为空操作。

3.24.3 wvui_chart_push(wvui_widget_t *w, uint8_t value)

```
void wvui_chart_push(wvui_widget_t *w, uint8_t value);
```

功能描述	追加一个样本；环形模式下写满后自动回绕，覆盖最旧数据。
输入参数	w：控件；value：样本值，取值 0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_chart_push(ch, adc_val >> 4);
```

注意事项：

非环形模式下写满后新样本被丢弃，需先 clear 或扩大缓冲区。

3.24.4 wvui_chart_clear(wvui_widget_t *w)

```
void wvui_chart_clear(wvui_widget_t *w);
```

功能描述	清空全部样本，写指针归零。
输入参数	w：控件。
输出参数	无

返回值	无

使用示例：

```
wvui_chart_clear(ch);
```

注意事项：

绑定模式下清零会实际改写外部缓冲区的内容。

3.24.5 wvui_chart_set_colors(wvui_widget_t *w, wvui_color_t line, wvui_color_t base)

```
void wvui_chart_set_colors(wvui_widget_t *w, wvui_color_t line, wvui_color_t base);
```

功能描述	设置折线颜色与零线基准颜色。
输入参数	w：控件；line：折线色；base：零线色。
输出参数	无
返回值	无

使用示例：

```
wvui_chart_set_colors(ch, C_BLUE, C_GRAY);
```

注意事项：

零线色仅在样本跨越零点时明显，单极性数据可设为背景色。

3.24.6 wvui_chart_set_ring(wvui_widget_t *w, uint8_t ring)

```
void wvui_chart_set_ring(wvui_widget_t *w, uint8_t ring);
```

功能描述	设置是否使用环形缓冲模式。
输入参数	w：控件；ring：非 0 为环形。
输出参数	无
返回值	无

使用示例：

```
wvui_chart_set_ring(ch, 1);
```

注意事项：

环形模式适合实时波形，写入指针越过末尾后从 0 继续。

3.24.7 wvui_chart_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_chart_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。

输出参数	无
返回值	无

使用示例：

```
wvui_chart_set_opa(ch, 255);
```

注意事项：

折线与零线同时受该值影响；波形叠加在其他内容上时可适当降低。

3.24.8 wvui_chart_count(const wvui_widget_t *w)

```
uint16_t wvui_chart_count(const wvui_widget_t *w);
```

功能描述	读回当前有效样本数量。
输入参数	w：控件。
输出参数	无
返回值	样本数量。

使用示例：

```
uint16_t n = wvui_chart_count(ch);
```

注意事项：

环形模式下该值恒为绑定总数；已写入数量需配合 wvui_chart_head 判断。

3.24.9 wvui_chart_head(const wvui_widget_t *w)

```
uint16_t wvui_chart_head(const wvui_widget_t *w);
```

功能描述	读回环形模式的写指针位置。
输入参数	w：控件。
输出参数	无
返回值	下一次写入的索引。

使用示例：

```
uint16_t h = wvui_chart_head(ch);
```

注意事项：

未开启环形模式时该值恒等于样本数量。

3.25 wvui 步进器 API（WVUI_TYPE_STEPPER）

3.25.1 wvui_stepper_set_range(wvui_widget_t *w, int32_t min, int32_t max, uint8_t decimals)

```
void wvui_stepper_set_range(wvui_widget_t *w, int32_t min, int32_t max, uint8_t decimals);
```

功能描述	设置取值区间与显示的小数位数。
输入参数	w：控件；min：下限；max：上限；decimals：小数位数。
输出参数	无
返回值	无

使用示例：

```
wvui_stepper_set_range(st, 0, 1000, 1);
```

注意事项：

设置后当前值将被夹紧至新范围；min 大于 max 时按空区间处理。

3.25.2 wvui_stepper_set_step(wvui_widget_t *w, int32_t step)

```
void wvui_stepper_set_step(wvui_widget_t *w, int32_t step);
```

功能描述	设置单次加减的步长。
输入参数	w：控件；step：步长。
输出参数	无
返回值	无

使用示例：

```
wvui_stepper_set_step(st, 10);
```

注意事项：

步长应与小数位匹配，例如 1 位小数时步长取 1 表示 0.1。

3.25.3 wvui_stepper_set_value(wvui_widget_t *w, int32_t value)

```
void wvui_stepper_set_value(wvui_widget_t *w, int32_t value);
```

功能描述	设置当前值，超出区间时自动夹紧。
输入参数	w：控件；value：目标值。
输出参数	无
返回值	无

使用示例：

```
wvui_stepper_set_value(st, 250);
```

注意事项：

该接口按程序设值处理，不触发值变化回调。

3.25.4 wvui_stepper_add(wvui_widget_t *w, int32_t delta)

```
void wvui_stepper_add(wvui_widget_t *w, int32_t delta);
```

功能描述	在当前值上叠加增量，结果夹紧到区间。
输入参数	w：控件；delta：增量，可为负。
输出参数	无
返回值	无

使用示例：

```
wvui_stepper_add(st, 5);
```

注意事项：

与按键加减不同，该接口按程序改值处理，不触发变化回调。

3.25.5 wvui_stepper_set_repeat(wvui_widget_t *w, uint8_t frames)

```
void wvui_stepper_set_repeat(wvui_widget_t *w, uint8_t frames);
```

功能描述	设置长按加减键时的连发间隔。
输入参数	w：控件；frames：间隔帧数，0 表示不连发。
输出参数	无
返回值	无

使用示例：

```
wvui_stepper_set_repeat(st, 6);
```

注意事项：

间隔按帧计；设为 0 时按住仅触发一次加减，不会连发。

3.25.6 wvui_stepper_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_stepper_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_stepper_set_opa(st, 255);
```

注意事项：

透明度同时作用于数值文本与加减按键，不改变按键的可用状态。

3.25.7 wvui_stepper_set_enabled(wvui_widget_t *w, uint8_t enabled)

```
void wvui_stepper_set_enabled(wvui_widget_t *w, uint8_t enabled);
```

功能描述	设置可用状态，禁用时不响应加减与长按。
输入参数	w：控件；enabled：非 0 表示可用。
输出参数	无
返回值	无

使用示例：

```
wvui_stepper_set_enabled(st, 0);
```

注意事项：

禁用后数值仍可读，仅不再响应加减与长按连发。

3.25.8 wvui_stepper_set_change_cb(wvui_widget_t *w, wvui_stepper_change_cb_t cb, void *user)

```
void wvui_stepper_set_change_cb(wvui_widget_t *w, wvui_stepper_change_cb_t cb, void *user);
```

功能描述	设置值变化回调及透传参数。
输入参数	w：控件；cb：回调；user：透传指针。
输出参数	无
返回值	无

使用示例：

```
static void on_st(wvui_widget_t *w, void *u) { /* ... */ }
wvui_stepper_set_change_cb(st, on_st, NULL);
```

注意事项：

仅按键触发的加减会回调，程序调用 set_value 不回调。

3.25.9 wvui_stepper_value(const wvui_widget_t *w)

```
int32_t wvui_stepper_value(const wvui_widget_t *w);
```

功能描述	读回当前值。
输入参数	w：控件。
输出参数	无
返回值	当前值。

使用示例：

```
int32_t v = wvui_stepper_value(st);
```

注意事项：

返回值为定点整数，显示时需按 decimals 插入小数点。

3.25.10 wvui_stepper_decimals(const wvui_widget_t *w)

```
uint8_t wvui_stepper_decimals(const wvui_widget_t *w);
```

功能描述	读回小数位数。
输入参数	w：控件。
输出参数	无
返回值	小数位数。

使用示例：

```
uint8_t d = wvui_stepper_decimals(st);
```

注意事项：

小数位由 set_range 设置，该接口仅读取，不能单独修改。

3.25.11 wvui_stepper_text(const wvui_widget_t *w)

```
const char *wvui_stepper_text(const wvui_widget_t *w);
```

功能描述	读回按当前值格式化后的显示文本。
输入参数	w：控件。
输出参数	无
返回值	指向控件内部缓冲区的字符串，参数无效时为 NULL。

使用示例：

```
const char *s = wvui_stepper_text(st);
```

注意事项：

返回指针由控件内部维护，不得释放，也不应跨值变更长期缓存。

3.26 wvui 文本列表 API（WVUI_TYPE_LIST）

3.26.1 wvui_list_bind(wvui_widget_t *w, const char *const *items, uint16_t count)

```
void wvui_list_bind(wvui_widget_t *w, const char *const *items, uint16_t count);
```

功能描述	绑定字符串表，不拷贝内容，直接引用该指针数组。
输入参数	w：控件；items：字符串指针数组；count：条目数。
输出参数	无
返回值	无

使用示例：

```
static const char *const items[] = {"模式 A", "模式 B", "模式 C"};
```

```
wvui_list_bind(ls, items, 3);
```

注意事项：

字符串表与各条目文本均须常驻，控件使用期间不可释放。

条目数变化后需重新 bind，并调用 wvui_list_refresh。

3.26.2 wvui_list_set_font(wvui_widget_t *w, const wvui_font_t *font)

```
void wvui_list_set_font(wvui_widget_t *w, const wvui_font_t *font);
```

功能描述	设置行文本使用的字体。
输入参数	w：控件；font：字体对象，NULL 表示使用默认字体。
输出参数	无
返回值	无

使用示例：

```
wvui_list_set_font(ls, &font16);
```

注意事项：

更换字体后需重新调用 refresh，滚动范围才会按新行高重算。

3.26.3 wvui_list_set_row_height(wvui_widget_t *w, int16_t row_h)

```
void wvui_list_set_row_height(wvui_widget_t *w, int16_t row_h);
```

功能描述	设置每行高度。
输入参数	w：控件；row_h：像素，0 表示按字体行高自动计算。
输出参数	无
返回值	无

使用示例：

```
wvui_list_set_row_height(ls, 28);
```

注意事项：

修改行高后需调用 refresh，否则可滚动区间仍按旧行高计算。

3.26.4 wvui_list_set_selected(wvui_widget_t *w, uint16_t index)

```
void wvui_list_set_selected(wvui_widget_t *w, uint16_t index);
```

功能描述	选中指定条目并自动滚动到可见区域。
输入参数	w：控件；index：条目索引。
输出参数	无
返回值	无

使用示例：

```
wvui_list_set_selected(ls, 2);
```

注意事项：

索引越界时为空操作；该接口不触发选中回调。

3.26.5 wvui_list_set_colors(wvui_widget_t *w, wvui_color_t bg, wvui_color_t fg, wvui_color_t sel_fg)

```
void wvui_list_set_colors(wvui_widget_t *w, wvui_color_t bg, wvui_color_t fg, wvui_color_t sel_fg);
```

功能描述	设置背景色、常态文字色与选中文字色。
输入参数	w：控件；bg：背景色；fg：常态文字色；sel_fg：选中文字色。
输出参数	无
返回值	无

使用示例：

```
wvui_list_set_colors(ls, C_WHITE, C_BLACK, C_BLUE);
```

注意事项：

选中行文字使用 sel_fg，该行底色沿用控件背景，不单独提供选中底色。

3.26.6 wvui_list_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_list_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_list_set_opa(ls, 255);
```

注意事项：

透明度作用于全部行与背景，不改变选中行的文字配色。

3.26.7 wvui_list_set_enabled(wvui_widget_t *w, uint8_t enabled)

```
void wvui_list_set_enabled(wvui_widget_t *w, uint8_t enabled);
```

功能描述	设置是否可交互；禁用时不响应点选与拖动滚动。
输入参数	w：控件；enabled：非 0 表示可用。
输出参数	无
返回值	无

使用示例：

```
wvui_list_set_enabled(ls, 0);
```

注意事项：

禁用后列表不再响应点选与拖动，已选中的条目保持不变。

3.26.8 wvui_list_set_select_cb(wvui_widget_t *w, wvui_list_select_cb_t cb, void *user)

```
void wvui_list_set_select_cb(wvui_widget_t *w, wvui_list_select_cb_t cb, void *user);
```

功能描述	设置选中变化回调及透传参数。
输入参数	w：控件；cb：回调；user：透传指针。
输出参数	无
返回值	无

使用示例：

```
static void on_ls(wvui_widget_t *w, void *u) { /* ... */ }
wvui_list_set_select_cb(ls, on_ls, NULL);
```

注意事项：

仅用户点选会触发回调，wvui_list_set_selected 不触发。

3.26.9 wvui_list_refresh(wvui_widget_t *w)

```
void wvui_list_refresh(wvui_widget_t *w);
```

功能描述	重算滚动范围并标记重绘。
输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_list_refresh(ls);
```

注意事项：

修改行高、字体或条目内容后必须调用，否则滚动范围仍为旧值。

3.26.10 wvui_list_count(const wvui_widget_t *w)

```
uint16_t wvui_list_count(const wvui_widget_t *w);
```

功能描述	读回条目数。
输入参数	w：控件。
输出参数	无

返回值	条目数量。

使用示例：

```
uint16_t n = wvui_list_count(ls);
```

注意事项：

数量由 bind 的 count 决定，重新绑定后需调用 refresh 才会更新滚动范围。

3.26.11 wvui_list_selected(const wvui_widget_t *w)

```
uint16_t wvui_list_selected(const wvui_widget_t *w);
```

功能描述	读回当前选中索引。
输入参数	w：控件。
输出参数	无
返回值	选中索引，无选中时返回值超出有效范围。

使用示例：

```
uint16_t sel = wvui_list_selected(ls);
```

注意事项：

无选中时返回值将超出有效索引范围，使用前应据此判断。

3.26.12 wvui_list_scroll(const wvui_widget_t *w)

```
int32_t wvui_list_scroll(const wvui_widget_t *w);
```

功能描述	读回滚动位置。
输入参数	w：控件。
输出参数	无
返回值	相对内容顶部的像素位移。

使用示例：

```
int32_t s = wvui_list_scroll(ls);
```

注意事项：

单位为像素，除以行高可换算为已滚过的行数，用于判断是否已到底部。

3.26.13 wvui_list_item(const wvui_widget_t *w, uint16_t index)

```
const char *wvui_list_item(const wvui_widget_t *w, uint16_t index);
```

功能描述	获取指定条目的字符串指针。
输入参数	w：控件；index：条目索引。
输出参数	无

返回值	字符串指针，索引无效或未绑定时为 NULL。

使用示例：

```
const char *s = wvui_list_item(ls, 0);
```

注意事项：

返回的是绑定时传入的地址，控件既不拷贝也不释放。

3.27 wvui 滚轮选择器 API（WVUI_TYPE_ROLLER）

3.27.1 wvui_roller_bind(wvui_widget_t *w, const char *const *items, uint16_t count)

```
void wvui_roller_bind(wvui_widget_t *w, const char *const *items, uint16_t count);
```

功能描述	绑定字符串表，不拷贝内容，直接引用该指针数组。
输入参数	w：控件；items：字符串指针数组；count：条目数。
输出参数	无
返回值	无

使用示例：

```
static const char *const items[] = {"1", "2", "3", "4"};
wvui_roller_bind(rl, items, 4);
```

注意事项：

字符串表与文本均须常驻，控件存续期间不可释放。

3.27.2 wvui_roller_set_font(wvui_widget_t *w, const wvui_font_t *font)

```
void wvui_roller_set_font(wvui_widget_t *w, const wvui_font_t *font);
```

功能描述	设置条目文本字体。
输入参数	w：控件；font：字体对象，NULL 表示使用默认字体。
输出参数	无
返回值	无

使用示例：

```
wvui_roller_set_font(rl, &font16);
```

注意事项：

更换字体后条目高度变化，需同步调整 set_row_height 与控件高度。

3.27.3 wvui_roller_set_row_height(wvui_widget_t *w, int16_t row_h)

```
void wvui_roller_set_row_height(wvui_widget_t *w, int16_t row_h);
```

功能描述	设置每行高度。

输入参数	w：控件；row_h：像素，0 表示按字体行高自动计算。
输出参数	无
返回值	无

使用示例：

```
wvui_roller_set_row_height(rl, 32);
```

注意事项：

控件高度通常应为行高的整数倍，否则将出现半行被裁切的情况。

3.27.4 wvui_roller_set_selected(wvui_widget_t *w, uint16_t index)

```
void wvui_roller_set_selected(wvui_widget_t *w, uint16_t index);
```

功能描述	选中指定条目并滚动到该行位置。
输入参数	w：控件；index：条目索引。
输出参数	无
返回值	无

使用示例：

```
wvui_roller_set_selected(rl, 2);
```

注意事项：

该接口不触发停稳选中回调。

3.27.5 wvui_roller_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_roller_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_roller_set_opa(rl, 255);
```

注意事项：

透明度作用于全部可见行，选中行不会因透明度而额外突出。

3.27.6 wvui_roller_set_enabled(wvui_widget_t *w, uint8_t enabled)

```
void wvui_roller_set_enabled(wvui_widget_t *w, uint8_t enabled);
```

功能描述	设置是否可交互，禁用时不响应拖动。

输入参数	w：控件；enabled：非 0 表示可用。
输出参数	无
返回值	无

使用示例：

```
wvui_roller_set_enabled(rl, 0);
```

注意事项：

禁用后仍显示当前选中项，仅不再响应拖动与惯性滚动。

3.27.7 wvui_roller_set_select_cb(wvui_widget_t *w, wvui_roller_select_cb_t cb, void *user)

```
void wvui_roller_set_select_cb(wvui_widget_t *w, wvui_roller_select_cb_t cb, void *user);
```

功能描述	设置滚动停稳后选中变化的回调及透传参数。
输入参数	w：控件；cb：回调；user：透传指针。
输出参数	无
返回值	无

使用示例：

```
static void on_rl(wvui_widget_t *w, void *u) { /* ... */ }
wvui_roller_set_select_cb(rl, on_rl, NULL);
```

注意事项：

回调仅在滚动停稳并换行时触发一次，拖动过程中不触发。

3.27.8 wvui_roller_refresh(wvui_widget_t *w)

```
void wvui_roller_refresh(wvui_widget_t *w);
```

功能描述	重算滚动范围并标记重绘。
输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_roller_refresh(rl);
```

注意事项：

修改行高、字体或条目文本后必须调用。

3.27.9 wvui_roller_count(const wvui_widget_t *w)

```
uint16_t wvui_roller_count(const wvui_widget_t *w);
```

功能描述	读回条目数。
输入参数	w：控件。
输出参数	无
返回值	条目数量。

使用示例：

```
uint16_t n = wvui_roller_count(rl);
```

注意事项：

数量由 bind 的 count 决定；未绑定时为 0，此时控件只绘制空框。

3.27.10 wvui_roller_selected(const wvui_widget_t *w)

```
uint16_t wvui_roller_selected(const wvui_widget_t *w);
```

功能描述	读回当前选中索引。
输入参数	w：控件。
输出参数	无
返回值	选中索引。

使用示例：

```
uint16_t sel = wvui_roller_selected(rl);
```

注意事项：

拖动过程中该值随行变化实时更新，停稳后才触发选中回调。

3.27.11 wvui_roller_scroll(const wvui_widget_t *w)

```
int32_t wvui_roller_scroll(const wvui_widget_t *w);
```

功能描述	读回滚动位置。
输入参数	w：控件。
输出参数	无
返回值	相对顶部的像素位移。

使用示例：

```
int32_t s = wvui_roller_scroll(rl);
```

注意事项：

单位为像素，除以行高可换算为当前滚动到的行号。

3.27.12 wvui_roller_item(const wvui_widget_t *w, uint16_t index)

```
const char *wvui_roller_item(const wvui_widget_t *w, uint16_t index);
```

功能描述	获取指定条目的字符串指针。
输入参数	w：控件；index：条目索引。
输出参数	无
返回值	字符串指针，索引无效或未绑定时为 NULL。

使用示例：

```
const char *s = wvui_roller_item(rl, 0);
```

注意事项：

返回绑定时传入的地址，控件既不拷贝也不释放；索引越界时返回 NULL。

3.28 wvui 浮动提示 API（WVUI_TYPE_TOAST）

3.28.1 wvui_toast_show(wvui_widget_t *w, const char *text, uint16_t frames)

```
void wvui_toast_show(wvui_widget_t *w, const char *text, uint16_t frames);
```

功能描述	显示提示文本，到期后自动淡出。
输入参数	w：控件；text：文本，不拷贝，需常驻；frames：存活帧数。
输出参数	无
返回值	无

使用示例：

```
wvui_toast_show(ts, "已保存", 90);
```

注意事项：

重复调用会重置显示计时与文本指针。

未设置字体时只绘制底框，不显示文字。

3.28.2 wvui_toast_hide(wvui_widget_t *w)

```
void wvui_toast_hide(wvui_widget_t *w);
```

功能描述	立即隐藏提示。
输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_toast_hide(ts);
```

注意事项：

隐藏不触发淡出过渡；如需渐变消失，应依赖 show 的到期流程。

3.28.3 wvui_toast_set_font(wvui_widget_t *w, const wvui_font_t *font)

```
void wvui_toast_set_font(wvui_widget_t *w, const wvui_font_t *font);
```

功能描述	设置文本字体。
输入参数	w：控件；font：字体对象。
输出参数	无
返回值	无

使用示例：

```
wvui_toast_set_font(ts, &font16);
```

注意事项：

控件没有默认字体，未设置或传 NULL 时仅绘制底框。

3.28.4 wvui_toast_set_colors(wvui_widget_t *w, wvui_color_t bg, wvui_color_t fg, wvui_color_t border)

```
void wvui_toast_set_colors(wvui_widget_t *w, wvui_color_t bg, wvui_color_t fg, wvui_color_t border);
```

功能描述	设置背景色、文字色与边框色。
输入参数	w：控件；bg：背景色；fg：文字色；border：边框色。
输出参数	无
返回值	无

使用示例：

```
wvui_toast_set_colors(ts, C_BLACK, C_WHITE, C_DGRAY);
```

注意事项：

深色背景配浅色文字在多数界面下可读性最佳，边框可设为同色以弱化轮廓。

3.28.5 wvui_toast_set_radius(wvui_widget_t *w, uint8_t radius)

```
void wvui_toast_set_radius(wvui_widget_t *w, uint8_t radius);
```

功能描述	设置圆角半径。
输入参数	w：控件；radius：像素，默认为 6。
输出参数	无
返回值	无

使用示例：

```
wvui_toast_set_radius(ts, 8);
```

注意事项：

默认 6 像素；提示框通常较矮，圆角超过高度一半时会变为胶囊形。

3.28.6 wvui_toast_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_toast_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置不透明度上限，淡出时在此基础上按比例衰减。
输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_toast_set_opa(ts, 230);
```

注意事项：

该值为不透明度上限，淡出阶段将在此基础上继续衰减至 0。

3.28.7 wvui_toast_is_active(const wvui_widget_t *w)

```
uint8_t wvui_toast_is_active(const wvui_widget_t *w);
```

功能描述	查询是否正在显示。
输入参数	w：控件。
输出参数	无
返回值	1 表示显示中。

使用示例：

```
if (wvui_toast_is_active(ts)) { /* 正在显示 */ }
```

注意事项：

从 show 调用开始至淡出结束均返回 1，可用于避免重复弹出。

3.28.8 wvui_toast_left(const wvui_widget_t *w)

```
uint16_t wvui_toast_left(const wvui_widget_t *w);
```

功能描述	获取剩余存活帧数。
输入参数	w：控件。
输出参数	无
返回值	剩余帧数，未显示时为 0。

使用示例：

```
uint16_t left = wvui_toast_left(ts);
```

注意事项：

计数按帧递减，换算为秒需除以实际帧率；淡出阶段仍计入剩余帧数。

3.29 wvui 模态对话框 API（WVUI_TYPE_MSGBOX）

3.29.1 wvui_msgbox_open(wvui_widget_t *w)

```
void wvui_msgbox_open(wvui_widget_t *w);
```

功能描述	打开对话框，加入顶层链并置为当前模态。
输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_msgbox_open(mb);
```

注意事项：

打开后其他控件不再接收输入，必须先 close 才能恢复。

重复 open 为幂等操作，不会重复入链。

3.29.2 wvui_msgbox_close(wvui_widget_t *w)

```
void wvui_msgbox_close(wvui_widget_t *w);
```

功能描述	关闭对话框，先清除模态再摘除顶层链。
输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_msgbox_close(mb);
```

注意事项：

顺序固定为先清模态再摘链，以避免输入层残留悬挂引用。

3.29.3 wvui_msgbox_set_text(wvui_widget_t *w, const char *title, const char *message)

```
void wvui_msgbox_set_text(wvui_widget_t *w, const char *title, const char *message);
```

功能描述	设置标题与正文文本。
输入参数	w：控件；title：标题；message：正文。
输出参数	无

返回值	无

使用示例：

```
wvui_msgbox_set_text(mb, "提示", "确定要恢复出厂设置吗？");
```

注意事项：

文本不拷贝，指针必须常驻。

3.29.4 wvui_msgbox_set_button_text(wvui_widget_t *w, const char *ok_text, const char *cancel_text)

```
void wvui_msgbox_set_button_text(wvui_widget_t *w, const char *ok_text, const char *cancel_text);
```

功能描述	设置两个按钮的文本。
输入参数	w：控件；ok_text：确定按钮文本；cancel_text：取消按钮文本。
输出参数	无
返回值	无

使用示例：

```
wvui_msgbox_set_button_text(mb, "确定", "取消");
```

注意事项：

传入 NULL 时使用控件内置的默认文本。

3.29.5 wvui_msgbox_set_buttons(wvui_widget_t *w, uint8_t two_buttons)

```
void wvui_msgbox_set_buttons(wvui_widget_t *w, uint8_t two_buttons);
```

功能描述	设置按钮数量与横向布局。
输入参数	w：控件；two_buttons：1 表示双按钮，0 表示单按钮。
输出参数	无
返回值	无

使用示例：

```
wvui_msgbox_set_buttons(mb, 1);
```

注意事项：

双按钮为横向等分排列；单按钮时取消文本与取消回调均不参与绘制与响应。

3.29.6 wvui_msgbox_set_font(wvui_widget_t *w, const wvui_font_t *font)

```
void wvui_msgbox_set_font(wvui_widget_t *w, const wvui_font_t *font);
```

功能描述	设置标题与正文使用的字体。
输入参数	w：控件；font：字体对象，NULL 表示使用默认字体。

输出参数	无
返回值	无

使用示例：

```
wvui_msgbox_set_font(mb, &font16);
```

注意事项：

字体作用于标题、正文与按钮文本；更换后正文换行位置将重新计算。

3.29.7 wvui_msgbox_set_colors(wvui_widget_t *w, wvui_color_t bg, wvui_color_t fg, wvui_color_t border)

```
void wvui_msgbox_set_colors(wvui_widget_t *w, wvui_color_t bg, wvui_color_t fg, wvui_color_t border);
```

功能描述	设置背景色、正文色与边框色。
输入参数	w：控件；bg：背景色；fg：文字色；border：边框色。
输出参数	无
返回值	无

使用示例：

```
wvui_msgbox_set_colors(mb, C_WHITE, C_BLACK, C_LGRAY);
```

注意事项：

按钮底色不在此处设置，需另行调用 set_button_colors。

3.29.8 wvui_msgbox_set_button_colors(wvui_widget_t *w, wvui_color_t bg, wvui_color_t fg, wvui_color_t pressed)

```
void wvui_msgbox_set_button_colors(wvui_widget_t *w, wvui_color_t bg, wvui_color_t fg, wvui_color_t pressed);
```

功能描述	设置按钮常态底色、文字色与按下底色。
输入参数	w：控件；bg：常态底色；fg：文字色；pressed：按下底色。
输出参数	无
返回值	无

使用示例：

```
wvui_msgbox_set_button_colors(mb, C_LGRAY, C_BLACK, C_GRAY);
```

注意事项：

按下底色仅在按住瞬间生效，松手后恢复为常态底色。

3.29.9 wvui_msgbox_set_radius(wvui_widget_t *w, uint8_t radius)

```
void wvui_msgbox_set_radius(wvui_widget_t *w, uint8_t radius);
```

功能描述	设置对话框圆角半径。
输入参数	w：控件；radius：像素，默认为 8。
输出参数	无
返回值	无

使用示例：

```
wvui_msgbox_set_radius(mb, 10);
```

注意事项：

圆角同时影响对话框外框与内部按钮；取值过大时按钮会明显失去方角。

3.29.10 wvui_msgbox_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_msgbox_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_msgbox_set_opa(mb, 255);
```

注意事项：

该值作用于整个对话框（含按钮），不提供独立的背景蒙层透明度。

3.29.11 wvui_msgbox_set_cb(wvui_widget_t *w, wvui_msgbox_cb_t on_ok, wvui_msgbox_cb_t on_cancel, void *user)

```
void wvui_msgbox_set_cb(wvui_widget_t *w, wvui_msgbox_cb_t on_ok, wvui_msgbox_cb_t on_cancel, void *user);
```

功能描述	设置确定与取消两个按钮的回调，二者共享同一透传参数。
输入参数	w：控件；on_ok：确定回调；on_cancel：取消回调；user：透传指针。
输出参数	无
返回值	无

使用示例：

```
static void on_ok(wvui_widget_t *w, void *u) { /* ... */ }
wvui_msgbox_set_cb(mb, on_ok, NULL, NULL);
```

注意事项：

单按钮模式下仅 on_ok 会被调用。

3.29.12 wvui_msgbox_is_open(const wvui_widget_t *w)

```
uint8_t wvui_msgbox_is_open(const wvui_widget_t *w);
```

功能描述	查询该对话框是否为当前模态。
输入参数	w：控件。
输出参数	无
返回值	1 表示已打开。

使用示例：

```
if (wvui_msgbox_is_open(mb)) { /* 已打开 */ }
```

注意事项：

仅判断本控件是否为当前模态；同时打开多个对话框时，先前打开的将返回 0。

注意事项：

回调内禁止删除控件自身，应置标志并在下一帧删除。

4.30 wvui 下拉选择 API（WVUI_TYPE_DROPDOWN）

4.30.1 wvui_dropdown_bind(wvui_widget_t *w, const char *const *items, uint16_t count)

```
void wvui_dropdown_bind(wvui_widget_t *w, const char *const *items, uint16_t count);
```

功能描述	绑定字符串表，不拷贝内容，直接引用该指针数组。
输入参数	w：控件；items：字符串指针数组；count：条目数。
输出参数	无
返回值	无

使用示例：

```
static const char *const items[] = {"低", "中", "高"};
wvui_dropdown_bind(dd, items, 3);
```

注意事项：

字符串表必须常驻，控件存续期间不可释放。

4.30.2 wvui_dropdown_open(wvui_widget_t *w)

```
void wvui_dropdown_open(wvui_widget_t *w);
```

功能描述	展开弹出层。
输入参数	w：控件。
输出参数	无

返回值	无

使用示例：

```
wvui_dropdown_open(dd);
```

注意事项：

幂等操作，已展开时不再重复入链。

控件被禁用时该调用无效。

4.30.3 wvui_dropdown_close(wvui_widget_t *w)

```
void wvui_dropdown_close(wvui_widget_t *w);
```

功能描述	收起弹出层。
输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_dropdown_close(dd);
```

注意事项：

幂等操作，未展开时为空操作。

4.30.4 wvui_dropdown_toggle(wvui_widget_t *w)

```
void wvui_dropdown_toggle(wvui_widget_t *w);
```

功能描述	按当前状态展开或收起弹出层。
输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_dropdown_toggle(dd);
```

注意事项：

等价于按当前状态调用 open 或 close；禁用状态下调用无效。

4.30.5 wvui_dropdown_set_selected(wvui_widget_t *w, uint16_t index)

```
void wvui_dropdown_set_selected(wvui_widget_t *w, uint16_t index);
```

功能描述	设置选中项，收起态显示该条文本。
输入参数	w：控件；index：条目索引。

输出参数	无
返回值	无

使用示例：

```
wvui_dropdown_set_selected(dd, 1);
```

注意事项：

该接口不触发选中回调。

4.30.6 wvui_dropdown_set_font(wvui_widget_t *w, const wvui_font_t *font)

```
void wvui_dropdown_set_font(wvui_widget_t *w, const wvui_font_t *font);
```

功能描述	设置收起态与弹出层的文本字体。
输入参数	w：控件；font：字体对象，NULL 表示使用默认字体。
输出参数	无
返回值	无

使用示例：

```
wvui_dropdown_set_font(dd, &font16);
```

注意事项：

字体同时用于收起态与弹出层条目；更换后需重设行高以匹配新的字高。

4.30.7 wvui_dropdown_set_row_height(wvui_widget_t *w, uint8_t row_h)

```
void wvui_dropdown_set_row_height(wvui_widget_t *w, uint8_t row_h);
```

功能描述	设置弹出层每行高度。
输入参数	w：控件；row_h：像素，小于 4 时按 4 处理。
输出参数	无
返回值	无

使用示例：

```
wvui_dropdown_set_row_height(dd, 26);
```

注意事项：

行高小于 4 时将被强制为 4；过小会导致文字上下被裁剪。

4.30.8 wvui_dropdown_set_max_visible(wvui_widget_t *w, uint8_t rows)

```
void wvui_dropdown_set_max_visible(wvui_widget_t *w, uint8_t rows);
```

功能描述	设置弹出层最多同时显示的行数。
输入参数	w：控件；rows：行数，0 时按 1 处理。

输出参数	无
返回值	无

使用示例：

```
wvui_dropdown_set_max_visible(dd, 4);
```

注意事项：

条目数超过该值时，弹出层内部可滚动，需配合刷新重算范围。

4.30.9 wvui_dropdown_set_colors(wvui_widget_t *w, wvui_color_t bg, wvui_color_t fg, wvui_color_t border)

```
void wvui_dropdown_set_colors(wvui_widget_t *w, wvui_color_t bg, wvui_color_t fg, wvui_color_t border);
```

功能描述	设置收起态的背景色、文字色与边框色。
输入参数	w：控件；bg：背景色；fg：文字色；border：边框色。
输出参数	无
返回值	无

使用示例：

```
wvui_dropdown_set_colors(dd, C_WHITE, C_BLACK, C_GRAY);
```

注意事项：

仅影响收起态；弹出层底色与条目颜色由选中行配色与默认底色决定。

4.30.10 wvui_dropdown_set_sel_colors(wvui_widget_t *w, wvui_color_t sel_bg, wvui_color_t sel_fg)

```
void wvui_dropdown_set_sel_colors(wvui_widget_t *w, wvui_color_t sel_bg, wvui_color_t sel_fg);
```

功能描述	设置弹出层中选中行的底色与文字色。
输入参数	w：控件；sel_bg：选中行底色；sel_fg：选中行文字色。
输出参数	无
返回值	无

使用示例：

```
wvui_dropdown_set_sel_colors(dd, C_BLUE, C_WHITE);
```

注意事项：

选中行底色仅覆盖该行，其余行仍使用弹出层默认底色。

4.30.11 wvui_dropdown_set_radius(wvui_widget_t *w, uint8_t radius)

```
void wvui_dropdown_set_radius(wvui_widget_t *w, uint8_t radius);
```

功能描述	设置收起态与弹出层的圆角半径。

输入参数	w：控件；radius：像素。
输出参数	无
返回值	无

使用示例：

```
wvui_dropdown_set_radius(dd, 4);
```

注意事项：

圆角同时作用于收起态控件与展开后的弹出层。

4.30.12 wvui_dropdown_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_dropdown_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_dropdown_set_opa(dd, 255);
```

注意事项：

透明度作用于收起态与弹出层整体；设为 0 时不可见，但仍能响应输入。

4.30.13 wvui_dropdown_set_enabled(wvui_widget_t *w, uint8_t enabled)

```
void wvui_dropdown_set_enabled(wvui_widget_t *w, uint8_t enabled);
```

功能描述	设置可用状态；禁用时已展开的弹出层会自动收起。
输入参数	w：控件；enabled：非 0 表示可用。
输出参数	无
返回值	无

使用示例：

```
wvui_dropdown_set_enabled(dd, 0);
```

注意事项：

禁用会立即收起已展开的弹出层，重新使能不会自动再次展开。

4.30.14 wvui_dropdown_set_select_cb(wvui_widget_t *w, wvui_dropdown_cb_t cb, void *user)

```
void wvui_dropdown_set_select_cb(wvui_widget_t *w, wvui_dropdown_cb_t cb, void *user);
```

功能描述	设置选中变化回调及透传参数。
输入参数	w：控件；cb：回调；user：透传指针。
输出参数	无
返回值	无

使用示例：

```
static void on_dd(wvui_widget_t *w, void *u) { /* ... */ }
wvui_dropdown_set_select_cb(dd, on_dd, NULL);
```

注意事项：

仅在弹出层中点选并改变选中项时触发。

4.30.15 wvui_dropdown_count(const wvui_widget_t *w)

```
uint16_t wvui_dropdown_count(const wvui_widget_t *w);
```

功能描述	读回条目数。
输入参数	w：控件。
输出参数	无
返回值	条目数量。

使用示例：

```
uint16_t n = wvui_dropdown_count(dd);
```

注意事项：

返回 bind 时传入的条目数；未绑定时为 0，此时展开弹出层为空白。

4.31 wvui 数码管显示 API（WVUI_TYPE_SEGDISP）

4.31.1 wvui_segdisp_set_text(wvui_widget_t *w, const char *text)

```
void wvui_segdisp_set_text(wvui_widget_t *w, const char *text);
```

功能描述	直接设置要显示的字符串，按字符映射到数码管笔画。
输入参数	w：控件；text：字符串，不拷贝，需常驻。
输出参数	无
返回值	无

使用示例：

```
wvui_segdisp_set_text(sd7, "12:30");
```

注意事项：

超出位数长度的字符被截断。

4.31.2 wvui_segdisp_set_value(wvui_widget_t *w, uint32_t value)

```
void wvui_segdisp_set_value(wvui_widget_t *w, uint32_t value);
```

功能描述	按数值刷新显示，由内部格式化为数字字符串。
输入参数	w：控件；value：待显示的数值。
输出参数	无
返回值	无

使用示例：

```
wvui_segdisp_set_value(sd7, 1234);
```

注意事项：

小数位由 set_decimals 决定，位数不足时高位熄灭。

4.31.3 wvui_segdisp_set_digits(wvui_widget_t *w, uint8_t digits)

```
void wvui_segdisp_set_digits(wvui_widget_t *w, uint8_t digits);
```

功能描述	设置数码管的位数。
输入参数	w：控件；digits：位数。
输出参数	无
返回值	无

使用示例：

```
wvui_segdisp_set_digits(sd7, 4);
```

注意事项：

位数与笔段尺寸共同决定控件所需宽度。

4.31.4 wvui_segdisp_set_decimals(wvui_widget_t *w, uint8_t decimals)

```
void wvui_segdisp_set_decimals(wvui_widget_t *w, uint8_t decimals);
```

功能描述	设置数值显示时的小数位数。
输入参数	w：控件；decimals：小数位数。
输出参数	无
返回值	无

使用示例：

```
wvui_segdisp_set_decimals(sd7, 2);
```

注意事项：

小数点占用一个位宽，小数位数增加时应相应增加 set_digits。

4.31.5 wvui_segdisp_set_seg_size(wvui_widget_t *w, uint8_t seg_w, uint8_t seg_h, uint8_t seg_t)

```
void wvui_segdisp_set_seg_size(wvui_widget_t *w, uint8_t seg_w, uint8_t seg_h, uint8_t seg_t);
```

功能描述	设置单个笔段的宽、高与线宽。
输入参数	w：控件；seg_w：笔段宽；seg_h：笔段高；seg_t：笔画粗细。
输出参数	无
返回值	无

使用示例：

```
wvui_segdisp_set_seg_size(sd7, 12, 24, 3);
```

注意事项：

粗细应小于宽度与高度中的较小者，否则笔画会相连。

4.31.6 wvui_segdisp_set_gap(wvui_widget_t *w, uint8_t gap)

```
void wvui_segdisp_set_gap(wvui_widget_t *w, uint8_t gap);
```

功能描述	设置相邻位之间的间距。
输入参数	w：控件；gap：像素。
输出参数	无
返回值	无

使用示例：

```
wvui_segdisp_set_gap(sd7, 4);
```

注意事项：

间距仅增大位与位之间的空白，不改变单个数码管的笔画尺寸。

4.31.7 wvui_segdisp_set_colors(wvui_widget_t *w, wvui_color_t on, wvui_color_t off)

```
void wvui_segdisp_set_colors(wvui_widget_t *w, wvui_color_t on, wvui_color_t off);
```

功能描述	设置点亮笔段色与熄灭笔段色。
输入参数	w：控件；on：点亮色；off：熄灭色。
输出参数	无
返回值	无

使用示例：

```
wvui_segdisp_set_colors(sd7, C_RED, C_DGRAY);
```

注意事项：

熄灭色若与背景色一致，可得到仅点亮笔段可见的视觉效果。

4.31.8 wvui_segdisp_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_segdisp_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_segdisp_set_opa(sd7, 255);
```

注意事项：

点亮与熄灭笔段同时受该值影响；调低后熄灭段将先难以辨认。

4.31.9 wvui_segdisp_value(const wvui_widget_t *w)

```
uint32_t wvui_segdisp_value(const wvui_widget_t *w);
```

功能描述	读回当前数值。
输入参数	w：控件。
输出参数	无
返回值	当前数值。

使用示例：

```
uint32_t v = wvui_segdisp_value(sd7);
```

注意事项：

用 set_text 设置文本时不更新该数值，读回的仍是上次 set_value 的值。

4.31.10 wvui_segdisp_text(const wvui_widget_t *w)

```
const char *wvui_segdisp_text(const wvui_widget_t *w);
```

功能描述	读回当前显示字符串。
输入参数	w：控件。
输出参数	无
返回值	字符串指针，参数无效时为 NULL。

使用示例：

```
const char *s = wvui_segdisp_text(sd7);
```

注意事项：

set_value 后返回内部格式化缓冲区指针，其内容随下次设值变化。

4.32 wvui 跑马灯 API（WVUI_TYPE_MARQUEE）

4.32.1 wvui_marquee_set_text(wvui_widget_t *w, const char *text)

```
void wvui_marquee_set_text(wvui_widget_t *w, const char *text);
```

功能描述	设置文本，并重新判断文本是否超出宽度而需要滚动。
输入参数	w：控件；text：文本，不拷贝，需常驻。
输出参数	无
返回值	无

使用示例：

```
wvui_marquee_set_text(mq, "系统运行状态提示");
```

注意事项：

文本宽度不超过控件宽度时静止显示，不启动滚动。

4.32.2 wvui_marquee_set_font(wvui_widget_t *w, const wvui_font_t *font)

```
void wvui_marquee_set_font(wvui_widget_t *w, const wvui_font_t *font);
```

功能描述	设置字体，并按新字体重判滚动条件。
输入参数	w：控件；font：字体对象，NULL 表示使用默认字体。
输出参数	无
返回值	无

使用示例：

```
wvui_marquee_set_font(mq, &font16);
```

注意事项：

更换字体会重新测量文本宽度；原本静止的短文本可能转为滚动状态。

4.32.3 wvui_marquee_set_color(wvui_widget_t *w, wvui_color_t fg)

```
void wvui_marquee_set_color(wvui_widget_t *w, wvui_color_t fg);
```

功能描述	设置文字颜色。
输入参数	w：控件；fg：文字色。
输出参数	无

返回值	无

使用示例：

```
wvui_marquee_set_color(mq, C_WHITE);
```

注意事项：

仅改变文字色，控件背景需由所在页面或父容器提供。

4.32.4 wvui_marquee_set_speed(wvui_widget_t *w, uint8_t px_per_frame)

```
void wvui_marquee_set_speed(wvui_widget_t *w, uint8_t px_per_frame);
```

功能描述	设置滚动速度。
输入参数	w：控件；px_per_frame：每帧左移像素数。
输出参数	无
返回值	无

使用示例：

```
wvui_marquee_set_speed(mq, 2);
```

注意事项：

速度按帧计算，帧率越高视觉速度越快。

4.32.5 wvui_marquee_set_gap(wvui_widget_t *w, uint8_t gap)

```
void wvui_marquee_set_gap(wvui_widget_t *w, uint8_t gap);
```

功能描述	设置循环时文本首尾之间的空白间隔。
输入参数	w：控件；gap：像素。
输出参数	无
返回值	无

使用示例：

```
wvui_marquee_set_gap(mq, 24);
```

注意事项：

间隔出现在文本尾部与循环回到开头之间，取 16 至 32 像素时视觉效果较舒适。

4.32.6 wvui_marquee_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_marquee_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无

返回值	无

使用示例：

```
wvui_marquee_set_opa(mq, 255);
```

注意事项：

文字与滚动过程同时受该值影响，可用作整体淡入淡出。

4.32.7 wvui_marquee_refresh(wvui_widget_t *w)

```
void wvui_marquee_refresh(wvui_widget_t *w);
```

功能描述	重新测量文本宽度并重判滚动条件。
输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_marquee_refresh(mq);
```

注意事项：

控件尺寸或文本内容在外部被修改后，须调用本函数。

4.32.8 wvui_marquee_text(const wvui_widget_t *w)

```
const char *wvui_marquee_text(const wvui_widget_t *w);
```

功能描述	读回文本指针。
输入参数	w：控件。
输出参数	无
返回值	文本指针，未设置时为 NULL。

使用示例：

```
const char *s = wvui_marquee_text(mq);
```

注意事项：

返回绑定时传入的地址，控件不拷贝；修改缓冲区内容后需调用 refresh。

4.32.9 wvui_marquee_text_width(const wvui_widget_t *w)

```
int16_t wvui_marquee_text_width(const wvui_widget_t *w);
```

功能描述	读回缓存的文本像素宽度。
输入参数	w：控件。
输出参数	无

返回值	文本宽度像素值。

使用示例：

```
int16_t tw = wvui_marquee_text_width(mq);
```

注意事项：

该值为上次测量结果，修改文本后需调用 refresh 才会更新。

4.32.10 wvui_marquee_offset(const wvui_widget_t *w)

```
int16_t wvui_marquee_offset(const wvui_widget_t *w);
```

功能描述	读回当前左移偏移量。
输入参数	w：控件。
输出参数	无
返回值	偏移像素值。

使用示例：

```
int16_t off = wvui_marquee_offset(mq);
```

注意事项：

偏移在 0 到"文本宽度+间隔"之间循环，可据此估算一个完整周期的时间。

4.32.11 wvui_marquee_is_running(const wvui_widget_t *w)

```
uint8_t wvui_marquee_is_running(const wvui_widget_t *w);
```

功能描述	查询是否正在滚动。
输入参数	w：控件。
输出参数	无
返回值	1 表示滚动中。

使用示例：

```
if (wvui_marquee_is_running(mq)) { /* 滚动中 */ }
```

注意事项：

文本较窄或未设置文本时返回 0，此时不产生动画开销。

4.33 wvui 分组容器 API（WVUI_TYPE_GROUP）

4.33.1 wvui_group_add(wvui_widget_t *g, wvui_widget_t *child)

```
void wvui_group_add(wvui_widget_t *g, wvui_widget_t *child);
```

功能描述	将子控件加入分组，此后随分组一起显示与绘制。
输入参数	g：分组控件；child：子控件。

输出参数	无
返回值	无

使用示例：

```
wvui_group_add(grp, lbl);
wvui_group_add(grp, sw);
```

注意事项：

同一个子控件只应属于一个分组；如需重复加入，应先从原分组移除。

分组隐藏时子控件一并隐藏，但子控件自身的可见性状态予以保留。

4.33.2 wvui_group_set_hidden(wvui_widget_t *g, uint8_t hidden)

```
void wvui_group_set_hidden(wvui_widget_t *g, uint8_t hidden);
```

功能描述	显示或隐藏整个分组，包括其中的全部子控件。
输入参数	g：分组；hidden：非 0 隐藏。
输出参数	无
返回值	无

使用示例：

```
wvui_group_set_hidden(grp, 1);
```

注意事项：

隐藏期间子控件仍占用内存与布局位置，仅不参与绘制与命中。

4.33.3 wvui_group_is_hidden(const wvui_widget_t *g)

```
uint8_t wvui_group_is_hidden(const wvui_widget_t *g);
```

功能描述	查询分组隐藏状态。
输入参数	g：分组。
输出参数	无
返回值	1 表示隐藏。

使用示例：

```
if (wvui_group_is_hidden(grp)) { /* 已隐藏 */ }
```

注意事项：

该值仅反映分组自身状态，子控件单独设置的可见性不计入其中。

4.33.4 wvui_group_set_colors(wvui_widget_t *g, wvui_color_t bg, wvui_color_t border)

```
void wvui_group_set_colors(wvui_widget_t *g, wvui_color_t bg, wvui_color_t border);
```

功能描述	设置分组自身背景色与边框色。
输入参数	g：分组；bg：背景色；border：边框色。
输出参数	无
返回值	无

使用示例：

```
wvui_group_set_colors(grp, C_WHITE, C_LGRAY);
```

注意事项：

仅影响分组自身的背景与边框，不改变子控件颜色。

背景未开启绘制时只有边框可见。

4.33.5 wvui_group_set_bg_visible(wvui_widget_t *g, uint8_t visible)

```
void wvui_group_set_bg_visible(wvui_widget_t *g, uint8_t visible);
```

功能描述	设置是否绘制分组自身的背景。
输入参数	g：分组；visible：非 0 表示绘制背景。
输出参数	无
返回值	无

使用示例：

```
wvui_group_set_bg_visible(grp, 0);
```

注意事项：

分组通常仅作为布局容器，无需背景；关闭后可减少一次填充绘制。

4.33.6 wvui_group_set_border(wvui_widget_t *g, uint8_t border_w, uint8_t radius)

```
void wvui_group_set_border(wvui_widget_t *g, uint8_t border_w, uint8_t radius);
```

功能描述	设置分组边框宽度与圆角半径。
输入参数	g：分组；border_w：边框宽度，0 表示不绘制；radius：圆角。
输出参数	无
返回值	无

使用示例：

```
wvui_group_set_border(grp, 1, 6);
```

注意事项：

边框绘制在分组矩形内侧，过宽将挤占子控件可用的可视区域。

4.33.7 wvui_group_set_opa(wvui_widget_t *g, uint8_t opa)

```
void wvui_group_set_opa(wvui_widget_t *g, uint8_t opa);
```

功能描述	设置整组不透明度乘子，作用于分组及其子控件。
输入参数	g：分组；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_group_set_opa(grp, 128);
```

注意事项：

该值会与子控件自身不透明度相乘，用于整体淡入淡出。

4.33.8 wvui_group_child_count(const wvui_widget_t *g)

```
uint16_t wvui_group_child_count(const wvui_widget_t *g);
```

功能描述	读回分组内子控件数量。
输入参数	g：分组。
输出参数	无
返回值	子控件数量。

使用示例：

```
uint16_t n = wvui_group_child_count(grp);
```

注意事项：

仅统计通过 wvui_group_add 加入的直接子控件，嵌套分组内的控件不计入。

4.34 wvui 轮播 API（WVUI_TYPE_SLIDESHOW）

4.34.1 wvui_slideshow_bind(wvui_widget_t *w, const wvui_asset_id_t *ids, uint16_t count)

```
void wvui_slideshow_bind(wvui_widget_t *w, const wvui_asset_id_t *ids, uint16_t count);
```

功能描述	绑定图片资源 ID 数组，不拷贝内容。
输入参数	w：控件；ids：资源 ID 数组；count：图片数量。
输出参数	无
返回值	无

使用示例：

```
static const wvui_asset_id_t ids[] = {IMG_A, IMG_B, IMG_C};
```

```
wvui_slideshow_bind(ss, ids, 3);
```

注意事项：

ID 数组必须常驻，控件存续期间不可释放。

所引用的图片资源必须已注册，否则该帧为空白。

4.34.2 wvui_slideshow_next(wvui_widget_t *w)

```
void wvui_slideshow_next(wvui_widget_t *w);
```

功能描述	切换到下一张，按滑动过渡播放。
输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_slideshow_next(ss);
```

注意事项：

最后一张之后回绕至第一张；过渡未结束时重复调用将重新开始滑动。

3.34.3 wvui_slideshow_set_index(wvui_widget_t *w, uint16_t index)

```
void wvui_slideshow_set_index(wvui_widget_t *w, uint16_t index);
```

功能描述	指定当前显示索引，立即跳转。
输入参数	w：控件；index：图片索引。
输出参数	无
返回值	无

使用示例：

```
wvui_slideshow_set_index(ss, 2);
```

注意事项：

跳转不播放滑动过渡；索引越界时为空操作。

3.34.4 wvui_slideshow_set_timing(wvui_widget_t *w, uint8_t hold_frames, uint8_t slide_frames)

```
void wvui_slideshow_set_timing(wvui_widget_t *w, uint8_t hold_frames, uint8_t slide_frames);
```

功能描述	设置每张停留帧数与滑动过渡帧数。
输入参数	w：控件；hold_frames：停留帧数；slide_frames：过渡帧数。
输出参数	无
返回值	无

使用示例：

```
wvui_slideshow_set_timing(ss, 90, 15);
```

注意事项：

slide_frames 为 0 时即为无过渡的硬切。

3.34.5 wvui_slideshow_set_autoplay(wvui_widget_t *w, uint8_t on)

```
void wvui_slideshow_set_autoplay(wvui_widget_t *w, uint8_t on);
```

功能描述	开启或关闭自动轮播。
输入参数	w：控件；on：非 0 开启。
输出参数	无
返回值	无

使用示例：

```
wvui_slideshow_set_autoplay(ss, 1);
```

注意事项：

按 hold_frames 计时，到期后自动调用 next。

3.34.6 wvui_slideshow_set_colors(wvui_widget_t *w, wvui_color_t bg)

```
void wvui_slideshow_set_colors(wvui_widget_t *w, wvui_color_t bg);
```

功能描述	设置背景色。
输入参数	w：控件；bg：背景色。
输出参数	无
返回值	无

使用示例：

```
wvui_slideshow_set_colors(ss, C_BLACK);
```

注意事项：

背景仅在图片未覆盖的区域可见，图片尺寸小于控件时尤为明显。

3.34.7 wvui_slideshow_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_slideshow_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_slideshow_set_opa(ss, 255);
```

注意事项：

滑动过渡期间新旧两张图同时绘制，半透明设置会使交叠区域出现重影。

3.34.8 wvui_slideshow_count(const wvui_widget_t *w)

```
uint16_t wvui_slideshow_count(const wvui_widget_t *w);
```

功能描述	读回图片数量。
输入参数	w：控件。
输出参数	无
返回值	图片数量。

使用示例：

```
uint16_t n = wvui_slideshow_count(ss);
```

注意事项：

数量由 bind 时传入的 count 决定；返回 0 表示尚未绑定任何资源。

3.34.9 wvui_slideshow_index(const wvui_widget_t *w)

```
uint16_t wvui_slideshow_index(const wvui_widget_t *w);
```

功能描述	读回当前索引。
输入参数	w：控件。
输出参数	无
返回值	当前图片索引。

使用示例：

```
uint16_t idx = wvui_slideshow_index(ss);
```

注意事项：

滑动过渡期间返回的是目标索引，而非屏幕上正在滑出的旧图。

3.34.10 wvui_slideshow_is_playing(const wvui_widget_t *w)

```
uint8_t wvui_slideshow_is_playing(const wvui_widget_t *w);
```

功能描述	查询是否正在播放滑动过渡。
输入参数	w：控件。
输出参数	无
返回值	1 表示播放中。

使用示例：

```
if (wvui_slideshow_is_playing(ss)) { /* 过渡中 */ }
```

注意事项：

slide_frames 设为 0 时该接口恒为 0，可据此判断当前是否为硬切模式。

3.34.11 wvui_slideshow_offset(const wvui_widget_t *w)

```
int16_t wvui_slideshow_offset(const wvui_widget_t *w);
```

功能描述	读回当前滑动偏移。
输入参数	w：控件。
输出参数	无
返回值	偏移像素值。

使用示例：

```
int16_t off = wvui_slideshow_offset(ss);
```

注意事项：

过渡结束或不在播放时该值回到 0，可用于外部同步指示器动画。

3.35 wvui 柱状图 API（WVUI_TYPE_BARCHART）

3.35.1 wvui_barchart_bind(wvui_widget_t *w, const uint8_t *samples, uint16_t count)

```
uint8_t wvui_barchart_bind(wvui_widget_t *w, const uint8_t *samples, uint16_t count);
```

功能描述	绑定柱高样本数组，不拷贝数据，直接引用该缓冲区。
输入参数	w：控件；samples：样本数组首地址；count：柱子数量。
输出参数	无
返回值	1 表示绑定成功，0 表示参数无效。

使用示例：

```
static uint8_t bars[8] = {10, 40, 80, 120, 90, 60, 30, 15};
wvui_barchart_bind(bc, bars, 8);
```

注意事项：

缓冲区必须常驻，控件销毁前不可释放。

3.35.2 wvui_barchart_set(wvui_widget_t *w, uint16_t index, uint8_t value)

```
void wvui_barchart_set(wvui_widget_t *w, uint16_t index, uint8_t value);
```

功能描述	写入指定索引的柱高并触发重绘。
输入参数	w：控件；index：柱子索引；value：柱高，取值 0..255。

输出参数	无
返回值	无

使用示例：

```
wvui_barchart_set(bc, 3, 200);
```

注意事项：

索引越界时为空操作。

3.35.3 wvui_barchart_set_colors(wvui_widget_t *w, wvui_color_t bar, wvui_color_t track)

```
void wvui_barchart_set_colors(wvui_widget_t *w, wvui_color_t bar, wvui_color_t track);
```

功能描述	设置柱体色与轨道底色。
输入参数	w：控件；bar：柱体色；track：轨道色。
输出参数	无
返回值	无

使用示例：

```
wvui_barchart_set_colors(bc, C_BLUE, C_LGRAY);
```

注意事项：

轨道色用于柱底背景；若无需轨道效果，可将其与背景色设为一致。

3.35.4 wvui_barchart_set_base_color(wvui_widget_t *w, wvui_color_t base)

```
void wvui_barchart_set_base_color(wvui_widget_t *w, wvui_color_t base);
```

功能描述	设置基线颜色。
输入参数	w：控件；base：基线色。
输出参数	无
返回值	无

使用示例：

```
wvui_barchart_set_base_color(bc, C_GRAY);
```

注意事项：

基线固定绘制在坐标系零点，颜色建议与柱体保持足够对比。

3.35.5 wvui_barchart_set_bar(wvui_widget_t *w, uint8_t bar_w, uint8_t gap)

```
void wvui_barchart_set_bar(wvui_widget_t *w, uint8_t bar_w, uint8_t gap);
```

功能描述	设置柱宽与柱间间距。

输入参数	w：控件；bar_w：柱宽像素；gap：间距像素。
输出参数	无
返回值	无

使用示例：

```
wvui_barchart_set_bar(bc, 12, 4);
```

注意事项：

柱子总数乘以（柱宽 + 间距）超出控件尺寸时，多余部分将被裁剪。

3.35.6 wvui_barchart_set_radius(wvui_widget_t *w, uint8_t radius)

```
void wvui_barchart_set_radius(wvui_widget_t *w, uint8_t radius);
```

功能描述	设置柱子端部圆角半径。
输入参数	w：控件；radius：像素。
输出参数	无
返回值	无

使用示例：

```
wvui_barchart_set_radius(bc, 3);
```

注意事项：

圆角不应超过柱宽的一半，否则柱子端部将退化为半圆。

3.35.7 wvui_barchart_set_vertical(wvui_widget_t *w, uint8_t vertical)

```
void wvui_barchart_set_vertical(wvui_widget_t *w, uint8_t vertical);
```

功能描述	设置柱子的排列方向。
输入参数	w：控件；vertical：非 0 表示纵向，0 表示横向。
输出参数	无
返回值	无

使用示例：

```
wvui_barchart_set_vertical(bc, 1);
```

注意事项：

切换排列方向后柱体占位随之改变，需重新确认柱宽、间距与控件尺寸是否匹配。

3.35.8 wvui_barchart_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_barchart_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。

输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_barchart_set_opa(bc, 255);
```

注意事项：

该不透明度同时作用于柱体、轨道与基线，用于整体淡出效果。

3.35.9 wvui_barchart_value(const wvui_widget_t *w, uint16_t index)

```
uint8_t wvui_barchart_value(const wvui_widget_t *w, uint16_t index);
```

功能描述	读回指定索引的柱高。
输入参数	w：控件；index：柱子索引。
输出参数	无
返回值	柱高 0..255，索引无效时为 0。

使用示例：

```
uint8_t v = wvui_barchart_value(bc, 0);
```

注意事项：

读取的是绑定缓冲区中的当前值；索引越界返回 0，无法与“柱高为 0”区分。

3.35.10 wvui_barchart_count(const wvui_widget_t *w)

```
uint16_t wvui_barchart_count(const wvui_widget_t *w);
```

功能描述	读回柱子数量。
输入参数	w：控件。
输出参数	无
返回值	柱子数量。

使用示例：

```
uint16_t n = wvui_barchart_count(bc);
```

注意事项：

数量由 bind 时传入的 count 决定，不会因控件尺寸变化而改变。

3.36 wvui 段状进度条 API（WVUI_TYPE_SEGBAR）

3.36.1 wvui_segbar_set_value(wvui_widget_t *w, uint8_t value)

```
void wvui_segbar_set_value(wvui_widget_t *w, uint8_t value);
```

功能描述	设置点亮段所占比例。
输入参数	w：控件；value：取值 0..255，按比例换算为点亮段数。
输出参数	无
返回值	无

使用示例：

```
wvui_segbar_set_value(sb, 160);
```

注意事项：

点亮段数由比例乘以总段数取整得到，分辨率略低于像素级进度条。

3.36.2 wvui_segbar_set_window(wvui_widget_t *w, uint8_t win, uint8_t period_frames)

```
void wvui_segbar_set_window(wvui_widget_t *w, uint8_t win, uint8_t period_frames);
```

功能描述	设置移动高亮窗口的宽度与移动周期。
输入参数	w：控件；win：窗口覆盖段数；period_frames：移动一个周期所需的帧数。
输出参数	无
返回值	无

使用示例：

```
wvui_segbar_set_window(sb, 3, 60);
```

注意事项：

win 为 0 时关闭高亮窗口动画。

3.36.3 wvui_segbar_set_offset(wvui_widget_t *w, uint8_t offset)

```
void wvui_segbar_set_offset(wvui_widget_t *w, uint8_t offset);
```

功能描述	设置段的起始偏移量。
输入参数	w：控件；offset：偏移段数。
输出参数	无
返回值	无

使用示例：

```
wvui_segbar_set_offset(sb, 2);
```

注意事项：

开启高亮窗口动画时该值由动画驱动，手动设置将被覆盖。

3.36.4 wvui_segbar_set_colors(wvui_widget_t *w, wvui_color_t on, wvui_color_t off)

```
void wvui_segbar_set_colors(wvui_widget_t *w, wvui_color_t on, wvui_color_t off);
```

功能描述	设置点亮段色与熄灭段色。
输入参数	w：控件；on：点亮色；off：熄灭色。
输出参数	无
返回值	无

使用示例：

```
wvui_segbar_set_colors(sb, C_GREEN, C_LGRAY);
```

3.36.5 wvui_segbar_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_segbar_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_segbar_set_opa(sb, 255);
```

3.36.6 wvui_segbar_value(const wvui_widget_t *w)

```
uint8_t wvui_segbar_value(const wvui_widget_t *w);
```

功能描述	读回当前比例值。
输入参数	w：控件。
输出参数	无
返回值	0..255。

使用示例：

```
uint8_t v = wvui_segbar_value(sb);
```

注意事项：

返回的是设置的比例原值，而非换算后的点亮段数。

3.36.7 wvui_segbar_window(const wvui_widget_t *w)

```
uint8_t wvui_segbar_window(const wvui_widget_t *w);
```

功能描述	读回高亮窗口宽度。
输入参数	w：控件。
输出参数	无
返回值	窗口覆盖段数。

使用示例：

```
uint8_t win = wvui_segbar_window(sb);
```

注意事项：

返回 0 表示高亮窗口动画处于关闭状态。

3.36.8 wvui_segbar_offset(const wvui_widget_t *w)

```
uint8_t wvui_segbar_offset(const wvui_widget_t *w);
```

功能描述	读回当前段偏移量。
输入参数	w：控件。
输出参数	无
返回值	偏移段数。

使用示例：

```
uint8_t off = wvui_segbar_offset(sb);
```

注意事项：

偏移以段为单位循环，超过总段数时按取模处理。

3.37 wvui 单色位图 API（WVUI_TYPE_MONOIMG）

3.37.1 wvui_monoimg_set_bitmap(wvui_widget_t *w, const uint8_t *bits, int16_t bw, int16_t bh)

```
uint8_t wvui_monoimg_set_bitmap(wvui_widget_t *w, const uint8_t *bits, int16_t bw, int16_t bh);
```

功能描述	绑定 1bpp 位图及其宽高，不拷贝数据。
输入参数	w：控件；bits：位图数据；bw：宽度；bh：高度。
输出参数	无
返回值	1 表示成功，0 表示参数无效。

使用示例：

```
wvui_monoimg_set_bitmap(mi, icon_bits, 24, 24);
```

注意事项：

行内按字节对齐，每行字节数为 (bw+7)/8，数据长度需按该值乘以 bh。

位图必须常驻，控件销毁前不可释放。

3.37.2 wvui_monoimg_set_color(wvui_widget_t *w, wvui_color_t color)

```
void wvui_monoimg_set_color(wvui_widget_t *w, wvui_color_t color);
```

功能描述	设置前景绘制色，即置位像素使用的颜色。
输入参数	w：控件；color：前景色。
输出参数	无
返回值	无

使用示例：

```
wvui_monoimg_set_color(mi, C_BLACK);
```

3.37.3 wvui_monoimg_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_monoimg_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_monoimg_set_opa(mi, 255);
```

注意事项：

置位像素按该不透明度混色，清零像素不绘制，因此不会遮挡背景。

3.37.4 wvui_monoimg_set_invert(wvui_widget_t *w, uint8_t invert)

```
void wvui_monoimg_set_invert(wvui_widget_t *w, uint8_t invert);
```

功能描述	设置是否反色显示。
输入参数	w：控件；invert：非 0 反色。
输出参数	无
返回值	无

使用示例：

```
wvui_monoimg_set_invert(mi, 1);
```

3.37.5 wvui_monoimg_bits(const wvui_widget_t *w)

```
const uint8_t *wvui_monoimg_bits(const wvui_widget_t *w);
```

功能描述	读回位图指针。
输入参数	w：控件。
输出参数	无
返回值	位图指针，未设置时为 NULL。

使用示例：

```
const uint8_t *bits = wvui_monoimg_bits(mi);
```

注意事项：

返回的是绑定时传入的地址，控件不拷贝也不释放，仅供只读访问。

3.37.6 wvui_monoimg_width(const wvui_widget_t *w)

```
int16_t wvui_monoimg_width(const wvui_widget_t *w);
```

功能描述	读回位图宽度。
输入参数	w：控件。
输出参数	无
返回值	宽度像素值。

使用示例：

```
int16_t bw = wvui_monoimg_width(mi);
```

注意事项：

位图宽度不必等于控件宽度，绘制时按控件矩形裁剪。

3.37.7 wvui_monoimg_height(const wvui_widget_t *w)

```
int16_t wvui_monoimg_height(const wvui_widget_t *w);
```

功能描述	读回位图高度。
输入参数	w：控件。
输出参数	无
返回值	高度像素值。

使用示例：

```
int16_t bh = wvui_monoimg_height(mi);
```

注意事项：

位图高度不必等于控件高度；超出部分被裁剪，不足部分留空。

3.38 wvui 图片 API（WVUI_TYPE_IMG）

3.38.1 wvui_img_set_asset(wvui_widget_t *w, wvui_asset_id_t id)

```
void wvui_img_set_asset(wvui_widget_t *w, wvui_asset_id_t id);
```

功能描述	设置图片资源 ID，绘制时按该 ID 打开资源。
输入参数	w：控件；id：图片资源 ID。
输出参数	无
返回值	无

使用示例：

```
wvui_img_set_asset(im, IMG_LOGO);
```

注意事项：

仅保存 ID，资源在绘制时才打开，因此资源须已注册。

3.38.2 wvui_img_set_color(wvui_widget_t *w, wvui_color_t color)

```
void wvui_img_set_color(wvui_widget_t *w, wvui_color_t color);
```

功能描述	设置纯 alpha 格式图片的重着色前景色。
输入参数	w：控件；color：着色。
输出参数	无
返回值	无

使用示例：

```
wvui_img_set_color(im, C_RED);
```

注意事项：

仅对纯 alpha（遮罩）格式生效，彩色图片将忽略该设置。

3.38.3 wvui_img_set_opa(wvui_widget_t *w, uint8_t opa)

```
void wvui_img_set_opa(wvui_widget_t *w, uint8_t opa);
```

功能描述	设置整体不透明度。
输入参数	w：控件；opa：0..255。
输出参数	无
返回值	无

使用示例：

```
wvui_img_set_opa(im, 200);
```

注意事项：

不透明度与资源自身的 alpha 相乘，取 128 附近可做半透明叠加效果。

3.38.4 wvui_img_asset(const wvui_widget_t *w)

```
wvui_asset_id_t wvui_img_asset(const wvui_widget_t *w);
```

功能描述	读回当前资源 ID。
输入参数	w：控件。
输出参数	无
返回值	资源 ID。

使用示例：

```
wvui_asset_id_t id = wvui_img_asset(im);
```

注意事项：

返回值为设置时的 ID 副本，并不表示该资源当前是否打开成功。

3.38.5 wvui_img_opa(const wvui_widget_t *w)

```
uint8_t wvui_img_opa(const wvui_widget_t *w);
```

功能描述	读回不透明度。
输入参数	w：控件。
输出参数	无
返回值	0..255。

使用示例：

```
uint8_t o = wvui_img_opa(im);
```

注意事项：

该值为控件级不透明度，渲染时还会与分组不透明度相乘。

3.39 wvui 滚动容器 API（WVUI_TYPE_SCROLL_PANEL）

3.39.1 wvui_scroll_panel_refresh(wvui_widget_t *w)

```
void wvui_scroll_panel_refresh(wvui_widget_t *w);
```

功能描述	按子控件占据的范围重新计算可滚动区间，并标记重绘。
输入参数	w：控件。
输出参数	无
返回值	无

使用示例：

```
wvui_scroll_panel_refresh(sp);
```

注意事项：

新增、删除或移动子控件后必须调用，否则可滚动范围仍为旧值。

内容高度不超过控件高度时滚动区间为零，面板不可滚动。

3.39.2 wvui_scroll_panel_set_scroll(wvui_widget_t *w, int32_t pos)

```
void wvui_scroll_panel_set_scroll(wvui_widget_t *w, int32_t pos);
```

功能描述	设置滚动位置，超出可滚区间时自动夹紧。
输入参数	w：控件；pos：目标滚动位置，像素。
输出参数	无
返回值	无

使用示例：

```
wvui_scroll_panel_set_scroll(sp, 0);
```

注意事项：

正值向下滚动内容；夹紧边界由 refresh 计算出的区间决定。

3.39.3 wvui_scroll_panel_scroll(const wvui_widget_t *w)

```
int32_t wvui_scroll_panel_scroll(const wvui_widget_t *w);
```

功能描述	读回当前滚动位置。
输入参数	w：控件。
输出参数	无
返回值	当前滚动位移，像素。

使用示例：

```
int32_t p = wvui_scroll_panel_scroll(sp);
```

注意事项：

该值在拖动或调用 set_scroll 后更新；只读调用本身不触发重绘。

4 移植接口

4.1 显示端口 wvui_port_t

成员	原型	说明
set_window	void (*)(void *ctx, wvui_rect_t r)	阶段 1：设置本次事务的矩形窗口。
start_transfer	void (*)(void *ctx)	阶段 2：启动一次像素传输。
write_pixels	void (*)(void *ctx, const void *buf, uint32_t px)	阶段 3：送出一块像素数据。
wait	void (*)(void *ctx)	阶段 4：等待在途传输完成。
finish	void (*)(void *ctx)	阶段 5：结束一次事务。

4.2 输入端口 wvui_input_port_t

成员	原型	说明
read	void (*)(void *ctx, wvui_indev_t *out)	读取当前触摸状态并填充坐标与按下标志，事件合成由输入层完成。

4.3 自定义控件

回调	原型	说明
draw	void (*)(wvui_widget_t *w)	纯绘制回调，每个切片调用一次，不得标脏或修改对象树。
event	uint8_t (*)(wvui_widget_t *w, wvui_event_t e, wvui_indev_t *d)	事件处理回调，非 0 表示已消费；禁止在此删除自身或祖先。
init	void (*)(wvui_widget_t *w)	创建时调用一次，按描述填默认值。
set_pos	void (*)(wvui_widget_t *w, int16_t x, int16_t y)	几何变化钩子，只需标脏自身。
on_delete	void (*)(wvui_widget_t *w)	摘链前自清理，禁止在此调用删除接口。
set_prop / get_prop	int8_t (*)(w, uint8_t prop, const uint8_t *in, uint16_t len)int16_t (*)(w, uint8_t prop, uint8_t *out, uint16_t cap)	属性读写；负载按小端序解释，返回 0 表示成功，或返回已写入字节数。

5 附录

5.1 控件类型号

类型号	宏	说明
1	WVUI_TYPE_LABEL	文本标签
2	WVUI_TYPE_BTN	按钮
3	WVUI_TYPE_SCROLL_PANEL	单轴滚动容器
4	WVUI_TYPE_IMG	图片
5	WVUI_TYPE_LINE	线段
6	WVUI_TYPE_PROGRESS	进度条
7	WVUI_TYPE_TOGGLE	开关
8	WVUI_TYPE_CHECKBOX	复选框
9	WVUI_TYPE_INDICATOR	指示灯
10	WVUI_TYPE_BOX	四角可配盒子
11	WVUI_TYPE_ARC	圆弧
12	WVUI_TYPE_SLIDER	滑块
13	WVUI_TYPE_GAUGE	仪表
14	WVUI_TYPE_CHART	折线图
15	WVUI_TYPE_STEPPER	步进器
16	WVUI_TYPE_LIST	文本列表
17	WVUI_TYPE_ROLLER	滚轮选择器
18	WVUI_TYPE_TOAST	浮动提示
19	WVUI_TYPE_MSGBOX	模态对话框
20	WVUI_TYPE_DROPDOWN	下拉选择
21	WVUI_TYPE_SEGDISP	数码管显示
22	WVUI_TYPE_MARQUEE	跑马灯

23	WVUI_TYPE_LABEL_EX	自动换行标签
24	WVUI_TYPE_GROUP	分组容器
25	WVUI_TYPE_SLIDESHOW	轮播
27	WVUI_TYPE_BARCHART	柱状图
28	WVUI_TYPE_SEGBAR	段状进度条
29	WVUI_TYPE_MONOIMG	单色位图

5.2 修订记录

版本	日期	说明
V1.0	2026-11-12	首次生成。